

УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ
ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

Дата генерации: 11.09.2026, 18:35:25
Файлов исходного кода: 82
Суммарный объём кода: 1.11 МВ
Глав/билетов в пособии: 30

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

1. 1. Дерево отрезков с операциями на отрезках [id: segment-tree]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: prefix-sums]
6. Планарность: K5, K3,3 и формула Эйлера [id: planarity]
7. 6. Графы. Представление, DFS, BFS [id: graph-basics]
8. 7. Графы. Планарные. Покраска [id: graph-planarity]
9. 8. Графы. Компоненты связности [id: graph-components]
10. 9. Графы. Топологическая сортировка [id: topological-sort]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 12. Графы. Точки сочленения [id: graph-articulation-points]
14. 13. Графы. Эйлеров цикл [id: graph-euler]
15. Билет 14. Дейкстра [id: dijkstra]
16. Билет 15. Форд-Беллман [id: bellman-ford]
17. Билет 16. Флойд-Уоршелл [id: floyd]
18. 17. Графы. Ускорения поиска [id: pathfinding-optimizations]
19. 17. Графы. Алгоритм Джонсона (Фокус с перевзвешиванием)
20. 18. Графы. Остовное дерево. Краскал [id: mst-kruskal]
21. 19. Графы. Остовное дерево. Прима [id: mst-prims]
22. 20. Графы. Остовное дерево. Борувка [id: mst-boruvka]
23. 21. Алгоритм Кнута-Морриса-Пракса (KMP) [id: string-kmp]
24. 22. Z-функция [id: string-z-function]
25. Билет 23. Алгоритм Ахо-Корасика [id: aho-corasick]

Билеты (учебный контент)

- 23. src/data/tickets/ticket1_5.ts
- 24. src/data/tickets/ticket14_20.ts
- 25. src/data/tickets/ticket21_24.ts
- 26. src/data/tickets/ticket6_13.ts

Интерактивные компоненты и симуляторы

- 27. src/components/ArticulationPointsViz.tsx
- 28. src/components/BellmanFordViz.tsx
- 29. src/components/BfsViz.tsx
- 30. src/components/ChapterImage.tsx
- 31. src/components/ChapterNav.tsx
- 32. src/components/ChapterView.tsx
- 33. src/components/DfsBridgesSimulator.tsx
- 34. src/components/DijkstraViz.tsx
- 35. src/components/DPVisualizer.tsx
- 36. src/components/EulerSimulator.tsx
- 37. src/components/ExpressQuiz.tsx
- 38. src/components/FloydViz.tsx
- 39. src/components/GraphTraversalViz.tsx
- 40. src/components/HeapViz.tsx
- 41. src/components/hints/demoKit.tsx
- 42. src/components/hints/demosExtra.tsx
- 43. src/components/hints/demosGraph.tsx
- 44. src/components/hints/demosStruct.tsx
- 45. src/components/hints/MiniDemo.tsx
- 46. src/components/hints/SimulatorModal.tsx
- 47. src/components/hints/TermPopover.tsx
- 48. src/components/JohnsonViz.tsx
- 49. src/components/KosarajuViz.tsx
- 50. src/components/KruskalSimulator.tsx
- 51. src/components/MnemonicCards.tsx
- 52. src/components/MobileToc.tsx
- 53. src/components/Navbar.tsx
- 54. src/components/PlanarityDemo.tsx
- 55. src/components/QueueViz.tsx
- 56. src/components/SalmonAutomatonWidget.tsx
- 57. src/components/SegmentTreeVisualizer.tsx

РАЗДЕЛ: КОНФИГУРАЦИЯ ПРОЕКТА

ФАЙЛ 1/82: add.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 1224 | РАЗМЕР:

Изменённые файлы

```text

```
A .github/workflows/deploy-pages.yml
M .github/workflows/rebuild-pdf.yml
M index.html
M public/export/full_code_all_pages.pdf
M public/export/full_code_all_pages.txt
A public/favicon.svg
M src/App.tsx
M src/components/Navbar.tsx
A src/components/PythonCompiler.tsx
M vite.config.ts
```

```

`github/workflows/deploy-pages.yml`

Полное содержимое после изменений

```yaml

```
name: Deploy app to GitHub Pages
```

```
on:
```

```
 push:
```

```
 branches:
```

```
 - main
```

```
 # Позволяет проверить Pages прямо из рабочей ветки
```

```
 - "arena/**"
```

```
 workflow_dispatch:
```

```
deploy:
 name: Publish to GitHub Pages
 needs: build
 runs-on: ubuntu-latest
 environment:
 name: github-pages
 url: ${ steps.deployment.outputs.page_url }
 steps:
 - name: Deploy
 id: deployment
 uses: actions/deploy-pages@v5
....

Patch относительно `main`

```diff
diff --git a/.github/workflows/deploy-pages.yml b/.github/workflows/deploy-pages.yml
new file mode 100644
index 00000000..17418e3
--- /dev/null
+++ b/.github/workflows/deploy-pages.yml
@@ -0,0 +1,60 @@
+name: Deploy app to GitHub Pages
+
+on:
+  push:
+    branches:
+      - main
+      # Позволяет проверить Pages прямо из рабочей ветки
+      - "arena/**"
+  workflow_dispatch:
+
+permissions:
+  contents: read
+  pages: write
+  id-token: write
+
+env:
+  CI: false
```

Универсальное пособие • полный исходный код

```
-----
+   environment:
+     name: github-pages
+     url: ${ steps.deployment.outputs.page_url }
+   steps:
+     - name: Deploy
+       id: deployment
+       uses: actions/deploy-pages@v5
+....
```

`github/workflows/rebuild-pdf.yml`

Полное содержимое после изменений

```yaml

name: Rebuild code PDF

# Пайплайн пересборки PDF при каждом коммите:

```
#
исходный код → full_code_all_pages.txt (script)
|
|→ page-0001.png ... page-NNNN.png (
|
|→ full_code_all_page
```

# Готовые TXT и PDF коммитаются обратно в ветку (их отда  
# промежуточные PNG сохраняются как artifacts запуска.

on:

push:

branches:

- "\*\*"

paths-ignore:

# чтобы коммит самого workflow не запускал бескон

- "public/export/\*\*"

- "\*\*/\*.md"

workflow\_dispatch:

inputs:

density:

- ```
        if [ -f "$policy" ]; then
            sudo sed -i 's/rights="none" pattern="\{P
        fi
    done
    convert -version
```
- name: Install dependencies
run: npm ci --no-audit --no-fund
 - name: Step 1 – код → txt
run: npm run export:txt
 - name: Step 2 – txt → png
env:
EXPORT_DENSITY: \${github.event.inputs.density}
run: npm run export:png
 - name: Step 3 – png → pdf
run: npm run export:pdf
 - name: Type-check приложения
run: npx tsc --noEmit
continue-on-error: true
 - name: Summary
run: |
{
 echo "### Экспорт пересобран"
 echo ""
 echo "| Артефакт | Размер |"
 echo "| --- | --- |"
 echo "| \public/export/full_code_all_pages
 echo "| \public/export/full_code_all_pages
 echo "| PNG-страниц | \$(ls tmp/export-pages.
} >> "\$GITHUB_STEP_SUMMARY"
 - name: Upload artifacts (PDF + TXT)
uses: actions/upload-artifact@v7

```
    steps:
      - name: Checkout
      - uses: actions/checkout@v4
      + uses: actions/checkout@v6
        with:
          fetch-depth: 0
          persist-credentials: true

      - name: Setup Node.js
      - uses: actions/setup-node@v4
      + uses: actions/setup-node@v7
        with:
          - node-version: "22"
          + node-version: "24"
            cache: npm

      - name: Install ImageMagick + DejaVu fonts
@@ -97,7 +97,7 @@ jobs:
        } >> "$GITHUB_STEP_SUMMARY"

      - name: Upload artifacts (PDF + TXT)
      - uses: actions/upload-artifact@v4
      + uses: actions/upload-artifact@v7
        with:
          name: full-code-export
          path: |
@@ -107,7 +107,7 @@ jobs:
          retention-days: 30

      - name: Upload artifacts (PNG-страницы)
      - uses: actions/upload-artifact@v4
      + uses: actions/upload-artifact@v7
        with:
          name: full-code-pages-png
          path: tmp/export-pages/*.png
    ....

## `index.html`
```

Универсальное пособие • полный исходный код

Полное содержимое после изменений

````xml

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64"
 <rect width="64" height="64" rx="16" fill="#4f46e5"/>
 <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#4f46e5"/>
 <circle cx="47" cy="21" r="4" fill="#34d399"/>
</svg>
```

````

Patch относительно `main`

````diff

```
diff --git a/public/favicon.svg b/public/favicon.svg
```

```
new file mode 100644
```

```
index 00000000..3bbbba0
```

```
--- /dev/null
```

```
+++ b/public/favicon.svg
```

```
@@ -0,0 +1,5 @@
```

```
+<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 64 64">
```

```
+ <rect width="64" height="64" rx="16" fill="#4f46e5"/>
```

```
+ <path d="M18 17h28v8H26v8h16v8H26v8h20v8H18z" fill="#4f46e5"/>
```

```
+ <circle cx="47" cy="21" r="4" fill="#34d399"/>
```

```
+</svg>
```

````

`src/App.tsx`

Полное содержимое после изменений

````tsx

```
import { useCallback, useEffect, useMemo, useState } from 'react';
```

```
import { chapters } from '../data/content';
```

```
import { Navbar } from '../components/Navbar';
```

```
import { Sidebar } from '../components/Sidebar';
```

```
import { MobileToc } from '../components/MobileToc';
```

```
import { ChapterView } from '../components/ChapterView';
```

```
import { ChapterNav } from '../components/ChapterNav';
```



```

<div className="min-h-screen bg-slate-950 text-slate-50" >
 <Navbar activeTab={activeTab} setActiveTab={setActiveTab} />

 {activeTab === "guide" ? (
 <>
 <MobileToc
 selectedChapterId={activeChapterId}
 setSelectedChapterId={goTo}
 currentTitle={activeChapter.title}
 index={index}
 total={chapters.length}
 />

 <div className="flex flex-col lg:flex-row max-w-7xl mx-auto" >
 {/* Сайдбар только на десктопе – на мобильном скрывается */}
 <div className="hidden lg:block lg:w-80 shrink-0" >
 <Sidebar selectedChapterId={activeChapterId} />
 </div>

 <main id="main-content" className="flex-1 m-0 p-4" >
 {/* Шапка главы с быстрыми переходами */}
 <div className="flex items-center justify-between" >
 <div className="min-w-0" >
 <div className="text-[10px] uppercase" >
 {activeChapter.category || "Универсальное пособие"}
 </div>
 <h2 className="text-base sm:text-xl font-medium" >
 {activeChapter.title}
 </h2>
 </div>
 <ChapterNav prev={prev} next={next} index={index} />
 </div>

 <div className="h-1 w-full bg-slate-800 rounded" >
 <div
 className="h-full bg-gradient-to-r from-slate-800 to-slate-700"
 style={{ width: `${progress}%` }}
 />

```

### Patch относительно `main`

```diff

diff --git a/src/App.tsx b/src/App.tsx

index 2091f33..4583756 100644

--- a/src/App.tsx

+++ b/src/App.tsx

@@ -7,9 +7,10 @@ import { ChapterView } from "../components

import { ChapterNav } from "../components/ChapterNav";

import { SimulatorModal } from "../components/hints/Sim

import { SimulatorHub } from "../components/SimulatorHub

+import { PythonCompiler } from "../components/PythonComp

export default function App() {

- const [activeTab, setActiveTab] = useState<"guide" |

+ const [activeTab, setActiveTab] = useState<"guide" |

const [activeChapterId, setActiveChapterId] = useSta

const [modalVizId, setModalVizId] = useState<string

@@ -95,7 +96,7 @@ export default function App() {

</main>

</div>

</>

-) : (

+) : activeTab === "simulator" ? (

<main className="px-4 sm:px-6 lg:px-10 py-6 lg

<SimulatorHub

onOpen={setModalVizId}

@@ -105,6 +106,8 @@ export default function App() {

}}

/>

</main>

+) : (

+ <PythonCompiler />

}}

<SimulatorModal vizId={modalVizId} onClose={() =>

```
    <div className="min-w-0 flex-1">
      <h1 className="text-xl font-extrabold text-slate-800">
        Универсальное пособие
      </h1>
      <p className="text-xs text-indigo-400 font-extrabold">
        Подготовка к экзаменам: Алгоритмы, Структуры
      </p>
    </div>
  </div>
```

```
<div className="flex items-center w-full lg:w-auto" >
  >
  <button
    id="tab-guide-btn"
    onClick={() => setActiveTab('guide')}
    className={`flex-1 lg:flex-none flex items-center justify-center text-center font-extrabold cursor-pointer duration-200 whitespace-nowrap ${
      activeTab === 'guide'
        ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
        : 'text-slate-400 hover:text-white'
    }`}
  >
    <BookOpen className="w-4 h-4" /> Учебное пособие
  </button>
  <button
    id="tab-simulator-btn"
    onClick={() => setActiveTab('simulator')}
    className={`flex-1 lg:flex-none flex items-center justify-center text-center font-extrabold cursor-pointer duration-200 whitespace-nowrap ${
      activeTab === 'simulator'
        ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
        : 'text-slate-400 hover:text-white'
    }`}
  >
    <Cpu className="w-4 h-4" /> Тренажёры
  </button>
  <button
    id="tab-compiler-btn"
    onClick={() => setActiveTab('compiler')}
    className={`flex-1 lg:flex-none flex items-center justify-center text-center font-extrabold cursor-pointer duration-200 whitespace-nowrap ${
      activeTab === 'compiler'
        ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
        : 'text-slate-400 hover:text-white'
    }`}
  >
    <Code className="w-4 h-4" /> Компилятор
  </button>
  <button
    id="tab-about-btn"
    onClick={() => setActiveTab('about')}
    className={`flex-1 lg:flex-none flex items-center justify-center text-center font-extrabold cursor-pointer duration-200 whitespace-nowrap ${
      activeTab === 'about'
        ? 'bg-indigo-600 text-white shadow-lg shadow-indigo-500'
        : 'text-slate-400 hover:text-white'
    }`}
  >
    <Info className="w-4 h-4" /> О проекте
  </button>
</div>
```

```
-----
        className="flex items-center justify-center
        er:bg-amber-600/20 border border-amber-500/
        title="Скачать всё пособие одним автономным
    >
        {busy ? <Loader2 className="w-4 h-4 animate
    </button>
    </div>
  </div>
</header>
);
};
},

```

Patch относительно `main`

```
````diff
diff --git a/src/components/Navbar.tsx b/src/components
index a528778..6f8cd36 100644
--- a/src/components/Navbar.tsx
+++ b/src/components/Navbar.tsx
@@ -1,11 +1,11 @@
 import React, { useState } from 'react';
- import { BookOpen, Cpu, Sparkles, FileDown, FileText,
+ import { BookOpen, Cpu, Sparkles, FileDown, FileText,
 import { chapters } from '../data/content';
 import { downloadBookHtml } from '../utils/exportHtml'

 interface NavbarProps {
- activeTab: 'guide' | 'simulator';
- setActiveTab: (tab: 'guide' | 'simulator') => void;
+ activeTab: 'guide' | 'simulator' | 'compiler';
+ setActiveTab: (tab: 'guide' | 'simulator' | 'compile
 }

 export const Navbar: React.FC<NavbarProps> = ({ active
@@ -60,9 +60,20 @@ export const Navbar: React.FC<Navbar
 >
 <Cpu className="w-4 h-4" /> Тренажёры

```

```
const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v0.27.7/index.js`;
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v0.27.7/index.html`;

const STARTER_CODE = `# Первый запуск загрузит Python в
name = "алгоритмы"

for number in range(1, 4):
 print(f"{number}. Учим {name}!")
`;

type PyodideRuntime = {
 runPythonAsync: (source: string) => Promise<unknown>;
 setStdout: (options: { batched: (message: string) => void }) => void;
 setStderr: (options: { batched: (message: string) => void }) => void;
 setStdin: (options: { stdin: () => string | number | null }) => void;
};

type PyodideModule = {
 loadPyodide: (options: { indexURL: string }) => Promise<PyodideRuntime>;
};

let runtimePromise: Promise<PyodideRuntime> | null = null;

function getPythonRuntime() {
 if (!runtimePromise) {
 runtimePromise = import(/* @vite-ignore */ PYODIDE_MODULE_URL)
 .then((module as PyodideModule).loadPyodide({ indexURL: PYODIDE_INDEX_URL }));
 }
 return runtimePromise;
}

function errorText(error: unknown) {
 if (error instanceof Error) return error.message;
 return String(error);
}
```

```

 }
 }, [code, stdin, running, runtimeReady]);

const reset = () => {
 setCode(STARTER_CODE);
 setStdin("");
 setOutput("Нажмите «Запустить», чтобы увидеть резул
};

return (
 <main className="max-w-screen-2xl mx-auto px-4 sm:px-6">
 <div className="max-w-6xl mx-auto">
 <div className="flex flex-col sm:flex-row sm:items-center">
 <div>
 <div className="inline-flex items-center gap-2">
 <Terminal className="w-4 h-4" /> Боковая панель
 </div>
 <h2 className="text-2xl sm:text-3xl font-extrabold">
 <p className="text-slate-400 mt-2 max-w-2xl">
 Пишите небольшой код и запускайте его прямо в браузере,
 который переводит CPython в WebAssembly.
 </p>
 </div>
 <a
 href="https://pyodide.org/"
 target="_blank"
 rel="noreferrer"
 className="inline-flex items-center gap-1.5"
 >
 Как это работает <ExternalLink className="w-4 h-4" />

 </div>

 <div className="grid xl:grid-cols-[minmax(0,1.33fr),minmax(0,1.33fr)]">
 <section className="rounded-2xl border border-slate-200 p-4">
 <div className="flex flex-wrap items-center gap-2">
 <div className="flex items-center gap-2">


```

```

 placeholder="Напишите Python-код..."
 />

 <div className="border-t border-slate-800 border"
 <label className="block px-4 pt-3 text-slate-500"
 Ввод для input() • по одной строке на клавишу
 </label>
 <textarea
 id="python-stdin"
 value={stdin}
 onChange={(event) => setStdin(event.target.value)}
 spellCheck={false}
 rows={2}
 placeholder={'Например:\nАлиса'}
 className="block w-full resize-y bg-transparent border"
 style={{border: '1px solid #333'}}
 />
 </div>
 </section>

 <section className="rounded-2xl border border-slate-800 border"
 <div className="flex items-center justify-between"
 <div className="flex items-center gap-2 text-slate-500"
 <Terminal className="w-4 h-4 text-emerald-500"
 />
 <span className={`inline-flex items-center gap-2`}
 {runtimeReady && <CheckCircle2 className="w-4 h-4 text-emerald-500"
 />
 {runtimeReady ? "готов" : "не загружен"}

 </div>
 <pre className="min-h-[360px] max-h-[560px] border border-slate-800 p-4"
 x] leading-6 text-slate-300">
 {output}
 </pre>
 <div className="border-t border-slate-800 border"
 </section>
</div>

```

```

+
+for number in range(1, 4):
+ print(f"{number}. Учим {name}!")
+`;
+
+type PyodideRuntime = {
+ runPythonAsync: (source: string) => Promise<unknown>
+ setStdout: (options: { batched: (message: string) =>
+ setStderr: (options: { batched: (message: string) =>
+ setStdin: (options: { stdin: () => string | number |
+});
+
+type PyodideModule = {
+ loadPyodide: (options: { indexURL: string }) => Prom
+};
+
+let runtimePromise: Promise<PyodideRuntime> | null = n
+
+function getPythonRuntime() {
+ if (!runtimePromise) {
+ runtimePromise = import(/* @vite-ignore */ PYODIDE
+ (module as PyodideModule).loadPyodide({ indexURL
+ });
+ }
+ return runtimePromise;
+}
+
+function errorText(error: unknown) {
+ if (error instanceof Error) return error.message;
+ return String(error);
+}
+
+export function PythonCompiler() {
+ const [code, setCode] = useState(STARTER_CODE);
+ const [stdin, setStdin] = useState("");
+ const [output, setOutput] = useState("Нажмите «Запус
+ const [running, setRunning] = useState(false);
+ const [runtimeReady, setRuntimeReady] = useState(fal

```



```

+ };
+
+ return (
+ <main className="max-w-screen-2xl mx-auto px-4 sm:px-6">
+ <div className="max-w-6xl mx-auto">
+ <div className="flex flex-col sm:flex-row sm:items-center">
+ <div>
+ <div className="inline-flex items-center gap-2">
+ <Terminal className="w-4 h-4" /> Боковая панель
+ </div>
+ <h2 className="text-2xl sm:text-3xl font-extrabold">
+ <p className="text-slate-400 mt-2 max-w-2xl">
+ Пишите небольшой код и запускайте его прямо в браузере,
+ который переводит CPython в WebAssembly.
+ </p>
+ </div>
+ <a
+ href="https://pyodide.org/"
+ target="_blank"
+ rel="noreferrer"
+ className="inline-flex items-center gap-1.5"
+ >
+ Как это работает <ExternalLink className="text-slate-400" />
+
+ </div>
+
+ <div className="grid xl:grid-cols-[minmax(0,1fr),minmax(0,1fr)]">
+ <section className="rounded-2xl border border-slate-200 p-4">
+ <div className="flex flex-wrap items-center gap-2">
+ <div className="flex items-center gap-2">
+
+
+
+ </div>
+ <div className="flex items-center gap-2">
+ <button
+ type="button"
+ onClick={reset}

```

```

+ <textarea
+ id="python-stdin"
+ value={stdin}
+ onChange={(event) => setStdin(event.target.value)}
+ spellCheck={false}
+ rows={2}
+ placeholder={'Например:\nАлиса'}
+ className="block w-full resize-y bg-transparent border-slate-800 p-2"
+ />
+ </div>
+ </section>
+
+ <section className="rounded-2xl border border-slate-800 p-4">
+ <div className="flex items-center justify-between">
+ <div className="flex items-center gap-2">
+ <Terminal className="w-4 h-4 text-emerald-500 font-mono" />
+ </div>
+ <span className={`inline-flex items-center gap-2`}
+ {runtimeReady && <CheckCircle2 className="w-4 h-4 text-emerald-500" />
+ {runtimeReady ? "готов" : "не загружен"}}
+
+ </div>
+ <pre className="min-h-[360px] max-h-[560px] leading-6 text-slate-300">
+ {output}
+ </pre>
+ <div className="border-t border-slate-800 p-4">
+ </div>
+ </section>
+ </div>
+
+ <div className="mt-5 grid md:grid-cols-3 gap-3">
+ <div className="flex gap-2 rounded-xl border border-slate-800 p-4">
+ <Info className="w-4 h-4 shrink-0 text-indigo-500 font-mono" />
+ Ctrl или Cmd + Enter запускает код. L
+ </div>
+ <div className="flex gap-2 rounded-xl border border-slate-800 p-4">
+ <Info className="w-4 h-4 shrink-0 text-amber-500 font-mono" />

```

```
 },
 server: {
 host: "0.0.0.0",
 port: 5173,
 strictPort: true,
 // предпросмотр может открываться через внешний про
 allowedHosts: true,
 },
 preview: {
 host: "0.0.0.0",
 port: 4173,
 strictPort: true,
 allowedHosts: true,
 },
 },
});
\\
```

### Patch относительно `main`

```
```diff
diff --git a/vite.config.ts b/vite.config.ts
index c0456e7..e19d945 100644
--- a/vite.config.ts
+++ b/vite.config.ts
@@ -10,6 +10,9 @@ const __dirname = path.dirname(__file)

// https://vite.dev/config/
export default defineConfig({
+ // На GitHub Pages приложение живёт в /algv0/, локал
+ // VITE_BASE можно переопределить, если репозиторий
+ base: process.env.VITE_BASE || "/",
  plugins: [react(), tailwindcss(), viteSingleFile()],
  resolve: {
    alias: {
  },
}
```

Универсальное пособие • полный исходный код

- Обязательная сетка аналогий (2 колонки: неформальный и формальный)
 - "Душные" детали (код, формулы, сложные доказательства)
4. ****Не ломай верстку:**** Не придумывай свои CSS-классы и классы элементов. Используй только классы в файлах `src/data/content.ts`. Не используй чистый markdown.

ФАЙЛ 4/82: index.html

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 13 | РАЗМЕР: 512 байт

```
<!doctype html>
<html lang="ru" class="dark bg-slate-950 text-slate-100">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, height=device-height, initial-scale=1.0" />
    <title> Дискретка для тупых – Интерактивный гид по дискретной математике
  </head>
  <body class="bg-slate-950 text-slate-100 min-h-screen">
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```

ФАЙЛ 5/82: metadata.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 8 | РАЗМЕР: 312 байт

```
{
  "name": "Remix Remix: ExamPrep Reader",
  "description": "Удобная web-читалка для подготовки к экзаменам",
  "requestFramePermissions": [],
  "majorCapabilities": [
    "MAJOR_CAPABILITY_SERVER_SIDE_GEMINI_API"
  ]
}
```

Универсальное пособие • полный исходный код

```
-----  
    "@vitejs/plugin-react": "5.1.1",  
    "esbuild": "^0.28.2",  
    "tailwindcss": "4.1.17",  
    "typescript": "5.9.3",  
    "vite": "7.3.2",  
    "vite-plugin-singlefile": "2.3.0"  
  },  
  "description": "Интерактивное пособие по алгоритмам и  
}
```

ФАЙЛ 7/82: README.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 115 | РАЗМЕР:

algv0 – Универсальное пособие по алгоритмам и структуре

Интерактивная web-читалка (React + Vite + Tailwind) с
и автоматическим экспортом всего исходного кода в PDF.

Быстрый старт

```
```bash  
npm install
npm run dev # предпросмотр на http://0.0.0.0:5
npm run build # сборка в dist/ (single-file)
```
```

Экспорт кода: код → txt → png → pdf

Пайплайн собирает ****весь**** исходный код репозитория в о

| Команда | Шаг | Результат |
|----------------------|-----------|---------------------|
| `npm run export:txt` | код → txt | `public/export/ful` |
| `npm run export:png` | txt → png | `tmp/export-pages/` |
| `npm run export:pdf` | png → pdf | `public/export/ful` |

Универсальное пособие • полный исходный код

известные термины (BFS, куча, бор, суффиксная ссылка, подчёркиваются, а по наведению/тапу показывается карта мини-анимацией и кнопкой «Показать симулятор» (симулятор).
Словарь — `src/data/glossary.ts`, мини-анимации — `src/anim`
* **Реестр интерактивов** — `src/components/vizRegistry`
и для панели под главой, и для модальки из подсказки.

Структура

...

| | |
|--------------------|---|
| src/ | |
| App.tsx | — оболочка, привязка глав к визуализации |
| components/ | — интерактивные симуляторы (Действия) |
| data/ | — контент пособия (главы/билеты) |
| scripts/export/ | — пайплайн код → txt → png → pdf |
| public/export/ | — сгенерированные TXT и PDF (обновляемые) |
| .github/workflows/ | — автоматизация |
| ... | |

Структура приложения и парсинг элементов

Если вы хотите парсить HTML конспект внешними инструментами, используйте следующие теги и классы.

Заголовки

- * Заголовки секций: Используют стандартный тег `

` и классы `Section` и `SectionTitle`.
- * Подзаголовки: Используют тег `

`.

Текст и параграфы

- * Обычный текст содержится в тегах `

`.
- * Блоки "Шиза" (Аналогии) и другая текстовая инфографика: Используют тег `

` и классы `Text` и `TextBlock`.

Математические формулы

В приложении используется MathJax.

- * В самом исходном коде страницы (и внутри тегов) формулы

Уровень 1. Нет вообще ничего: ни аналогии, ни 2D

| | | |
|--------------------|-----------|----------------------|
| Глава | id | Что делать |
| --- | --- | --- |
| Алгоритмы на карте | `alg-map` | Страница-заглушка (8 |

Уровень 2. Темы, которых в пособии НЕТ ВООБЩЕ (дыры)

Это самая большая проблема: визуализаторы для них написаны, но соответствующих глав в `src/data/content.ts` нет – и

| | | |
|---------------|--------------------------------|-----------------------------|
| Билет | Тема | Статус |
| --- | --- | --- |
| **1** | Дерево отрезков (Segment Tree) | Главы нет. К |
| | ит вхолостую. | |
| **7** | Планарность и раскраска графов | Номерной глав |
| | огии**. | |
| **11** | Мосты в графах | Номерной главы нет; есть бе |
| **13** | Эйлеровы пути и циклы | Номерной главы нет; |

Дополнительно – «осиротевшие» визуализаторы без глав (к

- * `stack-dfs` → `StackViz.tsx` – ****Стек и DFS****
- * `queue-bfs` → `QueueViz.tsx` + `BfsViz.tsx` – ****Очередь и BFS****
- * `heap-beam-search` → `HeapViz.tsx` – ****Куча и лучевой поиск****
- * `segment-trees` → `SegmentTreeVisualizer.tsx` – ****Дерево отрезков****
- * `dynamic-programming` → `DPVisualizer.tsx` – ****Динамическое программирование****

Уровень 3. Есть 2D, но НЕТ бытовой аналогии («суха

| | | |
|--|--------------|-------------|
| Глава | id | Комментарий |
| --- | --- | --- |
| Планарность: K5, K3,3 и формула Эйлера | `planarity-e | |

Универсальное пособие • полный исходный код

| | | | | |
|----|--------------------------------------|---|---|---|
| 17 | 17. Графы. Алгоритм Джонсона | - | - | - |
| 18 | 18. Графы. Остовное дерево. Краскал | 1 | - | - |
| 19 | 19. Графы. Остовное дерево. Прима | 1 | - | - |
| 20 | 20. Графы. Остовное дерево. Борувка | 1 | - | - |
| 21 | 21. Префикс-функция (π), КМП | 4 | - | - |
| 22 | 22. Z-функция | 4 | - | - |
| 23 | 23. Алгоритм Ахо–Корасик | 2 | - | - |
| 24 | 24. Классы сложности, сведение задач | 4 | - | - |
| - | Обзор: Кратчайшие пути и Графы | - | - | - |
| - | Алгоритмы на карте | - | - | - |
| - | Эйлер: Путь ≠ Цикл | - | - | - |
| - | Мосты в графах: код + визуализация | 1 | - | - |

Рекомендуемый порядок работ

1. ****Вернуть потерянные билеты 1, 7, 11, 13**** и подключить (`segment-trees`, `stack-dfs`, `queue-bfs`, `heap-be...`)
это 5 готовых компонентов, которые сейчас просто не...
2. ****Дописать аналогии**** в 5 главах из «Уровня 3» (по ш...
3. ****Добавить 2D**** к 4 главам «Уровня 4» – минимум inli...
4. ****Решить судьбу `alg-map`**** – раскрыть или удалить.

ФАЙЛ 9/82: tsconfig.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 32 | РАЗМЕР: 6...

```
{
  "compilerOptions": {
    "target": "ES2020",
    "useDefineForClassFields": true,
    "lib": ["ES2020", "DOM", "DOM.Iterable"],
    "module": "ESNext",
    "skipLibCheck": true,
    "types": ["node"],
```



```
-----
const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
  plugins: [react(), tailwindcss(), viteSingleFile()],
  resolve: {
    alias: {
      "@": path.resolve(__dirname, "src"),
    },
  },
  server: {
    host: "0.0.0.0",
    port: 5173,
    strictPort: true,
    // предпросмотр может открываться через внешний про
    allowedHosts: true,
  },
  preview: {
    host: "0.0.0.0",
    port: 4173,
    strictPort: true,
    allowedHosts: true,
  },
});
```

=====

РАЗДЕЛ: ЯДРО ПРИЛОЖЕНИЯ

=====

ФАЙЛ 11/82: src/App.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 118 | РАЗМЕР: 4.9 КБ

```
import { useCallback, useEffect, useMemo, useState } from "react";
import { chapters } from "../data/content";
import { Navbar } from "../components/Navbar";
```

```
const progress = useMemo(() => ((index + 1) / chapters
```

```
return (
```

```
  <div className="min-h-screen bg-slate-950 text-slate-50">
```

```
    <Navbar activeTab={activeTab} setActiveTab={setActiveTab}>
```

```
      {activeTab === "guide" ? (
```

```
        <>
```

```
          <MobileToc
```

```
            selectedChapterId={activeChapterId}
```

```
            setSelectedChapterId={goTo}
```

```
            currentTitle={activeChapter.title}
```

```
            index={index}
```

```
            total={chapters.length}
```

```
          />
```

```
        <div className="flex flex-col lg:flex-row max-w-7xl">
```

```
          {/* Сайдбар только на десктопе – на мобильном скрывается */}
```

```
          <div className="hidden lg:block lg:w-80 shrink-0">
```

```
            <Sidebar selectedChapterId={activeChapterId}>
```

```
          </div>
```

```
          <main id="main-content" className="flex-1 min-w-0">
```

```
            {/* Шапка главы с быстрыми переходами */}
```

```
            <div className="flex items-center justify-between">
```

```
              <div className="min-w-0">
```

```
                <div className="text-[10px] uppercase">
```

```
                  {activeChapter.category || "Универсальное пособие"}>
```

```
                </div>
```

```
                <h2 className="text-base sm:text-xl font-medium">
```

```
                  {activeChapter.title}>
```

```
                </h2>
```

```
              </div>
```

```
              <ChapterNav prev={prev} next={next} index={index}>
```

```
            </div>
```

```
          <div className="h-1 w-full bg-slate-800 rounded">
```

```
            <div
```

ФАЙЛ 12/82: src/hooks/useTermHints.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 124 | РАЗМЕР: 5.0 КБ

```
import { useEffect } from "react";
import { GLOSSARY_LABELS } from "../data/glossary";

/**
 * Ограничители, чтобы текст не рябил, но и чтобы подсказки
 *
 * Считаем не «сколько раз встретился термин», а «сколько
 * написание». Иначе в главе про splay первые же «Splay
 * лимит, и слова «Zig-Zag», «вращений», «AVL-дереве» о
 */
const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;

const SKIP_SELECTOR = "code, pre, script, style, a, tex

const escapeRe = (s: string) => s.replace(/[\.\*\+\?\^\$\{\}\(\)|\

/** Одна большая регулярка из всех меток: длинные раньше
function buildRegex(): RegExp {
  const alternation = GLOSSARY_LABELS.map((l) => escapeRe(l))
  // границы «не буква/цифра» – \b не работает с кириллицей
  return new RegExp(`(?<![\\p{L}\\p{N}]_)(?=${alternation})`);
}

let cachedRegex: RegExp | null = null;
const labelToId = new Map(GLOSSARY_LABELS.map((l) => [l, ...]));

/**
 * Проходит по тексту главы и оборачивает известные термины
 * <span class="term-hint" data-term-id="...">, чтобы на них
 * повесить всплывающую карточку (как превью ссылок в Википедии)
```

```

    const usedTerm = perTerm.get(id) ?? 0;
    const usedSurface = perSurface.get(surface) ?? 0;
    if (usedTerm >= MAX_PER_TERM || usedSurface >= MAX_SURFACE) {
      perTerm.set(id, usedTerm + 1);
      perSurface.set(surface, usedSurface + 1);
    }

    if (m.index > last) frag.appendChild(document.createElement("div"));
    const span = document.createElement("span");
    span.className = "term-hint";
    span.dataset.termId = id;
    span.tabIndex = 0;
    span.setAttribute("role", "button");
    span.setAttribute("aria-label", `Подсказка: ${m[1]}`);
    span.textContent = m[1];
    frag.appendChild(span);
    last = m.index + m[1].length;
  }

  if (last === 0) continue;
  if (last < text.length) frag.appendChild(document.createElement("div"));
  if (textNode.parentNode?.replaceChild(frag, textNode)) {
    //
  }
}

/**
 * Навешивает разметку терминов на контейнер при смене
 *
 * Дополнительно перепроверяет разметку после каждого рендера
 * (или что-то ещё) перезаписал innerHTML главы, подсказки
 * а не пропадают до перезагрузки страницы.
 */
export function useTermHints(ref: React.RefObject<HTMLDivElement>) {
  const run = () => {
    const el = ref.current;
    if (!el) return;
    try {
      annotateTerms(el);
    }
  };
}

```

```
.no-scrollbar::-webkit-scrollbar {
  display: none;
}
.no-scrollbar {
  scrollbar-width: none;
}
}

@keyframes pulse-gold {
  0%,
  100% {
    stroke: #eab308;
    stroke-width: 5;
    filter: drop-shadow(0 0 2px rgba(234, 179, 8, 0.5))
  }
  50% {
    stroke: #fef08a;
    stroke-width: 8;
    filter: drop-shadow(0 0 12px rgba(234, 179, 8, 0.9))
  }
}

.animate-pulse-gold {
  animation: pulse-gold 1.5s infinite;
}

@keyframes tocIn {
  from {
    transform: translateX(-100%);
  }
  to {
    transform: translateX(0);
  }
}

@keyframes hintIn {
  from {
    opacity: 0;
  }
}
```

```
    max-width: 100%;
    overflow-wrap: anywhere;
}

.chapter-body :where(section, div, p, li) {
    min-width: 0;
}

.chapter-body :where(pre, code) {
    white-space: pre-wrap;
    word-break: break-word;
}

.chapter-body pre {
    overflow-x: auto;
    max-width: 100%;
    font-size: clamp(11px, 2.9vw, 13px);
}

.chapter-body table {
    display: block;
    width: 100%;
    overflow-x: auto;
    border-collapse: collapse;
    font-size: clamp(11px, 3vw, 14px);
}

.chapter-body summary {
    cursor: pointer;
    padding: 0.35rem 0;
}

@media (max-width: 767px) {
    .chapter-body :where(.grid) {
        grid-template-columns: minmax(0, 1fr) !important;
    }
    .chapter-body :where(h2) {
        font-size: 1.25rem !important;
    }
}
```

```
    from {  
      width: 0%;  
    }  
    to {  
      width: 100%;  
    }  
  }  
}
```

ФАЙЛ 14/82: src/main.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 11 | РАЗМЕР: 230 В

```
import { StrictMode } from "react";  
import { createRoot } from "react-dom/client";  
import "./index.css";  
import App from "./App";  
  
createRoot(document.getElementById("root")!).render(  
  <StrictMode>  
    <App />  
  </StrictMode>  
);
```

ФАЙЛ 15/82: src/types.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 9 | РАЗМЕР: 153 В

```
export interface Chapter {  
  id: string;  
  title: string;  
  type: "html" | "markdown";  
  content: string;  
  description?: string;  
  category?: string;
```

```
export const additionalTickets: Chapter[] = [
  {
    id: "sparse-table",
    title: "2. Двумерная разреженная матрица (Sparse Table)",
    type: "html",
    description: "Разреженная таблица для идемпотентных операций",
    category: "Продвинутые структуры",
    content: `
<section id="sparse-table" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-indigo-600">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
      <h3 class="text-xl font-bold text-blue-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">
        <p class="text-xl font-mono text-white">
          ух отрезков:  $\min(ST[L][k], ST[R - 2^k + 1][k])$ 
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
            Аналогия 1: Школьные линейки
          </div>
          <p class="text-slate-300 text-sm mb-4">
            зок длины 7. Ты берешь две заготовленные заготовки
            ь отрезок длины 7 (перекрывая друг друга по 1 единицу)
            ничего складывать, просто пересекаем куски.
          </div>
          <div class="bg-slate-900 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 right-4 bottom-4 border border-rose-500 shadow-md">
              Аналогия 2: Ступени
            </div>
          </div>
        </div>
      </div>
    </div>
  </div>
`
  }
]
```



```
        <p class="text-lg text-blue-300 italic">
        <p class="text-xl font-mono text-white">
метода: Split и Merge.</p>
</div>
<div class="grid grid-cols-1 xl:grid-cols-2">
    <div class="bg-slate-800 p-6 rounded-lg">
        <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
            Аналогия 1: Удар катаной (Sp
        </div>
        <p class="text-slate-300 text-sm mb
по значению X. Все, кто меньше X улетают в.
        </div>
        <div class="bg-slate-900 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
                Аналогия 2: Слияние капель (M
            </div>
            <p class="text-slate-300 text-sm mb
вого). Мы просто смотрим на корни: у кого Y
как потомок.</p>
        </div>
</div>
<details class="bg-slate-900/40 rounded-lg l
    <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
        Код (Скрыто)
    </summary>
    <div class="p-5 text-sm text-slate-300">
        <pre class="bg-slate-950 p-4 rounde
pair<Node*, Node*> split(Node* t, int x) {
    if (!t) return {nullptr, nullptr};
    if (t->val <= x) {
        auto [L, R] = split(t->right, x);
        t->right = L;
        return {t, R};
    } else {
        auto [L, R] = split(t->left, x);
```

```

        </p>
        <p class="text-slate-300 text-sm mb-4">
        p>
        <p class="text-slate-300 text-sm">
        ).</p>
        </div>
    </div>

    <details class="bg-slate-900/40 rounded-lg border-
        <summary class="p-4 font-bold text-slate-300
        -b border-slate-700/50">
        Код и сложность (Скрыто)
        </summary>
        <div class="p-5 text-sm text-slate-300">
        <pre class="bg-slate-950 p-4 rounded">
def insert(root, key, pri):
    if root is None:
        return Node(key, pri) # пустое кресло
    if key <= root.key: # равные — по
        root.left = insert(root.left, key, pri)
    else:
        root.right = insert(root.right, key, pri)
    return root

for key, pri in pairs_sorted_by_pri_desc: # 8 → -4
    root = insert(root, key, pri)</pre>
    <p>Сортировка  $O(n \log n)$  плюс  $n$  спуску, тогда  $O(n^2)$ . Сложность даёт сортировка:
    </div>
</details>
</div>
</div>
</section>`,
    },
    {
        id: "splay-tree",
        title: "4. Splay-дерево",
        type: "html",

```

```

        </div>
    </div>
</div>

<!-- 1. Зачем -->
<div class="bg-slate-800/60 p-6 rounded-xl border-2 border-emerald-500 shadow-md">
    <h3 class="text-lg font-bold text-white mb-4">Вставка в бинарное дерево
    <p class="text-slate-300 text-sm mb-4">В бинарном дереве элементы добавляются в зависимости от значения. Если элемент меньше, чем текущий, он добавляется в левую ветвь, если больше — в правую. В процессе поиска по возрастанию — и дерево вырождается в линейную структуру.
    <pre class="bg-slate-950 p-4 rounded-lg overflow-x-auto font-mono text-slate-300">вставили 10, 20, 30, 40, 50

```

```

10
  \
   20
    \
     30
      \
       40
        \
         50

```

высота = 5, поиск 50 = 5 сравнений
это уже не дерево, а односвязный список

```

    </pre>
    <p class="text-slate-300 text-sm mt-4">Есть два способа избежать этого:
    <ul class="list-disc list-inside text-slate-300">
        <li><span class="text-white font-bold">1. Равновесные деревья</span>
            каждой вставки чинят дерево, чтобы оно оставалось сбалансированным.
        </li>
        <li><span class="text-white font-bold">2. Бинарные деревья</span>
            специально не пытаются быть кривым. Но каждый раз, когда мы добавляем элемент, оно становится кривее.
        </li>
    </ul>
</div>

```

```

<!-- 2. Аналогии -->
<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
    <div class="bg-slate-800 p-6 rounded-lg border-2 border-emerald-500 shadow-md">
        <div class="absolute -top-3 left-4 bg-slate-900 text-white font-mono text-sm">
            border-emerald-500 shadow-md">

```

Аналогия 1: стопка бумаг на столе

```

        <p class="text-amber-400 font-bold">
        <p class="text-slate-300 text-xs">П
одно и то же место, поэтому В просто переве
    </div>
</div>
</div>

<!-- 4. Три случая -->
<div class="bg-slate-800/60 p-6 rounded-xl border"
    <h3 class="text-lg font-bold text-white mb-3">
    <p class="text-slate-300 text-sm mb-4">Обозначим
    узел, который поднимаем, <span class="font-weight-bold">
    >g</span> – дед (родитель родителя). Смотри

    <div class="overflow-x-auto -mx-2 px-2">
        <table class="w-full text-xs sm:text-sm"
            <thead>
                <tr class="text-left text-slate-300">
                    <th class="py-2 pr-3 font-bold">
                    <th class="py-2 pr-3 font-bold">
                    <th class="py-2 pr-3 font-bold">
                    <th class="py-2 font-bold">
                </tr>
            </thead>
            <tbody class="text-slate-300">
                <tr class="border-b border-slate-700">
                    <td class="py-3 pr-3 font-bold">
                    <td class="py-3 pr-3">деда
                    <td class="py-3 pr-3">сам х
                    <td class="py-3">х стал корн
                </tr>
                <tr class="border-b border-slate-700">
                    <td class="py-3 pr-3 font-bold">
                    <td class="py-3 pr-3">х и п
аво-право)</td>
                    <td class="py-3 pr-3"><span class="font-weight-bold">
                    <td class="py-3">х поднялся
                </tr>
            </tbody>
        </table>
    </div>

```

```

                <p class="text-emerald-400 font-bol
                <pre class="overflow-x-auto text-[1

```

20 / 40 / 20 → 20 / 20 \ 60 → A / 40 \ 60

было: линия из 3 узлов

стало: кустик высоты 2

глубина ветки упала вдвое!</pre>

```

                <p class="text-slate-400 text-xs mt
                ем.</p>
                </div>
            </div>
            <p class="text-slate-300 text-sm mt-4"><spa
            з, змейка (zig-zag) – крути СНИЗУ вверх.</p>
        </div>

```

<!-- 6. Полный пример -->

```

<div class="bg-slate-800/60 p-6 rounded-xl bord
    <h3 class="text-lg font-bold text-white mb-1
    <p class="text-slate-300 text-sm mb-4">Дере
        3 шага, а потом делаем Splay(10).</p>
    <pre class="bg-slate-950 p-4 rounded-lg ove
    0 leading-relaxed">старт                после Zi
        глубина 3 → 1                10 в корне

```

40 / 30 / 20 / 10 → 30 / 10 \ 40 \ 20 → 10 \ 30 \ 20 \ 40

итог: было 4 узла в линию (высота 4) → стало высота 3,
а сама десятка теперь в корне: следующий запрос к

```

        <div class="bg-slate-900 rounded-lg p-3
    )</span> <span class="text-slate-400">— Spl
    росто отрезаем ссылки.</span></div>
        <div class="bg-slate-900 rounded-lg p-3
    k)</span> <span class="text-slate-400">— sp
    /div>
        <div class="bg-slate-900 rounded-lg p-3
    )</span> <span class="text-slate-400">— Spl
    него не будет правого сына — туда и вешаем
    </div>
</div>

<!-- 9. Код -->
<details class="bg-slate-900/40 rounded-lg bord
    <summary class="p-4 font-bold text-slate-40
    open:border-b border-slate-700/50">
        Код: rotate + splay + find (нажми, что
    </summary>
    <div class="p-5 text-sm text-slate-300 spac
        <pre class="bg-slate-950 p-4 rounded ov
    xed">// узел: { key, left, right, parent }

// один поворот: x встаёт на место своего родителя
function rotate(x) {
    const p = x.parent, g = p.parent;

    if (p.left === x) {                // правый поворот
        p.left = x.right;
        if (x.right) x.right.parent = p;
        x.right = p;
    } else {                            // левый поворот (зеркал
        p.right = x.left;
        if (x.left) x.left.parent = p;
        x.left = p;
    }

    p.parent = x;                      // родитель уехал вниз
    x.parent = g;                      // x занял его место

```

```

<div class="grid grid-cols-1 md:grid-cols-2 gap-4" style="background-color: #f0f0f0; padding: 10px;">
  <div class="bg-emerald-950/30 rounded-xl p-5" style="background-color: #008000; color: white; padding: 10px;">
    <p class="text-emerald-300 font-bold mb-2">Простой код: нет высот, цветов,
    <ul class="list-disc list-inside text-slate-300" style="list-style-type: none;">
      <li>Простой код: нет высот, цветов,
      <li>Меньше памяти, чем у AVL и красных чертов
      <li>Сам подстраивается под «горячие» данные
      <li>split / merge получаются почти бесплатно
    </ul>
  </div>
  <div class="bg-rose-950/30 rounded-xl p-5" style="background-color: #ff0000; color: white; padding: 10px;">
    <p class="text-rose-300 font-bold mb-2">Сложный код:
    <ul class="list-disc list-inside text-slate-300" style="list-style-type: none;">
      <li>Нет гарантии на <em>отдельную</em> запись
      <li>Не годится для реального времени
      <li>Дерево меняется даже при чтении
      <li>Постоянные записи в память при чтении
    </ul>
  </div>
</div>

```

<!-- 11. Вопросы -->

```

<details class="bg-slate-900/40 rounded-lg border border-slate-700/50" style="background-color: #000080; color: white; padding: 10px;">
  <summary class="p-4 font-bold text-slate-400" style="background-color: #000080; color: white; padding: 5px;">
    Вопросы, которые задают на экзамене (иногда)
  </summary>

```

```

  <div class="p-5 text-sm text-slate-300 space-y-4" style="background-color: #000080; color: white; padding: 10px;">
    <div>
      <strong class="text-white block mb-2">Вопросы, которые задают на экзамене (иногда)
      <p class="text-slate-400">Последний вопрос: «Сделайте бамбук, который не ломается, не гниет, не требует полива, не нуждается в удобрении или преемнике). Splay делают всегда, даже если бамбук уже сломался, и амортизация сломалась бы.</p>
    </div>
    <div>
      <strong class="text-white block mb-2">Почему Splay делают всегда, даже если бамбук уже сломался, и амортизация сломалась бы.</p>
      <p class="text-slate-400">Потому что Splay делают всегда, даже если бамбук уже сломался, и амортизация сломалась бы.
    </div>
  </div>

```

```

</div>
<div class="grid grid-cols-1 xl:grid-cols-2" >
  <div class="bg-slate-800 p-6 rounded-lg" >
    <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md" >
      ✖ Аналогия 1: Вырезание прямоугольника
    </div>
    <p class="text-slate-300 text-sm mb-2" >
      Из него маленький целевой прямоугольник в центре.
      Его верхний угол отрезается дважды! Поэтому
    </p>
  </div>
</div>
</div>
</section>`,
  },
  {
    id: "graph-planar-colors",
    title: "7. Графы. Планарные. Покраска",
    type: "html",
    description: "Формула Эйлера и теорема о 4 красках.",
    category: "Графы. Теория",
    content: `
<section id="graph-planar-colors" class="mb-12 scroll-mt-12" >
  <div class="flex items-center mb-6 flex-wrap gap-3" >
    <span class="bg-blue-600 text-white px-4 py-1 rounded" >
      Теория
    </span>
    <h2 class="text-2xl sm:text-3xl font-bold text-white" >
      7. Графы. Планарные. Покраска
    </h2>
  </div>
  <div class="space-y-8" >
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600" >
      <h3 class="text-xl font-bold text-blue-400" >
        Теорема о 4 красках
      </h3>
      <div class="bg-slate-900 p-4 rounded-lg mb-4" >
        <p class="text-lg text-blue-300 italic" >
          Любой планарный граф можно раскрасить 4 цветами.
        </p>
        <p class="text-xl font-mono text-white" >
          Любой граф можно раскрасить 4 цветами.
        </p>
      </div>
      <div class="grid grid-cols-1 gap-6" >
        <div class="bg-slate-800 p-6 rounded-lg" >

```



```

        </div>
        <p class="text-slate-300 text-sm mb-4">
        еская сортировка – это расписание, которое
        >
        </div>
        <div class="bg-slate-900 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 border-rose-500 shadow-md">
                Ограничения: Циклы
            </div>
            <p class="text-slate-300 text-sm mb-4">
            получить опыт, нужна работа". Алгоритм выяв
            </div>
        </div>
    </div>
</div>
</section>`,
    },
    {
        id: "scc-kosaraju",
        title: "10. Графы. Компоненты сильной связности (SCC)",
        type: "html",
        description: "Разбиение орграфа на макро-вершины. Алгоритм Косарая",
        category: "Графы. Продвинутое",
        content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">Графы
        <h2 class="text-2xl sm:text-3xl font-bold text-white">10. Графы. Компоненты сильной связности (SCC)
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border">
            <h3 class="text-xl font-bold text-emerald-400">Макро-вершины
            <div class="bg-slate-900 p-4 rounded-lg mb-4">
                <p class="text-lg text-emerald-300 italic">Макро-вершина – это компонент сильной связности (SCC).
                <p class="text-xl font-mono text-white">Макро-вершины инвертированы в порядке top-sort.</p>
            </div>
        </div>
    </div>
</section>`
    }
]

```

```

        order border-emerald-500 shadow-md">
            Аналогия 1: Единственный мост
        </div>
        <p class="text-slate-300 text-sm mb-4">
            Этот мост, экономикой обоих кусков придет
        </div>
    </div>
</div>
</section>`,
    },
    {
        id: "graph-articulation",
        title: "12. Графы. Точки сочленения",
        type: "html",
        description: "Вершины, удаление которых убивает связность",
        category: "Графы. Продвинутые",
        content: `
<section id="graph-articulation" class="mb-12 scroll-mt-12">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">Графы
        <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">Точки сочленения
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
            <h3 class="text-xl font-bold text-rose-400">Точки сочленения
            <div class="bg-slate-900 p-4 rounded-lg mb-4">
                <p class="text-lg text-rose-300 italic">Вершины, удаление которых убивает связность
                <p class="text-xl font-mono text-white">
                    теи > 1).</p>
            </div>
            <div class="grid grid-cols-1 gap-6">
                <div class="bg-slate-800 p-6 rounded-lg">
                    <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
                        Аналогия 1: Центральный роутер
                    </div>
                    <p class="text-slate-300 text-sm mb-4">

```

```

    </div>
</section>`,
  },
  {
    id: "pathfinding-accel",
    title: "17. Графы. Ускорения поиска",
    type: "html",
    description: "",
    category: "Графы. Пути",
    content: `
<section id="pathfinding-accel" class="mb-12 scroll-mt-12">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">Графы
    <h2 class="text-2xl sm:text-3xl font-bold text-blue-400">17. Графы. Пути
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
      <h3 class="text-xl font-bold text-blue-400">Ускорения поиска
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">Вспомогательная таблица
        <p class="text-xl font-mono text-white">
          d-длина.</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия: Копаем туннель под землей.
          </div>
          <p class="text-slate-300 text-sm mb-4">
            Если копая туннель в 2 РАЗА меньше работы (геометрически: площадь
            </p>
          </div>
        </div>
      </div>
    </div>
  </div>
</section>`,
  },

```

```

    type: "html",
    description: "",
    category: "Графы. Остовные",
    content: `
<section id="mst-prima" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
      <h3 class="text-xl font-bold text-emerald-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">
        <p class="text-xl font-mono text-white">
          которое торчит из посещенных в непосещенных
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 right-4 bottom-4
            order border-emerald-500 shadow-md">
            ✦ Аналогия: Заражение вирусом
          </div>
          <p class="text-slate-300 text-sm mb-4">
            ли вирус). Мы стартуем из одного города, и
            Сеть растет только из одного связного куска
          </div>
        </div>
      </div>
    </div>
  </div>
</section>`,
  },
  {
    id: "mst-boruvka",
    title: "20. Графы. Остовное дерево. Борувка",
    type: "html",
    description: "",
    category: "Графы. Остовные",
  },

```

```

    <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
</div>
<div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
        <h3 class="text-xl font-bold text-blue-400">
        <div class="bg-slate-900 p-4 rounded-lg mb-4">
            <p class="text-lg text-blue-300 italic">
            <p class="text-xl font-mono text-white">
i], который одновременно является её суффиксом.
        </div>
        <div class="grid grid-cols-1 xl:grid-cols-2">
            <!-- Аналогия 1 -->
            <div class="bg-slate-800 p-6 rounded-lg">
                <div class="absolute -top-3 left-4 right-4 bottom-4
rder border-emerald-500 shadow-md">
                    Аналогия 1: Бумеранг (Префикс и суффикс)
                </div>
                <p class="text-slate-300 text-sm mb-4">
идишь "АБРА". Если ты ошибешься при поиске окончания, то получишь
концовка "АБРА" совпадает с началом "АБРА", что приводит к бесконечному
меня перепроверок!</p>
            </div>
        </div>
        <details class="bg-slate-900/40 rounded-lg border">
            <summary class="p-4 font-bold text-slate-300">
                Код (Скрыто)
            </summary>
            <div class="p-5 text-sm text-slate-300">
                <pre class="bg-slate-950 p-4 rounded">
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
    int j = pi[i-1];
    while (j > 0 && s[i] != s[j]) j = pi[j-1];
    if (s[i] == s[j]) j++;
    pi[i] = j;
}</pre>

```

```

        <summary class="p-4 font-bold text-slate-700 border-slate-700/50">
            Код (Скрыто)
        </summary>
        <div class="p-5 text-sm text-slate-300">
            <pre class="bg-slate-950 p-4 rounded">
int l = 0, r = 0;
for (int i = 1; i < n; i++) {
    if (i <= r) z[i] = min(r - i + 1, z[i - 1]);
    while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
    if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
}</pre>
            </div>
        </details>
    </div>
</div>
</section>`,
    },
    {
        id: "complexity-classes",
        title: "24. Классы сложности, сведение задач",
        type: "html",
        description: "",
        category: "Теория сложности",
        content: `
<section id="complexity-classes" class="mb-12 scroll-mt">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-purple-600 text-white px-4 py-1">
        <h2 class="text-2xl sm:text-3xl font-bold text-white">
    </div>
    <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border">
            <h3 class="text-xl font-bold text-purple-400">
            <div class="bg-slate-900 p-4 rounded-lg mb-4">
                <p class="text-lg text-purple-300 italic">
                <p class="text-xl font-mono text-white">
Сведение (Reduction) A->B: Если я умею реша
            </div>
        </div>
    </div>
    </div>
</section>
        `
    }
]

```

```

    content: string;
  }

export const chapters: Chapter[] = [
  {
    id: "euler-path-vs-cycle",
    title: "Эйлер: Путь ≠ Цикл (ты путаешь!)",
    type: "html",
    content: `<section id="euler-path-vs-cycle" class="
<div class="flex items-center mb-6">
  <span class="bg-indigo-600 text-white px-4 py-1
  <h2 class="text-3xl font-bold text-white">He by
</div>

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl bord
    <h3 class="text-xl font-bold text-blue-400

    <div class="bg-slate-900 p-4 rounded-lg mb-
      <p class="text-lg text-blue-300 italic
      <div class="text-left font-mono text-sm
        <p><strong class="text-white">Эйлер
ин раз. Вершины повторять можно.</p>
        <p><strong class="text-white">Эйлер
        <div class="p-3 bg-slate-950 rounde
          <p class="text-sm text-blue-300
          <p><strong class="text-white">П
          <p><strong class="text-white">Ц
        </div>
      </div>
    </div>
  </div>
  <div class="p-6 rounded-xl border
    <p class="text-slate-300 text-sm">
      <strong>Билет 13 (Различие пути и цикла
      <br>• <span class="text-green-400 font-
шел-вошёл-вышел).
      <br>• <span class="text-yellow-400 font
финиш).
    </p>
  </div>
</div>
`
  }
]

```

```

<div class="p-5 text-sm text-slate-300 space-y-4">
  <p class="text-blue-400">Теорема Эйлера
  <p>
    Связный граф содержит эйлеров цикл
    <br>Эйлеров путь (не цикл)  $\Rightarrow$  ровно 2 вершины с нечётной степенью
  </p>
  <p class="text-rose-400">Почему Гамильтон?
  <p>
    Задача коммивояжёра (TSP) – это взвешенный граф.
    Доказано, что любой полиномиальный алгоритм решает задачу.
    В общем, не парься. Просто запомни.
  </p>
</div>
</details>
</div>
</section>`,
},
{
  id: "bridges-code",
  title: "Мосты в графах: код + визуализация",
  type: "html",
  content: `<section id="bridges-code" class="mb-20 space-y-4">
<div class="flex items-center mb-6">
  <span class="bg-emerald-600 text-white px-4 py-2 rounded">Код
  <h2 class="text-3xl font-bold text-white">Мосты
</div>

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border">
    <h3 class="text-xl font-bold text-emerald-400">Мосты в графах
  </div>

  <div class="bg-slate-900 p-4 rounded-lg mb-6">
    <p class="text-lg text-emerald-300 italic">Мосты в графах – это графы, в которых
    <div class="text-left font-mono text-sm">
      <p><strong class="text-white">tin[v] – количество мостов,
      <p><strong class="text-white">low[v] – количество мостов,
      <div class="p-3 bg-slate-950 rounded">
        <span class="text-rose-400 font`

```


</div>

<details class="bg-slate-900/40 rounded-lg border-1 border-slate-700/50">
 <summary class="p-4 font-bold text-slate-400">
 order-slate-700/50">

Полный код на C++ (Скрыто)

</summary>

<div class="p-5 text-sm text-slate-300 space-y-2">
 <p>Классическая реализация – <code class="text-slate-400">
 оходов.</p>

<div class="bg-slate-950 p-4 rounded-lg border-1 border-slate-700/50">
 <pre class="text-xs font-mono text-slate-400">

```
#include <vector>
```

```
#include <algorithm>
```

```
using namespace std;
```

```
int n; // число вершин
```

```
vector<vector<int>>> adj; // список смежных ребер
```

```
vector<bool> visited;
```

```
vector<int> tin, low;
```

```
int timer;
```

```
void dfs(int v, int p = -1) {
```

```
    visited[v] = true;
```

```
    tin[v] = low[v] = timer++; // устанавливаем время
```

```
    for (int to : adj[v]) {
```

```
        if (to == p) continue; // не идём назад
```

```
        if (visited[to]) {
```

```
            // обратное ребро: обновляем low
```

```
            low[v] = min(low[v], tin[to]);
```

```
        } else {
```

```
            // ребро дерева DFS
```

```
            dfs(to, v); // рекурсивный вызов
```

```
            low[v] = min(low[v], low[to]);
```

```
<h3 class="text-xl font-bold text-blue-400">
```

```
<div class="bg-slate-900 p-4 rounded-lg mb-4">
```

```
<p class="text-lg text-blue-300 italic">
```

```
<p class="text-3xl font-mono text-white">
```

```
<div class="text-sm text-slate-400 space-x-2">
```

```
<p><span class="text-blue-400 font-l">
```

```
<p><span class="text-emerald-400 fo">
```

```
<p><span class="text-rose-400 font-l">
```

```
</div>
```

```
</div>
```

```
<p class="text-slate-300 text-sm">
```

```
<strong>Следствие:</strong> Для планарно  
ode>.
```

```
Если ребер больше – граф гарантированно
```

```
</p>
```

```
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
```

```
<div class="bg-slate-800 p-6 rounded-lg border">
```

```
<div class="absolute -top-3 left-4 bg-slate-900  
order-green-500 shadow-md">
```

```
К5: Полный граф на 5 вершинах
```

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
```

```
У К5: <code class="text-white">V=5,  
-white">10 &gt; 9</code> – БАХ, непланарен!
```

```
Все 5 вершин соединены со всеми. На
```

```
Теорема Куратовского: К5 – эталон не
```

```
Если внутри графа спрятан К5 – забудь
```

```
</p>
```

```
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border">
```

```
<div class="absolute -top-3 left-4 bg-rose-500  
der-rose-500 shadow-md">
```

```
К3,3: 3 дома и 3 колодца
```

```
<div class="flex items-center mb-6">
  <span class="bg-rose-600 text-white px-4 py-1 rounded">
    <h2 class="text-3xl font-bold text-white">Точки
  </div>
```

```
<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border">
    <h3 class="text-xl font-bold text-rose-400">

    <div class="bg-slate-900 p-4 rounded-lg mb-4">
      <p class="text-lg text-rose-300 italic">
        <div class="text-left font-mono text-sm">
          <p><strong class="text-white">Точка
рой увеличивает число компонент связности.</p>
        <div class="p-3 bg-slate-950 rounded">
          <span class="text-rose-400 font">
            <p class="text-xl font-bold tex">
              <p class="text-xs text-slate-400">
            </div>
          </div>
        </div>
      </div>
```

```
<p class="text-slate-300 text-sm">
  <strong>Важно:</strong> Это работает то
еются к точкам сочленения. То есть, <strong>
ег - это тупик со степенью 1).
</p>
```

```
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap">
  <div class="bg-slate-800 p-6 rounded-lg border">
    <div class="absolute -top-3 left-4 bg-slate-800
order-rose-500 shadow-md">
      Аналогия: Главный перекресток
    </div>
    <p class="text-slate-300 text-sm mb-4">
      Представь город, где два района соед
её перекроют (удалят вершину) – районы буд
```

```

        low[v] = min(low[v], tin[to]);
    } else {
        dfs(to, v);
        low[v] = min(low[v], low[to]);
        if (low[to] >= tin[v] && p != -1)
            IS_CUTPOINT(v);
        ++children;
    }
}
if (p == -1 && children > 1)
    IS_CUTPOINT(v);
}</pre>
```

</details>

<div class="bg-indigo-950/60 p-6 rounded-xl border

class="text-lq font-bold text-indigo-400">

<p class="text-slate-300 text-sm mb-4">

Это глубокий дуальный мост между ориент

</p>

- **Navigation:**
 - **Home:** Overview of the course and its objectives.
 - **Introduction:** A brief history of the field and its importance.
 - **Concepts:** Key concepts and theories discussed in the course.
 - **Methods:** Research methods and data analysis techniques.
 - **Applications:** Practical applications of the concepts and methods.
 - **Conclusion:** Summary of the course and its findings.

- class="text-rose-400">Точки**

онд> и показывают физическую уязвимость свя.

```
<li><strong class="text-emerald-400">SC
```

- зависимые "конгломераты".

- Как точка сочленения рушит**

Косарайю, свернув каждую SCC в одну супер-в

(сделаем неориентированным), то любая <stro

о и есть критическая SCC! Если удалить эту

Одно слабое звено изолирует целые касты S

</section>`

1.

1;

```
const finalChapters = Array.from(dedup.values());

finalChapters.sort((a, b) => {
  const numA = parseInt(a.title.match(/(\d+)/)?.[0] ||
  const numB = parseInt(b.title.match(/(\d+)/)?.[0] ||
  return numA - numB;
});

export const chapters: Chapter[] = finalChapters;

export const activeVizIds = [
  "euler-path-vs-cycle", "bridges-code", "planarity-euler",
  "graph-dfs-bfs", "stack-dfs", "queue-bfs", "heap-beam-se",
  "intro", "dijkstra", "bellman-ford", "floyd", "aho-corasi",
  "mst-kruskal", "mst-prima", "mst-boruvka", "string-kmp",
  "string-z-func", "segment-trees", "sparse-table", "dynam
];
```

ФАЙЛ 20/82: src/data/glossary.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 636 | РАЗМ

```
import type { DemoKind } from "../components/hints/Mini

/**
 * Глоссарий «википедийных» подсказок.
 *
 * Любое слово из `labels`, встреченное в тексте главы,
 * в подчёркнутый термин: при наведении (или тапе на мо
 * с объяснением на пальцах и мини-визуализацией.
 */
export interface GlossaryEntry {
  id: string;
  /** Варианты написания в тексте (регистр не важен). */
  labels: string[];
```

```
{
  id: "binsect",
  labels: ["binsect", "бинсект", "бинарный поиск", "д"],
  title: "Бинарный поиск (и его самозванец binsect)",
  short:
    "Бинарный поиск НАХОДИТ готовый элемент в уже отсортированном массиве, но не сортирует. «binsect» из конспекта – шуточный, quicksort.",
  formal:
    "Поиск одного элемента в отсортированном массиве:  $O(n \log n)$ . Бинарный поиск сортировку не выполняет.",
  complexity: "поиск:  $O(\log n)$ ; сортировкой не является.",
},
{
  id: "heap",
  labels: ["куча", "кучу", "кучи", "куче", "кучей", "куча"],
  title: "Куча (priority queue)",
  short:
    "Не отсортированный список, а «мешок, из которого можно извлечь элемент с минимальным значением».",
  formal:
    "Полное бинарное дерево в массиве: дети узла  $i$  лежат в  $2i$  и  $2i+1$  местах.",
  complexity: "вставка/извлечение  $O(\log n)$ , «посмотреть минимум»  $O(1)$ ",
  demo: "heap",
  simulator: "heap-beam-search",
},
{
  id: "stack",
  labels: ["стек", "стека", "стеке", "стеком", "LIFO"],
  title: "Стек (LIFO)",
  short: "Стопка тарелок: кладёшь сверху и берёшь сверху",
  formal: "push / pop / top за  $O(1)$ . На стеке живёт рекурсия.",
  demo: "stack",
  simulator: "stack-dfs",
},
{
  id: "queue",
  labels: ["очередь", "очереди", "очередью", "FIFO"],
  title: "Очередь (FIFO)",
  short: "Линейный список: кладёшь в конец и берёшь из начала",
  formal: "push / pop за  $O(1)$ . На очереди живёт BFS.",
  demo: "queue",
  simulator: "queue-bfs",
},
}
```

```

        "автомат",
        "автомата",
    ],
    title: "Обход бора в ширину (тот самый «какой-то обход бора»)",
    short:
        "В Ахо–Корасике бор обходят именно BFS: суффиксную",
        "или есть строго по уровням.",
    formal:
        "Кладём в очередь детей корня (их ссылка = корень",
        "и кладём с в очередь.",
    complexity: "O(суммарной длины словаря × алфавит)",
    demo: "trie-bfs",
    simulator: "aho-corasick",
},
{
    id: "suffix-link",
    labels: ["суффиксная ссылка", "суффиксные ссылки", "ссылка"],
    title: "Суффиксная ссылка",
    short:
        "«Куда откатиться, если следующая буква не подошла",
        "строки.",
    formal: "link(v) = go(link(parent(v)), ch). Считает",
    demo: "suffix-link",
    simulator: "aho-corasick",
},
{
    id: "toposort",
    labels: ["топологическая сортировка", "топсорт", "топология"],
    title: "Топологическая сортировка",
    short:
        "Порядок «сначала штаны, потом ботинки»: каждая с",
        "циклов.",
    formal: "DFS + вершины в обратном порядке выхода, л",
    complexity: "O(V + E)",
    demo: "toposort",
    simulator: "top-sort",
},
{

```

```

    id: "segment-tree",
    labels: ["дерево отрезков", "дереве отрезков", "дер"],
    title: "Дерево отрезков",
    short:
        "Матрёшка из отрезков: корень отвечает за весь ма
        товых кусков.",
    formal: "Строится за  $O(n)$ , запрос и обновление —  $O($ 
    demo: "segment-tree",
    simulator: "segment-trees",
},
{
    id: "bridge",
    labels: ["мост", "моста", "мосты", "мостов", "мостов"],
    title: "Мост",
    short:
        "Ребро, после удаления которого граф разваливается.
    formal: "Ребро  $(u,v)$  — мост  $\iff \text{low}[v] > \text{tin}[u]$ , где
        ребро в родителя — нет).",
    complexity: " $O(V + E)$  одним DFS",
    demo: "bridge",
    simulator: "bridges-code",
},
{
    id: "articulation",
    labels: ["точка сочленения", "точки сочленения", "т"],
    title: "Точка сочленения",
    short:
        "Вершина-незаменимый-сотрудник: уволь её — и колл
    formal: "Для не-корня:  $v$  — точка сочленения  $\iff \exists$  реб
    demo: "articulation",
    simulator: "graph-articulation",
},
{
    id: "mst",
    labels: [
        "остовное дерево", "остовного дерева", "остовное"
        "минимальное остовное дерево", "MST",
    ],

```



```

    short:
        "Трюк «поднять все дороги на одинаковую высоту»:
formal: "w'(u,v) = w(u,v) + h(u) - h(v), где h — разность
        к путей сохраняется.",
demo: "relax",
simulator: "johnson-algo",
},
{
    id: "scc",
    labels: ["компонента сильной связности", "компоненты"],
    title: "Компоненты сильной связности",
    short:
        "Группы вершин, где из каждой можно дойти до каждой",
formal: "Косарайю: DFS с сохранением порядка выхода",
complexity: "O(V + E)",
demo: "scc",
simulator: "scc-kosaraju",
},
{
    id: "prefix-function",
    labels: [
        "префикс-функция", "префикс-функции", "префикс-функция",
        "префикс", "префикса", "суффиксом", "суффикс", "суффиксы",
        "подстроки", "подстроку", "подстрока",
    ],
    title: "Префикс-функция  $\pi$  (КМП)",
    short:
        "Для каждой позиции запоминаем: какой кусок начался в этой позиции
        от кусок.",
formal:
        " $\pi[i]$  — длина наибольшего собственного префикса s[i..n],
        возвращая указатель по тексту.",
complexity: "O(n + m)",
demo: "prefix-function",
simulator: "string-kmp",
},
{
    id: "z-function",

```

```
{
  id: "negative-cycle",
  labels: ["отрицательный цикл", "отрицательного цикла"],
  title: "Отрицательный цикл",
  short:
    "Круг, пройдя по которому вы становитесь «богаче»",
  formal:
    "Форд–Беллман: если на  $V$ -й итерации ещё удаётся с",
  demo: "relax",
  simulator: "bellman-ford",
},
{
  id: "greedy",
  labels: [
    "жадный алгоритм", "жадный", "жадно", "жадная",
    "самое дешёвое ребро", "самое дешёвое ребро", "са
  ],
  title: "Жадный алгоритм",
  short:
    "На каждом шаге хватаем то, что лучше прямо сейчас",
  formal:
    "Корректность обычно доказывается «леммой о разре",
  demo: "mst",
  simulator: "mst-kruskal",
},
{
  id: "euler",
  labels: ["эйлеров цикл", "эйлеров путь", "эйлерова п
  title: "Эйлеров путь и цикл",
  short:
    "Пройти по каждому ребру ровно один раз, не отрыв
  formal:
    "Связный граф: эйлеров цикл  все степени чётные;
  simulator: "euler-path-vs-cycle",
},
{
  id: "planarity",
  labels: ["планарный", "планарность", "планарного",
```

```

        льные.",
    formal:
        "Куча хранит пары (расстояние, вершина). Достали
complexity: "O(E log V) с бинарной кучей",
demo: "relax",
simulator: "dijkstra",
},
{
    id: "bellman-ford",
    labels: ["Форд-Беллман", "Форда-Беллмана", "Беллман
title: "Алгоритм Форда-Беллмана",
short:
    "Тупой, но честный: V-1 раз проходим по ВСЕМ рёбра
    ",
    formal:
        "После k-й итерации гарантированно верны все крат
        ось — есть отрицательный цикл.",
    complexity: "O(V · E)",
    demo: "relax",
    simulator: "bellman-ford",
},
{
    id: "floyd",
    labels: [
        "Флойд", "Флойда", "Флойд-Уоршелл", "Флойда-Уорше
        "промежуточная вершина", "промежуточную вершину",
    ],
    title: "Флойд-Уоршелл: разрешаем вершину k",
    short:
        "Внешний цикл — это не «шаг», а «какие пересадки
        шить пересадку в k?",
    formal:
        "D[i][j] = min(D[i][j], D[i][k] + D[k][j]). Поряд
        ами.",
    complexity: "O(V³) времени, O(V²) памяти",
    demo: "floyd",
    simulator: "floyd",
},

```

```

    id: "components",
    labels: [
        "компонента связности", "компоненты связности", "
        "компоненте связности", "связности", "связный", "
    ],
    title: "Компоненты связности",
    short:
        "Граф-архипелаг: острова, между которыми нет мост
        бой.",
    formal:
        "Считаются одним проходом: пока есть непосещённая
    complexity: "O(V + E)",
    demo: "components",
    simulator: "graph-dfs-bfs",
},
{
    id: "graph",
    labels: [
        "граф", "графа", "графе", "графы", "графов", "гра
        "вершины", "вершина", "вершин", "ребра с весом",
    ],
    title: "Граф",
    short:
        "Точки (вершины) и связи между ними (рёбра). Горо
    formal:
        "G = (V, E). Ребро бывает ориентированным (улица
    demo: "graph",
    simulator: "graph-dfs-bfs",
},
{
    id: "coord-compress",
    labels: ["сжатие координат", "сжатия координат", "с
    title: "Сжатие координат",
    short:
        "Координаты бывают гигантские и дробные, а нам ну
        порядковым номером.",
    formal:
        "sort + unique + binary search: значение → индекс

```

```

    formal:
      "Sum(x1..x2, y1..y2) = P[x2][y2] - P[x1-1][y2] - P[x2][y1-1] + P[x1-1][y1-1]",
    complexity: "запрос O(1) после O(nm) предподсчёта",
    demo: "prefix-sum",
    simulator: "prefix-sums-2d",
  },
  {
    id: "reduction",
    labels: ["сведение", "сведения", "сведении", "сведён", "сведёно"],
    title: "Сведение задач",
    short:
      "«Я не умею решать A, зато умею B». Если A можно решить, то и B можно решить.",
    formal:
      "A ≤p B: есть полиномиальная функция, переводящая решение A в решение B.",
    demo: "np",
  },
  {
    id: "range",
    labels: ["отрезок", "отрезка", "отрезке", "отрезки", "отрезков"],
    title: "Запрос на отрезке",
    short:
      "Спрашиваем не про один элемент, а про кусок массива. Как считать суммы на отрезке?",
    formal:
      "Суммы — префиксные суммы O(1). Минимум без изменений.",
    demo: "prefix-sum",
    simulator: "prefix-sums-2d",
  },
  {
    id: "string",
    labels: ["строки", "строке", "строку", "строка", "строки"],
    title: "Строка как массив символов",
    short:
      "Строковые алгоритмы держатся на трёх вопросах: где встречается слово (Z-функция) и где встречаются слова словаря (бор).",
    formal:
      "Наивный поиск подстроки — O(N·M). Префикс-функция — O(N).",
  }
}

```

```

    simulator: "splay-rotations",
  },
  {
    id: "zig-zag",
    labels: ["Zig-Zag", "зиг-заг", "ZigZag", "зигзаг"],
    title: "Zig-Zag – x и родитель с разных сторон",
    short:
      "Узлы идут «змейкой»: p – левый сын g, а x – правый сын g. p и g становятся двумя детьми x.",
    formal: "Эквивалентно двойному повороту AVL-дерева",
    demo: "zig-zag",
    simulator: "splay-rotations",
  },
];

```

```

/** Быстрый доступ по id. */
export const GLOSSARY_BY_ID = new Map(GLOSSARY.map((e) => [e.id, e]));

/** Все метки, отсортированные от длинных к коротким (чтобы было удобнее искать) */
export const GLOSSARY_LABELS: { label: string; id: string }[] =
  GLOSSARY.map((e) => ({ label: e.labels[0], id: e.id }))
    .sort((a, b) => b.label.length - a.label.length);

```

ФАЙЛ 21/82: src/data/graphContent.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 275 | РАЗМЕР: 1.5 КБ

```

import { Chapter } from "../types";

export const graphChapters: Chapter[] = [
  {
    id: "intro",
    title: "Обзор: Кратчайшие пути и Графы",
    type: "html",
    category: "Графы",
    content: `<section id="intro-content" class="mb-12

```

```

<div class="bg-slate-800 p-6 rounded-lg" style="border: 1px solid #000; border-radius: 10px;">
  <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
    Аналогия 1: Позитивное планирование
  </div>
  <p class="text-slate-300 text-sm mb-2">
    (нельзя поехать и заработать). Он всегда выбирает кратчайший путь.
  </p>
  <div id="slot-chapter-image-dijkstra" class="h-400px">
  </div>

  <!-- Аналогия 2 -->
  <div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #000; border-radius: 10px;">
    <div class="absolute -top-3 left-4 right-4 bottom-4 border border-rose-500 shadow-md">
      Ограничения
    </div>
    <p class="text-slate-300 text-sm mb-2">
      (невозможность посещения одной и той же улицы), Дейкстра сломается, так как не может вернуться.
    </p>
  </div>

  <details class="bg-slate-900/40 rounded-lg" style="border: 1px solid #000; border-radius: 10px;">
    <summary class="p-4 font-bold text-slate-300" style="border: 1px solid #000; border-radius: 10px; border-bottom: none;">
      Строгое доказательство (Скрыто)
    </summary>
    <div class="p-5 text-sm text-slate-300" style="border: 1px solid #000; border-radius: 10px; border-top: none;">
      <p>Алгоритм использует приоритетную очередь (min-heap). Гарантирует корректность только для не-взвешенных графов (или с неотрицательными весами).
    </p>
    <p>g V). Гарантирует корректность только для не-взвешенных графов (или с неотрицательными весами).
    </p>
  </div>
  </details>

  <div id="slot-dijkstra-viz" class="mt-8"></div>
</div>
</div>
</section>`
},
{

```

```
        г станет ЕЩЕ короче — значит мы попали во в
        </div>
    </div>

    <div id="slot-bellman-viz" class="mt-8"></div>
  </div>
</div>
</section>`
},
{
  id: "floyd",
  title: "Билет 16. Флойд-Уоршелл",
  type: "html",
  category: "Графы",
  content: `<section id="floyd-content" class="mb-12">
    <div class="flex items-center mb-6">
      <span class="bg-blue-600 text-white px-4 py-1 rounded">
        <h2 class="text-3xl font-bold text-white">Флойд
    </div>

    <div class="space-y-8">
      <div class="bg-slate-700/50 p-6 rounded-xl border">
        <h3 class="text-xl font-bold text-indigo-400">

        <div class="bg-slate-900 p-4 rounded-lg mb-4">
          <p class="text-lg text-indigo-300 italic">
            <p class="text-xl font-mono text-white">
          </div>

        <div class="grid grid-cols-1 xl:grid-cols-2">
          <!-- Аналогия 1 -->
          <div class="bg-slate-800 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 border
            rder border-emerald-500 shadow-md">
              Аналогия 1: Автомагистрали
            </div>
            <p class="text-slate-300 text-sm mb-4">
              евле долететь из Москвы в Париж, если мы сд
```



```
<p class="text-slate-300 text-sm">Позволяет найти
  -mono text-amber-300 bg-black/30 px-1 rounded">
  у.</p>
</div>

<!-- Аналогии -->
<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
  <div class="bg-slate-800 p-6 rounded-lg border bo
    <div class="absolute -top-3 left-4 bg-slate-700
      emerald-500 shadow-md">
      Аналогия 1: Паутина (Бор)
    </div>
    <p class="text-slate-300 text-sm mb-4">Представ
      мы двигаемся по веткам. Если нужного продолж
      ("нити страховки") в самый длинный из извест
    </div>

    <div class="bg-slate-900 p-6 rounded-lg border-2 l
      <div class="absolute -top-3 left-4 bg-rose-900
        -500 shadow-md">
        Аналогия 2: Матёшка (Терминальные)
      </div>
      <p class="text-slate-300 text-sm mb-4">Представ
        нашли и "he", потому что оно спрятано внутри
        минальные ссылки</b> (term_link) – прямые мос
      <p>
    </div>
  </div>

<details class="bg-slate-900/40 rounded-lg border b
  <summary class="p-4 font-bold text-slate-400 outl
    pen:border-slate-700/50">
    Внутренняя структура автомата и код BFS (Скры
  </summary>
  <div class="p-5 text-sm text-slate-300 space-y-4">
    <p>Каждая вершина имеет таблицу переходов <code>
      ссылку <code>term_link</code> (ближайшая мен
    <div class="bg-black/50 p-4 rounded-lg font-mono
```

```

        </div>
      </details>
    </div>
  </section>`
  },
  {
    id: "alg-map",
    title: "Алгоритмы на карте",
    type: "html",
    category: "Графы",
    content: `<section id="alg-map-content" class="mb-12">
      <div class="flex items-center mb-6">
        <span class="bg-purple-600 text-white px-4 py-1">
          <h2 class="text-3xl font-bold text-white">Алгоритмы на карте
        </span>
      </div>

      <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border">
          <h3 class="text-xl font-bold text-purple-400">Алгоритмы на карте
          <p class="text-slate-300 text-sm mb-4">Когда мы имеем дело с графами, которые могут
            содержать большие дробные числа (долгота и широта), мы часто хотим
            использовать целые числа, например, от 0 до N, и уже по ним строим графы.</p>
        </div>
      </div>
    </div>
  </section>`
  }
];

```

ФАЙЛ 22/82: src/data/quizzes.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 106 | РАЗМЕР: 1.5 КБ

```

export interface QuizQuestion {
  question: string;
  options: string[];
  correctIndex: number;
}

```

```

    ],
    "bridges-code": [
        {
            question: "После DFS у нас  $low[u] = 4$ , а  $tin[v] =$ 
options: [
    "Да, так как  $low[u] (4) > tin[v] (2)$ ",
    "Нет,  $low[u]$  должно быть меньше или равно",
    "Только если граф связный и неориентированный"
],
            correctIndex: 0,
            explanation: "Условие моста:  $low[u] > tin[v]$ . В н
        },
        {
            question: "Для чего нужен массив  $low[v]$ ?",
options: [
    "Для того чтобы было больше переменных в коде",
    "Чтобы знать минимальный  $tin$  предка, до которого
    "Чтобы хранить глубину рекурсии"
],
            correctIndex: 1,
            explanation: " $low[v]$  хранит минимальное время входа
поиска мостов и точек сочленения."
        },
        {
            question: "Какова временная сложность алгоритма п
options: [
    " $O(V^2)$  — ищем мосты в квадрате",
    " $O(V + E)$  — каждый элемент один раз",
    " $O(2^V)$  — полный перебор"
],
            correctIndex: 1,
            explanation: "DFS обходит каждую вершину и каждое
        }
    ],
    "planarity-euler-formula": [
        {
            question: "Планарный граф имеет  $V=6$ ,  $E=12$ . Возмо
options: [

```



```

id: "sparse-table",
title: "2. Двумерная разреженная матрица (Sparse Table)",
type: "html",
description: "Разреженная таблица для идемпотентных операций",
category: "Продвинутые структуры",
content: `
<section id="sparse-table" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-indigo-600 text-white px-4 py-1 rounded">
      <h2 class="text-2xl sm:text-3xl font-bold text-white">
    </div>
    <div class="space-y-8">
      <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
        <h3 class="text-xl font-bold text-blue-400">
          <div class="bg-slate-900 p-4 rounded-lg mb-4">
            <p class="text-lg text-blue-300 italic">
              <p class="text-xl font-mono text-white">
                ух отрезков:  $\min(ST[L][k], ST[R - 2^k + 1][k])$ 
            </div>
            <div class="grid grid-cols-1 xl:grid-cols-2">
              <div class="bg-slate-800 p-6 rounded-lg">
                <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
                  Аналогия 1: Школьные линейки
                </div>
                <p class="text-slate-300 text-sm mb-4">
                  зок длины 7. Ты берешь две заготовленные заготовки
                  ь отрезок длины 7 (перекрывая друг друга по 1 единицу)
                  ничего складывать, просто пересекаем куски.
                </div>
                <div class="bg-slate-900 p-6 rounded-lg">
                  <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
                    Аналогия 2: Ступени
                  </div>
                  <p class="text-slate-300 text-sm mb-4">
                    ужно покрыть отрезок, мы берем две самые большие
                  </div>

```

```

<div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
    Аналогия 1: Удар катаной (Sp
</div>
<p class="text-slate-300 text-sm mb
по значению X. Все, кто меньше X улетают в.
</div>
<div class="bg-slate-900 p-6 rounded-lg
    <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
    Аналогия 2: Слияние капель (M
</div>
<p class="text-slate-300 text-sm mb
вого). Мы просто смотрим на корни: у кого Y
как потомок.</p>
</div>
</div>
<details class="bg-slate-900/40 rounded-lg l
    <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
    Код (Скрыто)
</summary>
<div class="p-5 text-sm text-slate-300
    <pre class="bg-slate-950 p-4 rounded
pair<Node*, Node*> split(Node* t, int x) {
    if (!t) return {nullptr, nullptr};
    if (t->val <= x) {
        auto [L, R] = split(t->right, x);
        t->right = L;
        return {t, R};
    } else {
        auto [L, R] = split(t->left, x);
        t->left = R;
        return {L, t};
    }
}</pre>
</div>
</details>

```

```

        </div>
        <p class="text-slate-300 text-sm mb-4">
и становиться кривым как бамбук. Но как тол
        </p>
    </div>
</div>
<details class="bg-slate-900/40 rounded-lg border
    <summary class="p-4 font-bold text-slate-300
-b border-slate-700/50">
        Код (Скрыто)
    </summary>
    <div class="p-5 text-sm text-slate-300">
        <pre class="bg-slate-950 p-4 rounded">
// Zig, Zag
function rotate(x) {
    let p = x.parent;
    if (p.left === x) { // Right rotate
        p.left = x.right;
        x.right = p;
    } else { // Left rotate
        p.right = x.left;
        x.left = p;
    }
}
}</pre>
        </div>
    </details>

    <div class="bg-indigo-950/60 p-6 rounded-xl">
        <h4 class="text-lg font-bold text-indigo-300">
            Как это работает?
        </h4>
        <div class="space-y-4 text-sm text-slate-300">
            <div>
                <strong class="text-white block">
                    Корень?</strong>
                <p>Оно поднимет узел, на котором
                    мы стоим, к корню. За счет сдвига этого листа в корень, д
                </p>
            </div>

```

```

        <p class="text-lg text-emerald-300 italic">
        <p class="text-xl font-mono text-white">
, y1...y2) = P[x2][y2] - P[x1-1][y2] - P[x2
</div>
<div class="grid grid-cols-1 xl:grid-cols-2"
    <div class="bg-slate-800 p-6 rounded-lg"
        <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
            ≈ Аналогия 1: Вырезание прямоу
        </div>
        <p class="text-slate-300 text-sm mb
з него маленький целевой прямоугольник в це
ый верхний угол отрезается дважды! Поэтому
        </div>
    </div>
</div>
</section>`
    }
];

```

ФАЙЛ 24/82: src/data/tickets/ticket14_20.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 248 | РАЗМ

```

import { Chapter } from '../types';

export const tickets14to20: Chapter[] = [
  {
    id: "dijkstra",
    title: "14. Графы. Поиск кратчайшего пути. Дейкстра",
    type: "html",
    category: "Графы. Пути",
    content: `
<section id="dijkstra-content" class="mb-12 scroll-mt-12"
  <div class="flex items-center mb-6 flex-wrap gap-3">

```



```

type: "html",
category: "Графы. Пути",
content: `
<section id="bellman-ford-content" class="mb-12 scroll-m-12">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
      <h2 class="text-2xl sm:text-3xl font-bold text-white">
        </div>
      <div class="space-y-8">
        <div class="bg-slate-700/50 p-6 rounded-xl border">
          <h3 class="text-xl font-bold text-rose-400">
            <div class="bg-slate-900 p-4 rounded-lg mb-4">
              <p class="text-lg text-rose-300 italic">
                <p class="text-xl font-mono text-white">
              </div>
            <div class="grid grid-cols-1 xl:grid-cols-2">
              <div class="bg-slate-800 p-6 rounded-lg">
                <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
                  ♂ Аналогия 1: Параноик
                </div>
                <p class="text-slate-300 text-sm mb-4">
еряет ВСЕ возможные дороги V-1 раз подряд.
              </div>
              <div class="bg-slate-900 p-6 rounded-lg">
                <div class="absolute -top-3 left-4 border
border-rose-500 shadow-md">
                  Отрицательный цикл
                </div>
                <p class="text-slate-300 text-sm mb-4">
опали во временную петлю.</p>
              </div>
            </div>
          <div id="slot-bellman-viz" class="mt-8"></div>
        </div>
      </div>
    </div>
  </section>`
},

```

```

        </ul>
      </div>
    </div>
  </div>
</section>`
  },
  {
    id: "johnson-algo",
    title: "17. Графы. Алгоритм Джонсона (Фокус с перев.",
    type: "html",
    category: "Графы. Пути",
    content: `
<section id="johnson-algo" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-amber-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-xl font-mono text-white">
      </div>
      <div class="grid grid-cols-1 gap-6 mb-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 right-4 bottom-4
er border-amber-500 shadow-md">
            ⚙ Суть перевзвешивания
          </div>
          <p class="text-slate-300 text-sm mb-4">
            Дейкстра (Билет 14) быстрая, но
            ает очень медленно. Мы хотим сделать <b>Джонсо
            е пути остались теми же.
          </p>
          <ol class="text-sm text-slate-300 space-y-4">
            <li>Добавляем <b>фиктивную вершину</b>
            <li>Запускаем от неё <b>медленный поиск в ширину</b>
            <li>Добавляем <b>«потенциалы»</b> <code class="text-pink-400"

```

```
        с самых дешевых дорог в стране". Он строит
        единую сеть.</p>
    </div>
</div>
</div>
</div>
</section>`
},
{
    id: "mst-prima",
    title: "19. Графы. Остовное дерево. Прима",
    type: "html",
    category: "Графы. Остовные",
    content: `
<section id="mst-prima" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">
            <h2 class="text-2xl sm:text-3xl font-bold text-white">
                </div>
            <div class="space-y-8">
                <div class="bg-slate-700/50 p-6 rounded-xl border">
                    <h3 class="text-xl font-bold text-emerald-400">
                        <div class="bg-slate-900 p-4 rounded-lg mb-4">
                            <p class="text-lg text-emerald-300 italic">
                                <p class="text-xl font-mono text-white">
                                    которое торчит из посещенных в непосещенных
                                </div>
                            <div class="grid grid-cols-1 gap-6">
                                <div class="bg-slate-800 p-6 rounded-lg">
                                    <div class="absolute -top-3 left-4 border
                                        order border-emerald-500 shadow-md">
                                        ✦ Аналогия: Заражение вирусом
                                    </div>
                                    <p class="text-slate-300 text-sm mb-4">
                                        ли вирус). Мы стартуем из одного города, и
                                        Сеть растет только из одного связного куска
                                    </div>
                                </div>
                            </div>
                        </div>
                    </div>
                </div>
            </div>
        </div>
    </div>
`
```

```
];
```

ФАЙЛ 25/82: src/data/tickets/ticket21_24.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 262 | РАЗМ

```
import { Chapter } from '../..types';
```

```
export const tickets21to24: Chapter[] = [
```

```
{
```

```
  id: "string-kmp",
```

```
  title: "21. Префикс-функция (π) — Взгляд назад (КМП
```

```
  type: "html",
```

```
  category: "Строки",
```

```
  content: `
```

```
<section id="string-kmp" class="mb-12 scroll-mt-10">
```

```
  <div class="flex items-center mb-6 flex-wrap gap-3">
```

```
    <span class="bg-indigo-600 text-white px-4 py-1">
```

```
    <h2 class="text-2xl sm:text-3xl font-bold text-v
```

```
</div>
```

```
<div class="space-y-8">
```

```
  <div class="bg-slate-700/50 p-6 rounded-xl border
```

```
    <h3 class="text-xl font-bold text-blue-400">
```

```
    <div class="bg-slate-900 p-4 rounded-lg mb-4">
```

```
      <p class="text-lg text-blue-300 italic">
```

```
      <p class="text-xl font-mono text-white">
```

```
сом, оканчивающимся в позиции i.</p>
```

```
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2">
```

```
  <!-- Аналогия 1 -->
```

```
  <div class="bg-slate-800 p-6 rounded-lg">
```

```
    <div class="absolute -top-3 left-4 b
```

```
  <div border-emerald-500 shadow-md">
```

```
    ♂ Аналогия 1: Поиск без отка
```

```
</div>
```

```

        /li>
        <li><b>abcabc</b>: Начало ("abc
e></li>
        <li><b>abcabcd</b>: "d" всё исп
"abca" vs "abcd"). Конец "abcd" не совпада
        </ul>
    </div>
</details>
<details class="bg-slate-900/40 rounded-lg l
    <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
        Код C++ (Скрыто)
    </summary>
    <div class="p-5 text-sm text-slate-300
        <pre class="bg-slate-950 p-4 rounde
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
    int j = pi[i-1];
    while (j > 0 && s[i] != s[j]) j = pi[j-1];
    if (s[i] == s[j]) j++;
    pi[i] = j;
}</pre>
        </div>
    </details>
</div>
</div>
</section>`
    },
    {
        id: "string-z-func",
        title: "22. Z-функция — Сдвиг и наложение",
        type: "html",
        category: "Строки",
        content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-indigo-600 text-white px-4 py-1">
        <h2 class="text-2xl sm:text-3xl font-bold text-v

```

```
<details class="bg-slate-900/40 rounded-lg border
  <summary class="p-4 font-bold text-slate-300
  -b border-slate-700/50">
```

Пример вычисления (Скрыто)

```
</summary>
<div class="p-5 text-sm text-slate-300 border
  <p>Тот же пример: <b>abcabcd</b></p>
  <ul class="list-disc pl-5 space-y-2">
    <li><code>Z[0]</code> – не считаем
    <li><code>Z[1]</code> (<b>bcabcd</b>
  [1] = 0</code></li>
    <li><code>Z[2]</code> (<b>cabcd</b>
  </li>
    <li><code>Z[3]</code> (<b>abcd</b>
  ? Нет, в начале 'a', тут 'd'). Совпало 3 бу
    <li><code>Z[4], Z[5], Z[6]</code>
  </li>
  </ul>
</div>
</details>
```

```
<details class="bg-slate-900/40 rounded-lg border
  <summary class="p-4 font-bold text-slate-300
  -b border-slate-700/50">
```

Код C++ (Скрыто)

```
</summary>
<div class="p-5 text-sm text-slate-300 border
  <pre class="bg-slate-950 p-4 rounded
int l = 0, r = 0;
for (int i = 1; i <= n; i++) {
  if (i <= r) z[i] = min(r - i + 1, z[i - 1]);
  while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
  if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
}</pre>
```

```
</div>
</details>
```

```
</div>
</div>
</section>
```

```

-500 shadow-md">
  Аналогия 2: Матёшка (Терминальные)
</div>
<p class="text-slate-300 text-sm mb-4">Представ
  нашли и "he", потому что оно спрятано внутри
</div>
</div>

<details class="bg-slate-900/40 rounded-lg border b
  <summary class="p-4 font-bold text-slate-400 ou
    r-slate-700/50">
    Код (Скрыто)
  </summary>
  <div class="p-5 text-sm text-slate-300 space-y-
    <pre class="bg-slate-950 p-4 rounded overfl
int get_link(int v) {
  if (t[v].link == -1) {
    if (v == 0 || t[v].p == 0) t[v].link = 0;
    else t[v].link = go(get_link(t[v].p), t[v].pch)
  }
  return t[v].link;
}

int go(int v, char c) {
  if (t[v].go[c] == -1) {
    if (t[v].next[c] != -1) t[v].go[c] = t[v].next[c];
    else t[v].go[c] = v == 0 ? 0 : go(get_link(v),
  }
  return t[v].go[c];
}</pre>
  </div>
</details>
</div>
</section>`
},
{
  id: "complexity-classes",
  title: "24. Классы сложности, сведение задач",

```

```
        </div>
      </div>
</section>`
  }
];
```

ФАЙЛ 26/82: src/data/tickets/ticket6_13.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 486 | РАЗМ

```
import { Chapter } from "../../types";

export const tickets6to13: Chapter[] = [
  {
    id: "graph-dfs-bfs",
    title: "6. Графы. Представление, DFS, BFS",
    type: "html",
    description: "Представление графов и базовые обходы",
    category: "Графы. База",
    content: `
<section id="graph-dfs-bfs" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">6
    <h2 class="text-2xl sm:text-3xl font-bold text-slate-900">Графы
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-slate-300">Представление
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-slate-300 italic">Граф — это структура данных, состоящая из
        <p class="text-xl font-mono text-white">
          (V + E).</p>
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2">
        <div class="bg-slate-800 p-6 rounded-lg">

```



```

        <li>Сильно связанные компоненты K
        <li>Поиск циклов и просто провер
    </ul>
</div>

<!-- Аналогия BFS -->
<div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
    <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
        BFS (Поиск в ширину) = Волна
    </div>
    <p class="text-slate-300 text-sm mb-0">
        <b>Что делает:</b> Концентрические волны
        Работает через Очередь (FIFO). Продвигается
    </p>
    <ul class="text-xs text-emerald-300">
        <li><b>Где используется:</b></li>
        <li>Кратчайший путь в Невзвешенном графе
        <li>Алгоритм Ахо-Корасик (сборка)
        <li>Алгоритм Форда-Фалкерсона / Поиск в ширину
        <li>Двунаправленный поиск для у
    </ul>
</div>
</div>

<details class="bg-slate-900/40 rounded-lg" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
    <summary class="p-4 font-bold text-slate-300" style="border: 1px solid #000; border-radius: 10px; padding: 5px;">
        -b border-slate-700/50">
        Код обходов C++ (Основа)
    </summary>
    <div class="grid grid-cols-1 md:grid-cols-2" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
        <pre class="bg-slate-950 p-3 rounded" style="border: 1px solid #000; border-radius: 10px; padding: 5px;">
// DFS (Рекурсия)
vector<bool> vis;
vector<vector<int>>> adj;

void dfs(int v) {
    vis[v] = true;

```

```

<div class="bg-slate-700/50 p-6 rounded-xl border
  <h3 class="text-xl font-bold text-blue-400
  <div class="bg-slate-900 p-4 rounded-lg mb-1
    <p class="text-lg text-blue-300 italic
    <p class="text-xl font-mono text-white"
  </p>
  </div>
  <div class="grid grid-cols-1 gap-6">
    <div class="bg-slate-800 p-6 rounded-lg
      <div class="absolute -top-3 left-4
        <div class="border border-emerald-500 shadow-md">
          Аналогия 1: Карта мира
        </div>
        <p class="text-slate-300 text-sm mb-1
          . Границы не пересекаются в одной точке по-
          ы не сливались цветами, Всегда можно всего
        </div>
      </div>
    </div>
  </div>
</div>
</section>`,
  },
  {
    id: "graph-components",
    title: "8. Графы. Компоненты связности",
    type: "html",
    description: "Нахождение кусков графа",
    category: "Графы. База",
    content: `
<section id="graph-components" class="mb-12 scroll-mt-1
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 r
    <h2 class="text-2xl sm:text-3xl font-bold text-
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl bord
      <h3 class="text-xl font-bold text-emerald-4
      <div class="bg-slate-900 p-4 rounded-lg mb-

```

```

</div>
<div class="grid grid-cols-1 xl:grid-cols-2"
  <div class="bg-slate-800 p-6 rounded-lg"
    <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
      Аналогия 1: Учеба в вузе
    </div>
    <p class="text-slate-300 text-sm mb
еская сортировка – это расписание, которое
>
  </div>
  <div class="bg-slate-900 p-6 rounded-lg"
    <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
      Ограничения: Циклы
    </div>
    <p class="text-slate-300 text-sm mb
получить опыт, нужна работа". Алгоритм выявл
  </div>
</div>

```

```

<details class="bg-slate-900/40 rounded-lg l
  <summary class="p-4 font-bold text-slate
-b border-slate-700/50">

```

Душный мод: Код на C++ (Скрыто)

```

</summary>
<div class="p-5 text-sm text-slate-300"
  <p class="text-indigo-400 font-bold"
  <div class="bg-slate-950 p-4 rounde
    <pre class="text-xs font-mono t

```

```

vector<int> adj[MAXN];
bool visited[MAXN];
vector<int> ans;

```

```

void dfs(int v) {
    visited[v] = true;
    for (int to : adj[v]) {
        if (!visited[to]) {

```

```
<div class="text-left font-mono text-sm">
  <p><strong class="text-white">Компонент
  н <i>ориентированного</i> графа, в котором
</div>
```

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
  <strong>«Это просто цикл?» – Нет!</strong>
</p>
```

```
<ul class="list-disc list-inside text-sm text-slate-300">
  <li>Если у тебя есть цикл <code>А → Б →
  <li>Если ты добавишь вершину <strong>Г</strong>,
  етыре вершины – одна большая SCC</strong>.
  <li>Правило: <b>Если два цикла имеют хо
</ul>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2">
  <div class="bg-slate-800 p-6 rounded-lg">
    <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
```

Аналогия: Интриги в офисе

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
  Сильная связность – это "клуб" с
  ни в одной SCC. Добавить курьера Гошу, котор
  йти наружу (к директору), но обратно в клуб
</p>
```

```
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg">
  <div class="absolute -top-3 left-4 border
border-rose-500 shadow-md">
```

Связь с Билетом 12

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
  <b>SCC (Билет 10)</b> склеивает
</br><br>
```

```
<b>Точки сочленения (Билет 12)</b>
```

```

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border"
    <h3 class="text-xl font-bold text-rose-400"
    <div class="bg-slate-900 p-4 rounded-lg mb-1"
      <p class="text-lg text-rose-300 italic"
      <p class="text-xl font-mono text-white"
и fup[v] > tin[u].</p>
    </div>
    <div class="grid grid-cols-1 gap-6">
      <div class="bg-slate-800 p-6 rounded-lg"
        <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
          Аналогия 1: Единственный мост
        </div>
        <p class="text-slate-300 text-sm mb
ЭТОТ МОСТ, ЭКОНОМИКОЙ ОБОИХ КУСКОВ ПРИДЕТ
        </div>
      </div>
    </div>
  </div>
</section>`,
  },
  {
    id: "graph-articulation",
    title: "12. Графы. Точки сочленения",
    type: "html",
    description: "Вершины, удаление которых убивает свя
    category: "Графы. Продвинутое",
    content: `
<section id="graph-articulation" class="mb-12 scroll-mt
  <div class="flex items-center mb-6 flex-wrap gap-3"
    <span class="bg-blue-600 text-white px-4 py-1 r
    <h2 class="text-2xl sm:text-3xl font-bold text-
  </div>

  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl bord
      <h3 class="text-xl font-bold text-rose-400

```

```

        </div>
        <p class="text-slate-300 text-sm mb-4">
            У тебя несколько компьютеров в одной
            абелем (мостом). Сами свитчи — это точки со
        </p>
    </div>
</div>

<details class="bg-slate-900/40 rounded-lg border-1
    <summary class="p-4 font-bold text-slate-400
    order-slate-700/50">
        Душный мод: Код и корень DFS (Скрыто)
    </summary>
    <div class="p-5 text-sm text-slate-300 space-y-4">
        <p class="text-blue-400 font-bold">Особый
        <p>
            Для стартовой вершины обхода правил
            то выше неё прыгать некуда). Поэтому для неё
            у него <strong>больше 1 независимого ребенка
        </p>
        <div class="bg-slate-950 p-4 rounded-lg border-1">
            <pre class="text-xs font-mono text-slate-300">
void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;
    int children = 0;

    for (int to : adj[v]) {
        if (to == p) continue;
        if (visited[to]) {
            low[v] = min(low[v], tin[to]);
        } else {
            dfs(to, v);
            low[v] = min(low[v], low[to]);
            if (low[to] >= tin[v] && p != -1)
                IS_CUTPOINT(v);
            ++children;
        }
    }
}

```

```
      <h2 class="text-2xl sm:text-3xl font-bold text-white">
    </div>
    <div class="space-y-8">
      <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
        <h3 class="text-xl font-bold text-indigo-400">
          <div class="bg-slate-900 p-4 rounded-lg mb-4">
            <p class="text-lg text-indigo-300 italic">
              <p class="text-xl font-mono text-white">
            </div>
            <div class="grid grid-cols-1 gap-6">
              <div class="bg-slate-800 p-6 rounded-lg">
                <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
                  Аналогия 1: Рисование не отрывая
                </div>
                <p class="text-slate-300 text-sm mb-4">
                  Если сходится нечетное число линий, значит, из
                  а везде должно быть четное число (зашел-вышел)
                </div>
              </div>
            </div>
          </div>
        </div>
      </div>
    </div>
  </section>,
  },
];
```

РАЗДЕЛ: ИНТЕРАКТИВНЫЕ КОМПОНЕНТЫ И СИМУЛЯТОРЫ

ФАЙЛ 27/82: src/components/ArticulationPointsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useState, useEffect } from "react";
import { Play, Pause, RotateCcw, Info } from "lucide-react";
```

```

});
edges.forEach((e) => {
  adj[e.u].push(e.v);
  adj[e.v].push(e.u);
});

interface StepInfo {
  desc: string;
  visited: Set<number>;
  ap: Set<number>;
  currentNode: number | null;
  tin: number[];
  low: number[];
}

export const ArticulationPointsViz: React.FC = () => {
  const [steps, setSteps] = useState<StepInfo[]>([]);
  const [currentStepIndex, setCurrentStepIndex] = useSt
  const [isPlaying, setIsPlaying] = useState(false);

  useEffect(() => {
    const newSteps: StepInfo[] = [];
    let timer = 0;
    const visited = new Set<number>();
    const ap = new Set<number>();
    const currentTin = Array(nodes.length).fill(-1);
    const currentLow = Array(nodes.length).fill(-1);

    const recordStep = (desc: string, u: number | null) => {
      newSteps.push({
        desc,
        visited: new Set(visited),
        ap: new Set(ap),
        currentNode: u,
        tin: [...currentTin],
        low: [...currentLow],
      });
    };
  });
};

```



```

        if (p === -1 && children > 1) {
            ap.add(v);
            recordStep(
                `Корень дерева DFS ${v} имеет >1 детей. Это
                v,
            );
        } else if (p === -1) {
            recordStep(
                `Корень дерева DFS ${v} имеет <=1 ребенка, он
                v,
            );
        }
    };
};

for (let i = 0; i < nodes.length; i++) {
    if (!visited.has(i)) {
        dfs(i);
    }
}

recordStep("Алгоритм завершен. Найдены все точки со
setSteps(newSteps);
}, []));

const currentStep = steps[currentStepIndex] || {
    desc: "",
    visited: new Set(),
    ap: new Set(),
    currentNode: null,
    tin: [],
    low: [],
};

useEffect(() => {
    let interval: NodeJS.Timeout;
    if (isPlaying && currentStepIndex < steps.length -
        interval = setInterval(() => {

```

```

        className="flex items-center justify-center
        ors"
    >
        <RotateCcw className="w-4 h-4" />
    </button>
</div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-3 g
    <div className="lg:col-span-2 relative h-96 lg:l
        <svg className="w-full h-full" viewBox="0 0 8
            {/* Edges */}
            {edges.map((edge, i) => {
                const u = nodes.find((n) => n.id === edge
                const v = nodes.find((n) => n.id === edge

            const isBridge =
                (edge.u === 3 && edge.v === 4) ||
                (edge.u === 4 && edge.v === 3);

            return (
                <line
                    key={"edge-" + i}
                    x1={u.x}
                    y1={u.y}
                    x2={v.x}
                    y2={v.y}
                    stroke={isBridge ? "#f43f5e" : "#475568"}
                    strokeWidth={isBridge ? "4" : "2"}
                    strokeDasharray={isBridge ? "8,4" : "0"}
                />
            );
        });
    }}}

    {/* Nodes */}
    {nodes.map((node) => {
        const isCurrent = currentStep.currentNode
        const isVisited = currentStep.visited.has

```

```

        >
            {node.label}
        </text>

        {isVisited && (
            <img alt="diamond icon" data-bbox="408 225 442 238"/>
            <text
                x={node.x}
                y={node.y - r - 15}
                textAnchor="middle"
                fill="#94a3b8"
                fontSize="10px"
            >
                tin:{" "}
                {currentStep.tin[node.id] >= 0
                    ? currentStep.tin[node.id]
                    : "-"}
            </text>
            <text
                x={node.x}
                y={node.y + r + 20}
                textAnchor="middle"
                fill="#94a3b8"
                fontSize="10px"
            >
                low:{" "}
                {currentStep.low[node.id] >= 0
                    ? currentStep.low[node.id]
                    : "-"}
            </text>
        </>
    )}
</g>
);
}}
</svg>
</div>

```

```

        .slice(0, currentStepIndex + 1)
        .reverse()
        .map((step, idx) => {
            <div
                key={idx}
                className={`text-xs p-2 rounded $
xt-slate-500`} `>
                >
                    {step.desc}
                </div>
            )})
        </div>
    </div>
</div>
<div className="mt-4 shrink-0">
    <div className="w-full bg-slate-800 h-2 roun
        <div
            className="bg-rose-500 h-full transition
            style={{
                width: `${(currentStepIndex / Math.ma
            }}
        />
    </div>
</div>
</div>
</div>
</div>
);
};

```

ФАЙЛ 28/82: src/components/BellmanFordViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```
import { useState, useEffect } from 'react';
```

```

    ];
    if (cycle) {
        return [...defaultEdges, { u: 'D', v: 'A', w: -6 }];
    } else {
        return [...defaultEdges, { u: 'D', v: 'A', w: -2 }];
    }
};

const edges = getEdges(hasCycle);

const getAdjList = (ed: Edge[]) => {
    const list: { [key: string]: { v: string; w: number } } = {};
    nodes.forEach(n => (list[n.id] = []));
    ed.forEach(e => list[e.u].push({ v: e.v, w: e.w }));
    return list;
};

const adjList = getAdjList(edges);

const codeLines = [
    { line: 1, text: 'function bellman_ford(graph, start)' },
    { line: 2, text: '    dist = {v: ∞}; dist[start] = 0;' },
    { line: 3, text: '    for i from 1 to V - 1: // |V| - 1 iterations' },
    { line: 4, text: '        for (u, v, weight) in E:' },
    { line: 5, text: '            if dist[u] + weight < dist[v]' },
    { line: 6, text: '                dist[v] = dist[u] + weight;' },
    { line: 7, text: '    for (u, v, weight) in E: // Проверка на отрицательный цикл' },
    { line: 8, text: '        if dist[u] + weight < dist[v]' },
    { line: 9, text: '            return "ОБНАРУЖЕН ОТРИЦАТЕЛЬНЫЙ ЦИКЛ"' },
    { line: 10, text: '    return dist' },
];

useEffect(() => {
    const simulationSteps: Step[] = [];
    let dist: { [key: string]: number } = { S: 0, A: 99 };
    let prev: { [key: string]: string | null } = { S: null };

    simulationSteps.push({
        iteration: 0, checkingEdge: null,
    });

```

```

    }

    if (simulationSteps[simulationSteps.length - 1].iteration === vCount) {
      let cycleFound = false;
      for (const edge of edges) {
        if (dist[edge.u] !== 999 && dist[edge.u] + edge.weight === dist[edge.v]) {
          cycleFound = true;
          simulationSteps.push({
            iteration: vCount, checkingEdge: { u: edge.u, v: edge.v },
            distances: { ...dist }, previous: { ...prev },
            log: `    ВНИМАНИЕ! Проверка цикла: Ребро ${edge.u} -> ${edge.v}`,
            activeCodeLine: 9,
          });
          break;
        }
      }
    }

    if (!cycleFound) {
      simulationSteps.push({
        iteration: vCount, checkingEdge: null,
        distances: { ...dist }, previous: { ...prev },
        log: '    Проверка завершена. Ни одно ребро не образует цикла.',
        activeCodeLine: 10,
      });
    }
  }

  setSteps(simulationSteps);
  setCurrentStepIndex(0);
  setIsPlaying(false);
}, [hasCycle]);

useEffect(() => {
  let timer: any = null;
  if (isPlaying && currentStepIndex < steps.length - 1) {
    timer = setTimeout(() => setCurrentStepIndex((p) => p + 1), 1000);
  } else if (currentStepIndex >= steps.length - 1) {
    setIsPlaying(false);
  }
}, [isPlaying, currentStepIndex, steps.length]);

```

```

    { /* Граф */ }
    <div className="w-full lg:w-2/3 bg-blue-950/20
      y-center min-h-[400px]">
      <div className="absolute top-3 left-3 bg-blue-
        ld shadow-sm">
        Итерация {currentStep.iteration} (Шаг {currentStep.step})
      </div>

      {currentStep.hasNegativeCycle && (
        <div className="absolute top-3 right-3 bg-rose-500/20
          animate-pulse shadow-lg shadow-rose-600/50">
            ⚠ Отрицательный цикл!
          </div>
        )}
    </div>

    <svg className="w-full h-auto max-h-[500px]" >
      <defs>
        <marker id="arrow-bf-normal" markerWidth="10" markerHeight="10"
          <path d="M0,0 L6,3 L0,6 z" fill="#64748b" />
        <marker id="arrow-bf-active" markerWidth="10" markerHeight="10"
          <path d="M0,0 L6,3 L0,6 z" fill="#f59e0b" />
        <marker id="arrow-bf-cycle" markerWidth="10" markerHeight="10"
          <path d="M0,0 L6,3 L0,6 z" fill="#e11d48" />
      </defs>

      {edges.map((edge, idx) => {
        const uNode = nodes.find((n) => n.id === edge.u);
        const vNode = nodes.find((n) => n.id === edge.v);
        const isChecking = currentStep.checkingEdges === idx;
        const isNeg = edge.w < 0;
        const isCyclePart = hasCycle && ((edge.u === 'D' || edge.v === 'D'));

        let strokeColor = '#475569';
        let marker = 'url(#arrow-bf-normal)';
        if (isChecking) { strokeColor = '#f59e0b'; }
        else if (currentStep.hasNegativeCycle && isNeg) {
          strokeColor = '#e11d48';
          marker = 'url(#arrow-bf-cycle)';
        }
        <path d={edge.d} stroke={strokeColor} stroke-width={2} />
      })}
    </svg>
  </div>
)

```

```

    { /* Список смежности */ }
    <div className="w-full lg:w-1/3 bg-emerald-950/
      <h4 className="text-lg font-bold text-emerald
      <div className="space-y-2 text-sm font-mono t
        {Object.keys(adjList).map(u => {
          <div key={u} className="flex flex-wrap g
            -colors">
              <span className="font-bold text-white
                {adjList[u].map((edge, idx) => {
                  const isChk = currentStep.checkingE
                  return (
                    <span key={idx} className={`px-2
                      edge.w < 0 ? 'bg-rose-900/80 text-rose-300
                        {edge.v} ({edge.w})
                      </span>
                    )
                  )
                })}
              </div>
            )
          )
        })}
      </div>
    </div>
  </div>
</div>

<div className="flex flex-col lg:flex-row gap-6">
  { /* Таблица расстояний */ }
  <div className="w-full lg:w-1/2 bg-rose-950/10
    <div>
      <h4 className="text-lg font-bold text-rose-
      <div className="grid grid-cols-4 gap-2 text
        {nodes.map(n => {
          const d = currentStep.distances[n.id];
          const isTgt = currentStep.checkingEdge
          return (
            <div key={n.id} className={`p-2 round
              : 'border-slate-700/60 bg-slate-800/40'}` }>
              <div className="text-xs text-slate
              <div className={`font-mono font-bo

```



```

    { id: 0, label: '0', x: 300, y: 50 },
    { id: 1, label: '1', x: 150, y: 150 },
    { id: 2, label: '2', x: 450, y: 150 },
    { id: 3, label: '3', x: 75, y: 250 },
    { id: 4, label: '4', x: 225, y: 250 },
    { id: 5, label: '5', x: 375, y: 250 },
    { id: 6, label: '6', x: 525, y: 250 },
  ];

const EDGES = [
  { from: 0, to: 1 }, { from: 0, to: 2 },
  { from: 1, to: 3 }, { from: 1, to: 4 },
  { from: 2, to: 5 }, { from: 2, to: 6 }
];

const ADJ: { [key: number]: number[] } = {
  0: [1, 2], 1: [3, 4], 2: [5, 6], 3: [], 4: [], 5: [],
};

interface Step {
  activeLine: number;
  queue: number[];
  visited: number[];
  currentNode: number | null;
  logs: string;
}

const generateBfsSteps = (): Step[] => {
  const steps: Step[] = [];
  const queue: number[] = [];
  const visited = new Set<number>();

  // Init
  steps.push({
    activeLine: 1,
    queue: [],
    visited: [],
    currentNode: null,
  });
};
```

```

        visited: Array.from(visited),
        currentNode: curr,
        logs: `Рассматриваем соседа: ${n}. Он еще не
    });

    visited.add(n);
    queue.push(n);

    steps.push({
        activeLine: 9,
        queue: [...queue],
        visited: Array.from(visited),
        currentNode: curr,
        logs: `Помечаем ${n} как посещенную и добавляем в очередь.
    });
    }
}
} else {
    steps.push({
        activeLine: 6,
        queue: [...queue],
        visited: Array.from(visited),
        currentNode: curr,
        logs: `Соседей нет или все уже посещены.`
    });
}
}

steps.push({
    activeLine: 12,
    queue: [],
    visited: Array.from(visited),
    currentNode: null,
    logs: "Очередь пуста. Алгоритм завершен!"
});

return steps;
};

```

```

    <button
      onClick={() => { setIsPlaying(false); setSt
      className="p-2 text-slate-400 hover:text-wh
      title="Сброс"
    >
      <RotateCcw className="w-5 h-5" />
    </button>
    <button
      onClick={() => setIsPlaying(p => !p)}
      className="p-2 text-blue-400 hover:text-blue
      title={isPlaying ? 'Пауза' : 'Воспроизведен
    >
      {isPlaying ? <Pause className="w-5 h-5" />
    </button>
    <button
      onClick={handleNext}
      disabled={stepIdx >= STEPS.length - 1}
      className="p-2 text-emerald-400 hover:text-e
      ed:hover:bg-transparent"
      title="Шаг вперед"
    >
      <StepForward className="w-5 h-5" />
    </button>
  </div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-2 g
  {/* Визуал графа */}
  <div className="bg-slate-800 border border-slate
    50px]">
    <svg className="w-full h-full min-h-[350px]" *
      {/* Рёбра */}
      {EDGES.map((e, idx) => {
        const n1 = NODES.find(n => n.id === e.from
        const n2 = NODES.find(n => n.id === e.to)
        const isVisited = step.visited.includes(n
        return (
          <line

```

```

        cx={n.x} cy={n.y}
        r="20"
        fill={fill}
        stroke={stroke}
        strokeWidth="3"
        className="transition-colors duration-300"
      />
      <text
        x={n.x} y={n.y}
        textAnchor="middle"
        dy=".3em"
        className="fill-white font-bold text-sm"
      >
        {n.label}
      </text>
    </g>
  );
}}}
</svg>
</div>

```

```

{/* Код и Очередь */}
<div className="flex flex-col gap-4">
  <div className="bg-slate-900 border border-slate-800 p-4">
    <div className="bg-slate-800 text-slate-400 p-4 font-mono">
      <span>bfs(graph, start)</span>
      <span className="text-blue-400">War {step}</span>
    </div>
    <div className="p-4">
      {[
        { line: 1, code: 'queue = Queue()' },
        { line: 2, code: 'queue.push(start); visit(start);' },
        { line: 3, code: '}' },
        { line: 4, code: 'while not queue.isEmpty() {',
          line2: 5, code: '  node = queue.pop()' },
        { line: 6, code: '  for neighbor in graph.neighbors(node) {',
          line2: 7, code: '    if not visited.get(neighbor):',
          line2: 8, code: '      queue.push(neighbor); visit(neighbor);',
          line2: 9, code: '    visited.add(neighbor);',
          line2: 10, code: '  }',
          line2: 11, code: '}' },
        { line: 12, code: 'return visited' },
      ]}
    </div>
  </div>

```

```
        </div>
      </div>
    </div>
  );
}
```

ФАЙЛ 30/82: src/components/ChapterImage.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import dijkstraImg from '../assets/dijkstra-kind.jpg';
import bellmanImg from '../assets/bellman-ford-truck.jpg';
import floydImg from '../assets/floyd-network.jpg';
```

```
const imageMap: Record<string, { src: string; alt: string }> = {
  dijkstra: {
    src: dijkstraImg,
    alt: 'Добрый Дейкстра',
    caption: 'Добрый — только позитив!',
    borderColor: 'border-blue-500/30',
    captionColor: 'text-blue-400',
  },
  'bellman-ford': {
    src: bellmanImg,
    alt: 'Фура по болоту',
    caption: 'Тяжёлая, но ей пофиг на ямы!',
    borderColor: 'border-rose-500/30',
    captionColor: 'text-rose-400',
  },
  floyd: {
    src: floydImg,
    alt: 'Все ко Всем Флойд',
    caption: 'Все связаны со всеми!',
    borderColor: 'border-emerald-500/30',
    captionColor: 'text-emerald-400',
  },
};
```



```

-----
import { useTermHints } from "../hooks/useTermHints";
import { TermPopover, type HintAnchor } from "../hints/TermPopover";
import { getViz } from "../vizRegistry";
import { ChapterNav } from "../ChapterNav";
import { downloadChapterHtml } from "../utils/exportHtml";

interface Props {
  chapter: Chapter;
  prev?: Chapter;
  next?: Chapter;
  index: number;
  total: number;
  onGo: (id: string) => void;
  onOpenSimulator: (vizId: string) => void;
}

const isTouch = () => typeof window !== "undefined" && "ontouchstart" in window;

export const ChapterView: React.FC<Props> = ({ chapter,
  const contentRef = useRef<HTMLDivElement>(null);
  const [anchor, setAnchor] = useState<HintAnchor | null>(null);
  const hideTimer = useRef<number | null>(null);
  const [saving, setSaving] = useState<"idle" | "work">("idle");

  useTermHints(contentRef, [chapter.id]);

  const cancelHide = useCallback(() => {
    if (hideTimer.current) {
      window.clearTimeout(hideTimer.current);
      hideTimer.current = null;
    }
  }, []);

  const scheduleHide = useCallback(() => {
    cancelHide();
    hideTimer.current = window.setTimeout(() => setAnchor(anchor), 1000);
  }, [cancelHide]);

```

```

const handleClick = (e: React.MouseEvent) => {
  const el = (e.target as HTMLElement).closest<HTMLElement>
  if (!el) return;
  e.preventDefault();
  if (anchor && anchor.termId === el.dataset.termId)
    else open(el);
};

const handleFocus = (e: React.FocusEvent) => {
  const el = (e.target as HTMLElement).closest<HTMLElement>
  if (el) open(el);
};

const viz = getViz(chapter.id);

return (
  <>
    <article className="chapter-body">
      /* панель загрузки: сохранить именно эту страницу */
      <div className="flex flex-wrap items-center gap-2">
        <button
          onClick={saveHtml}
          disabled={saving === "work"}
          title="Сохранить эту тему одним автономным файлом"
          className="flex items-center gap-1.5 text-x-small border-1px border-gray-300 hover:text-white hover:border-emerald-500"
        >
          {saving === "work" ? (
            <Loader2 className="w-3.5 h-3.5 animate-spin" />
          ) : saving === "done" ? (
            <Check className="w-3.5 h-3.5 text-emerald-500" />
          ) : (
            <Download className="w-3.5 h-3.5" />
          )}
          {saving === "done" ? "Файл сохранён" : "Скачать"}
        </button>
        <button
          onClick={() => window.print()}

```



```

        ate-300 hover:text-white hover:border-indigo
        >
        <Maximize2 className="w-3.5 h-3.5" /> P
        </button>
      </div>
      {viz.hint && <p className="text-xs text-sla
      <viz.Component />
    </section>
  }}

  <ChapterNav prev={prev} next={next} index={inde
</article>

<TermPopover
  anchor={anchor}
  onClose={() => setAnchor(null)}
  onOpenSimulator={(id) => {
    setAnchor(null);
    onOpenSimulator(id);
  }}
  onCardEnter={cancelHide}
  onCardLeave={scheduleHide}
/>
</>
);
};

```

ФАЙЛ 33/82: src/components/DfsBridgesSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```

import { useState, useEffect, useRef } from "react";
import { Undo, Play, Pause, SkipForward, RefreshCw } fr

```

```

interface CodeLine {
  line: string;

```

```

const GRAPH_NODES = [
  { id: 0, label: "A", x: 100, y: 120 },
  { id: 1, label: "B", x: 260, y: 120 },
  { id: 2, label: "C", x: 180, y: 200 },
  { id: 3, label: "D", x: 180, y: 300 },
  { id: 4, label: "E", x: 100, y: 370 },
  { id: 5, label: "F", x: 260, y: 370 },
  { id: 6, label: "G", x: 340, y: 200 }
];

const GRAPH_EDGES = [
  { u: 0, v: 1, key: "0-1" },
  { u: 1, v: 2, key: "1-2" },
  { u: 2, v: 0, key: "0-2" },
  { u: 2, v: 3, key: "2-3" },
  { u: 3, v: 4, key: "3-4" },
  { u: 4, v: 5, key: "4-5" },
  { u: 5, v: 3, key: "3-5" },
  { u: 1, v: 6, key: "1-6" }
];

const DFS_STEPS: DfsStep[] = [
  {
    currentNode: 0, visitedIndices: [0],
    tin: {0:1}, low: {0:1}, stack: [0], bridges: [],
    activeEdge: null, backEdge: null,
    log: "Входим в A. tin[A]=1, low[A]=1.",
    activeCodeLine: [0,1,2]
  },
  {
    currentNode: 1, visitedIndices: [0,1],
    tin: {0:1,1:2}, low: {0:1,1:2}, stack: [0,1], bridges: [],
    activeEdge: [0,1], backEdge: null,
    log: "Идём (A,B). В впервые. tin[B]=2, low[B]=2.",
    activeCodeLine: [4,5,7,10,11]
  },
  {
    currentNode: 6, visitedIndices: [0,1,6],

```

```

    activeCodeLine: [4,5,7,10,11]
},
{
    currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],
    tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
    activeEdge: [4,5], backEdge: null,
    log: "E→F. tin[F]=7, low[F]=7.",
    activeCodeLine: [4,5,7,10,11]
},
{
    currentNode: 5, visitedIndices: [0,1,6,2,3,4,5],
    tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
    activeEdge: null, backEdge: [5,3],
    log: "Из F видим D (посещён). Обратное ребро (F,D).",
    activeCodeLine: [7,8,9]
},
{
    currentNode: 4, visitedIndices: [0,1,6,2,3,4,5],
    tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
    activeEdge: null, backEdge: null,
    log: "Возврат F→E. low[E]=min(6,5)=5. (E,F) – не мо",
    activeCodeLine: [11,13,16,17]
},
{
    currentNode: 3, visitedIndices: [0,1,6,2,3,4,5],
    tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
    activeEdge: null, backEdge: null,
    log: "Возврат E→D. low[D]=min(6,5)=5. (D,E) – не мо",
    activeCodeLine: [11,13,16,17]
},
{
    currentNode: 2, visitedIndices: [0,1,6,2,3,4,5],
    tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
    activeEdge: null, backEdge: null,
    log: "Возврат D→C. low[5] = 5 > tin[2] = 4! (C,D) –",
    activeCodeLine: [13,14,15,16,17]
},
{

```

```

const step = DFS_STEPS[dfsIdx];

return (
  <div className="space-y-4">
    <div className="flex items-center justify-between">
      <h4 className="text-base font-bold text-white">
        <div className="flex items-center gap-1 bg-slate-700 rounded"
          <button onClick={() => { setDfsAuto(false); setDfsIdx(0); }}
            disabled={dfsIdx === 0}
            className="p-1.5 hover:bg-slate-700 rounded"
            <Undo className="h-3.5 w-3.5" />
          </button>
          <button onClick={() => setDfsAuto(!dfsAuto)}
            className={`px-2.5 py-1 rounded text-[10px] ${
              dfsAuto ? "bg-rose-600 text-white" : "bg-slate-700 text-white"
            }`}
            >{dfsAuto ? "<>" : "<"}<Pause className="h-3 w-3" />
          </button>
          <button onClick={() => { setDfsAuto(false); setDfsIdx(DFS_STEPS.length-1); }}
            disabled={dfsIdx === DFS_STEPS.length-1}
            className="p-1.5 hover:bg-slate-700 rounded"
            <SkipForward className="h-3.5 w-3.5" />
          </button>
          <button onClick={() => { setDfsAuto(false); setDfsIdx(0); }}
            className="p-1.5 hover:bg-slate-700 rounded"
            <RefreshCw className="h-3.5 w-3.5" />
          </button>
        </div>
      </div>
    <div className="grid grid-cols-1 md:grid-cols-5 gap-4">
      <div className="md:col-span-3 bg-slate-950 rounded"
        <div className="absolute inset-0 bg-[radial-gradient(circle,transparent_1px,slate-950_1px)]"
          </div>
        <svg className="w-full h-[280px] z-10 relative"
          <GRAPH_EDGES.map(e => {

```

```

        {vis && (
            <
                <rect x={n.x-18} y={n.y-28} width={
"transition-all duration-300" />
                <text x={n.x} y={n.y-20} textAnchor=
} l:{step.low[n.id]||"?"}</text>
            </>
        )}
    </g>;
    }}}
</svg>
<div className="flex items-center justify-bet
    <span>|||ar {dfsIdx+1}/{DFS_STEPS.length}</sp
    <div className="flex-1 mx-2 bg-slate-950 h-
        <div className="bg-indigo-600 h-full tran
    </div>
    </div>
</div>
</div>

{/* Code panel + stack */}
<div className="md:col-span-2 space-y-3">
    {/* Code panel */}
    <div className="bg-slate-950 rounded-lg borde
        <div className="bg-slate-900 px-3 py-1.5 te
        >
        <div className="p-2 overflow-x-auto">
            <pre className="text-[10px] font-mono lea
                {CODE_LINES.map(({cl, i}) => {
                    const hl = step.activeCodeLine.includ
                    return <div key={i}
                        className={`px-1 transition-all dur
                            hl ? "bg-indigo-600/20 text-indig
                        }`}>
                    <span className="text-slate-600 mr-1
                        {cl.line || ' '}
                    </div>;
                })}
    </div>

```

```
    </div>
  );
}
```

ФАЙЛ 34/82: src/components/DijkstraViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import { useState, useEffect } from 'react';

interface Node { id: string; x: number; y: number; name: string; }
interface Edge { u: string; v: string; w: number; }
interface Step {
  currentNode: string | null;
  checkingEdge: { u: string; v: string } | null;
  distances: { [key: string]: number };
  visited: { [key: string]: boolean };
  previous: { [key: string]: string | null };
  log: string;
  activeCodeLine: number;
}

export default function DijkstraViz() {
  const nodes: Node[] = [
    { id: 'A', x: 60, y: 150, name: 'Пиццерия (A)' },
    { id: 'B', x: 160, y: 60, name: 'Дом 1 (B)' },
    { id: 'C', x: 160, y: 240, name: 'Парк (C)' },
    { id: 'D', x: 340, y: 60, name: 'Офис (D)' },
    { id: 'E', x: 340, y: 240, name: 'Дом 2 (E)' },
    { id: 'F', x: 450, y: 150, name: 'ТЦ (F)' },
    { id: 'G', x: 250, y: 150, name: 'Метро (G)' },
  ];

  const edges: Edge[] = [
    { u: 'A', v: 'B', w: 4 },
    { u: 'A', v: 'C', w: 2 },
  ];
```

```

const visited: Record<string, boolean> = { A: false
const prev: Record<string, string | null> = { A: nu

simulationSteps.push({
  currentNode: null, checkingEdge: null,
  distances: { ...dist }, visited: { ...visited },
  log: ' Дейкстра готов к доставке. Начинаем из ПИ
  activeCodeLine: 2,
});

for (let iter = 0; iter < nodes.length; iter++) {
  let minD = 999;
  let u = null;
  for (const n of nodes) {
    if (!visited[n.id] && dist[n.id] < minD) {
      minD = dist[n.id];
      u = n.id;
    }
  }

  if (!u) break;

  visited[u] = true;
  simulationSteps.push({
    currentNode: u, checkingEdge: null,
    distances: { ...dist }, visited: { ...visited }
    log: ` Выбираем вершину ${u} с мин. непосещённ
    activeCodeLine: 5,
  });

  const neighbors = edges.filter(e => e.u === u);
  for (const edge of neighbors) {
    const v = edge.v;
    if (visited[v]) continue;

    simulationSteps.push({
      currentNode: u, checkingEdge: { u, v },
      distances: { ...dist }, visited: { ...visited

```

```

    }, [isPlaying, currentStepIndex, steps.length]);

    const currentStep = steps[currentStepIndex] || null;

    if (!currentStep) return <div className="text-white">

    return (
      <div className="bg-slate-800/90 p-6 rounded-2xl border
        {/* Шапка симулятора */}
        <div className="flex flex-wrap items-center justify
          <div className="flex items-center gap-3">
            <div className="p-2.5 bg-indigo-600 text-white
              <span className="text-2xl"> </span>
            </div>
            <div>
              <h3 className="text-2xl font-bold text-white
                <p className="text-sm text-indigo-300">Полн
              </div>
            </div>

          <div className="flex items-center gap-2">
            <button
              onClick={() => { setIsPlaying(false); setCu
              className="flex items-center gap-1 bg-slate
              ion-colors"
            >
              Сброс
            </button>
            <button
              onClick={() => setIsPlaying(!isPlaying)}
              className={`flex items-center gap-1 px-4 py
                isPlaying ? 'bg-amber-600 hover:bg-amber-5
              }`}
            >
              {isPlaying ? ' Пауза' : '▶ Пуск'}
            </button>
            <button
              onClick={() => { setIsPlaying(false); if (c

```



```

const isOptimal = currentStep.previous[ed

let marker = 'url(#arrow-dh-normal)';
if (isChecking) marker = 'url(#arrow-dh-a
else if (isOptimal) marker = 'url(#arrow-

return (
  <g key={idx}>
    <line
      x1={uNode.x} y1={uNode.y}
      x2={vNode.x} y2={vNode.y}
      stroke={isChecking ? '#f59e0b' : is
      strokeWidth={isChecking ? 5 : isOpt
      markerEnd={marker}
      className="transition-all duration-
      strokeDasharray={isChecking ? '4,4'
    />
    <circle
      cx={({uNode.x + vNode.x} / 2} cy={({u
      r="12" fill="#1e293b"
      stroke={isChecking ? '#f59e0b' : '#
    />
    <text
      x={({uNode.x + vNode.x} / 2} y={({uNo
      fill={isChecking ? '#f59e0b' : '#f8
      fontSize="11" fontWeight="bold" tex
    >
      {edge.w}
    </text>
  </g>
);
}})
}

{/* Вершины */}
{nodes.map((node) => {
  const isCurrent = currentStep.currentNode
  const isVisited = currentStep.visited[node
  const dist = currentStep.distances[node.i

```

```

        <div key={u} className={`p-3 rounded-lg
            isCurrentNode ? 'bg-emerald-900/60' :
            'text-slate-300'
            }`>
            <div className="flex items-center gap-2">
                <span className={`px-2 py-0.5 rounded-3
                    te-300'}`>
                    {u}
                </span>
                <span className="text-slate-400">→</span>
            </div>
            <div className="flex flex-wrap gap-2">
                {adjList[u].map((edge, idx) => {
                    const isCheckingThis = currentStep === idx
                    return (
                        <span key={idx} className={`px-2 py-0.5 rounded-3
                            isCheckingThis ? 'bg-amber-900/60 text-slate-300' :
                            'text-slate-300'
                        }`>
                            <span>{edge.v}</span><span class="font-size-0.8em">
                                <span className={isCheckingThis ? 'font-weight-bold' : ''}>
                                    {edge.w}</span>
                                </span>
                            </span>
                    );
                })}
            </div>
        </div>
    );
    }
}
</div>
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6">
    <div>
        <h4 className="text-lg font-bold text-rose-950">
            <div className="w-full lg:w-1/2 bg-rose-950/10">
                <div>
                    <h4 className="text-lg font-bold text-rose-950">

```

```

    { /* Код алгоритма */ }
    <div className="w-full lg:w-1/2 bg-slate-900/60" >
      <div>
        <h4 className="text-lg font-bold text-slate" >
          <div className="bg-slate-950/80 rounded-lg p-4" >
            {codeLines.map((item) => {
              const isActive = currentStep.activeCodeLine === item.line
              return (
                <div key={item.line} className={`py-1 px-2
                  ${isActive ? 'bg-indigo-900/80 text-white' : 'text-slate-400'}
                `}>
                  <span className="text-slate-600 select-none" >{item.line}</span>
                  <span className="whitespace-pre font-mono" >{item.code}</span>
                </div>
              )
            })}
          </div>
        </div>
      </div>
    </div>
  )
}

```

ФАЙЛ 35/82: src/components/DPVisualizer.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

```

import { useState, useEffect } from 'react';
import { Play, Pause, RotateCcw, ChevronLeft, ChevronRight } from 'lucide-react';

interface DPStep {
  id: number;
  title: string;
  code: string;
  active: boolean;
}

```

```

    if (n in memoMap) {
      generatedSteps.push({
        id: stepId++,
        n,
        action: 'memo_hit',
        desc: `✂ Кэш-хит! fibМемо(${n}) уже посчитано`,
        codeLine: 2,
        memoState: { ...memoMap }
      });
      return memoMap[n];
    }

    if (n <= 2) {
      generatedSteps.push({
        id: stepId++,
        n,
        action: 'base_case',
        desc: `Базовый случай: n <= 2 (n=${n}). Возвр`,
        codeLine: 3,
        memoState: { ...memoMap }
      });
      return 1;
    }

    generatedSteps.push({
      id: stepId++,
      n,
      action: 'call',
      desc: `Вычисляем рекурсивно: fibМемо(${n - 1})`,
      codeLine: 4,
      memoState: { ...memoMap }
    });

    const res1 = simulateFib(n - 1);
    const res2 = simulateFib(n - 2);
    memoMap[n] = res1 + res2;

```

```

    }, [isPlaying, currentStepIndex, steps, speed]));

const currentStep = steps[currentStepIndex] || null;
const memoState = currentStep?.memoState || {};

return (
  <div className="bg-slate-900 rounded-2xl border border-slate-800" >
    { /* Header & Controls */ }
    <div className="flex flex-col lg:flex-row lg:items-center" >
      <div>
        <div className="flex items-center gap-3 mb-2" >
          <div className="p-2 bg-emerald-500/10 border border-emerald-500" >
            <RefreshCw className="w-6 h-6" />
          </div>
          <h3 className="text-2xl font-extrabold text-white" >
            DP
          </h3>
          <p className="text-sm text-slate-400" >
            Смотри, как таблица DP кэширует результаты
          </p>
        </div>

        <div className="flex flex-wrap items-center gap-2" >
          <div className="flex items-center gap-2 bg-slate-800 p-2" >
            <span>N:</span>
            <select
              value={targetN}
              onChange={(e) => setTargetN(Number(e.target.value))}
              disabled={isPlaying}
              className="bg-transparent text-white font-mono" >
              <option value={4}>N = 4</option>
              <option value={5}>N = 5</option>
              <option value={6}>N = 6</option>
              <option value={7}>N = 7</option>
            </select>
          </div>

          <div className="flex items-center bg-slate-950 p-2" >

```

```

    { /* DP Table View */ }
    <div className="mb-8 bg-slate-950 p-4 md:p-6 rounded"
      <h4 className="text-xs font-bold uppercase tracking-wider"
        <div className="flex flex-wrap items-center gap-2"
          {Array.from({ length: targetN }, (_, i) => i + 1)}
            const isCached = num in memoState || num <= 2;
            const val = num <= 2 ? 1 : memoState[num];
            const isCurrent = currentStep?.n === num;

            return (
              <div
                key={num}
                className={`flex flex-col items-center justify-center gap-2
                  ${isCurrent
                    ? 'bg-emerald-950 border-emerald-500'
                    : isCached
                    ? 'bg-slate-900 border-emerald-500/50'
                    : 'bg-slate-900/50 border-slate-800'}
                `}
              >
                <span className="text-xs text-slate-400"
                  <span className={`text-xl md:text-2xl font-medium text-slate-400`}
                    {isCached ? val : '0'}
                  </span>
                  {num <= 2 && <span className="text-[9px] font-mono"
                </div>
              </div>
            );
          }}}
    </div>

```

```

    {currentStep && (
      <div className="mt-6 pt-4 border-t border-slate-800"
        <div className="text-sm font-medium text-slate-400"
          <span className="inline-block w-2 h-2 rounded-full"
            <span>{currentStep.desc}</span>
          </div>
          <div className="text-xs font-mono bg-slate-900 p-2"
            Текущий N: <strong className="text-emerald-400"

```

```

    <p className="text-sm text-slate-400">
      В классической рекурсии для  $N=\{targetN\}$  по
      вычисленное значение в таблице, мы момента.
    </p>
    <div className="p-4 bg-slate-900 rounded-xl text-slate-300">
      <span>Всего шагов симуляции: {steps.length}</span>
      <span className="text-emerald-400 font-bold">
    </div>
  </div>
})

{activeTab === 'code' && (
  <div className="bg-slate-950 rounded-2xl border">
    <div className="bg-slate-900 px-6 py-3 border">
      <span className="text-xs font-bold font-mono">
      <span className="text-xs text-emerald-400">
    </div>
    <pre className="p-6 font-mono text-sm space-x-2">
      {DP_CODE_LINES.map((line, idx) => {
        const lineNum = idx + 1;
        const isActive = currentStep?.codeLine === lineNum;
        return (
          <div
            key={lineNum}
            className={`flex items-center px-4 py-2 ${
              isActive
                ? 'bg-emerald-600/20 border-l-4 border-emerald-500'
                : 'text-slate-300 hover:bg-slate-900'
            }`}
          >
            <span className="w-8 text-xs text-slate-400">{lineNum}</span>
            <span>{line}</span>
          </div>
        );
      })}
    </pre>
    <div className="p-6 bg-slate-900/50 border-l-4 border-emerald-500">

```

```

    { id: 4, label: "E", x: 100, y: 250 }
  ];

const C1_EDGES = [
  { u: 0, v: 1 }, { u: 0, v: 2 },
  { u: 1, v: 3 }, { u: 2, v: 4 },
  { u: 1, v: 2 },
  { u: 3, v: 4 }
];

export function EulerSimulator() {
  const [c1Path, setC1Path] = useState<number[]>([]);
  const [c1VisitedEdges, setC1VisitedEdges] = useState<
  const [c1Msg, setC1Msg] = useState("Клики на вершину
  const [c1Done, setC1Done] = useState(false);

  const resetC1 = () => {
    setC1Path([]); setC1VisitedEdges([]); setC1Msg("Кли
  };

  const clickC1 = (vId: number) => {
    if (c1Done && c1Path.length > 0) return;
    if (c1Path.length === 0) {
      setC1Path([vId]);
      setC1Msg("Старт! Кликай соседние вершины, чтобы п
      return;
    }
    const last = c1Path[c1Path.length - 1];
    const edge = C1_EDGES.find(e => (e.u === last && e.v === vId));
    if (!edge) {
      setC1Msg("Нет ребра между этими вершинами! Выбери
      return;
    }
    const key = `${Math.min(last, vId)}-${Math.max(last, vId)}`;
    if (c1VisitedEdges.includes(key)) {
      setC1Msg("Это ребро уже пройдено! Эйлер не прощае
      setC1Done(true); return;
    }
  }
}

```



```

        const used = c1VisitedEdges.includes(key);
        return <line key={i} x1={u.x} y1={u.y} x2={v.x} y2={v.y}
            stroke={used ? "#6366f1" : "#475569"}
            strokeWidth={used ? 4 : 2}
            className="transition-all duration-300" />
    }}}
    {C1_NODES.map(n => {
        const isLast = c1Path[c1Path.length-1] === n.id;
        const visited = c1Path.includes(n.id);
        return <g key={n.id} className="cursor-pointer"
            {isLast && <circle cx={n.x} cy={n.y} r={11}
            <circle cx={n.x} cy={n.y} r={11}
                fill={isLast ? "#6366f1" : visited ? "#6366f1" : "#fff"}
                stroke={isLast ? "#fff" : visited ? "#6366f1" : "#fff"}
                strokeWidth={2.5}
                className="transition-all duration-200" />
            <text x={n.x} y={n.y+4} textAnchor="middle">{n.name}</text>
        </g>;
    })}
</svg>
<div className="absolute top-2 left-2 bg-slate-500 p-2"
    Pe6ëp: {c1VisitedEdges.length}/{C1_EDGES.length}</div>
</div>

<div className={`p-3 rounded-xl border text-sm ${
    c1Done && c1VisitedEdges.length === C1_EDGES.length
    ? "bg-emerald-950/40 border-emerald-500/30 text-emerald-50"
    : c1Done
    ? "bg-rose-950/40 border-rose-500/30 text-rose-50"
    : "bg-slate-900/50 border-slate-700 text-slate-50"}
}>
    {c1Msg}
</div>
</div>
);
}

```

```
        'Он жадный и считает, что каждый следующий шаг де.  
        закрытые вершины.',  
        'Отрицательные ребра создают гравитационный коллапс',  
        'Дейкстра просто не любил отрицательные балансы на  
    ],  
    correctAnswer: 1,  
    explanation: 'Именно! Дейкстра уверен: если ты прие  
        ля чего нужен алгоритм Беллмана-Форда.'  
},  
{  
    id: 3,  
    question: 'Какова ключевая фишка алгоритма Борувки  
    options: [  
        'Он использует меньше памяти благодаря коммунистич  
        'Он умеет работать с отрицательными весами без ци  
        'Он позволяет выполнять шаги слияния колхозов (ко  
        'Он был разработан в Токио и поэтому самый быстрый  
    ],  
    correctAnswer: 2,  
    explanation: 'Абсолютно верно! В Борувке каждая ком  
        лелить вычисления на GPU или многопроцессорных кл  
},  
{  
    id: 4,  
    question: 'В чем главная суть Динамического програм  
    options: [  
        'Писать код очень быстро, динамично нажимая на кл  
        'Запоминать (кешировать) результаты уже решенных  
        'Использовать динамическую типизацию как в Python  
        'Постоянно менять требования к проекту в процессе  
    ],  
    correctAnswer: 1,  
    explanation: 'Точно! Главный принцип ДП – мемоизаци  
},  
{  
    id: 5,  
    question: 'Какая структура данных идеально помогает  
    options: [  

```

```

const handleRestart = () => {
  setCurrentQIndex(0);
  setSelectedOption(null);
  setIsAnswered(false);
  setScore(0);
  setShowResults(false);
};

if (showResults) {
  return (
    <div className="bg-slate-900/90 border border-slate-800 py-12">
      <div className="w-20 h-20 bg-gradient-to-tr from-indigo-500 to-purple-600 shadow-xl shadow-indigo-500/30">
        <Award className="w-10 h-10 text-white" />
      </div>
      <h3 className="text-3xl font-bold text-white mb-4">Результаты</h3>
      <p className="text-slate-400 text-sm mb-6">Ты набрал {score} из {quizQuestions.length} вопросов</p>

      <div className="bg-slate-950 border border-slate-800 p-6">
        <p className="text-slate-400 text-sm uppercase font-medium">Оценка</p>
        <p className="text-5xl font-black text-indigo-300">{score}</p>
        <p className="text-slate-300 text-sm">
          {score === quizQuestions.length
            ? ' Идеально! Ты настоящий сеньор-раскрыватель! '
            : score >= 3
            ? ' Отличный результат! Главные концепции усвоены! '
            : ' Стоит повторить главы пособия и еще раз попробовать! '}
        </p>
      </div>

      <button
        onClick={handleRestart}
        className="inline-flex items-center gap-2 px-6 py-3 rounded-xl shadow-lg shadow-indigo-600/30"
      >
        <RotateCcw className="w-5 h-5" />
      </button>
    </div>
  );
}

```

```

        optionStyle = "bg-slate-800/40 border-s
    }
}

return (
  <div
    key={idx}
    onClick={() => handleSelectOption(idx)}
    className={"flex items-start gap-4 p-4"}
  >
    <div className="mt-0.5 shrink-0">
      {isAnswered && isCorrect ? (
        <CheckCircle2 className="w-6 h-6 text-emerald-500" />
      ) : isAnswered && isSelected && !isCorrect ? (
        <XCircle className="w-6 h-6 text-red-500" />
      ) : (
        <div className={"w-6 h-6 rounded-full border border-indigo-500 bg-indigo-500 text-white"}
          {String.fromCharCode(65 + idx)}
        </div>
      )}
    </div>
    <div className="flex-1 text-sm md:text-base">
      {option}
    </div>
  </div>
);
}}
</div>
</div>

{isAnswered && (
  <div className="bg-slate-950 border border-slate-800 p-4">
    <div className="flex items-center gap-2 mb-2">
      <AlertCircle className="w-4 h-4" />
      <span>Объяснение:</span>
    </div>
    <p className="text-slate-300 text-sm leading-relaxed">

```

```
const [steps, setSteps] = useState<Step[]>([]);
const [currentStepIndex, setCurrentStepIndex] = useSt
const [isPlaying, setIsPlaying] = useState(false);
```

```
const nodeNames = ['А 0', 'Б 1', 'В 2', 'Г 3', 'Д 4',
const nodesSvg = [
  { id: 0, name: 'Аэропорт 0', x: 100, y: 60 },
  { id: 1, name: 'Аэропорт 1', x: 250, y: 60 },
  { id: 2, name: 'Аэропорт 2', x: 400, y: 60 },
  { id: 3, name: 'Аэропорт 3', x: 100, y: 180 },
  { id: 4, name: 'Аэропорт 4', x: 400, y: 180 },
  { id: 5, name: 'Аэропорт 5', x: 175, y: 280 },
  { id: 6, name: 'Аэропорт 6', x: 325, y: 280 },
];
```

```
const initialMatrix = [
  [0, 4, 999, 2, 999, 999, 999],
  [999, 0, 3, 999, 999, 3, 999],
  [999, 999, 0, 999, 2, 999, 999],
  [999, 999, 999, 0, 4, 2, 999],
  [999, 999, 999, 999, 0, 999, 1],
  [999, 999, 999, 999, 999, 0, 5],
  [999, 1, 999, 999, 999, 999, 0]
];
```

```
const adjList: { [key: number]: { v: number; w: numbe
  0: [{ v: 1, w: 4 }, { v: 3, w: 2 }],
  1: [{ v: 2, w: 3 }, { v: 5, w: 3 }],
  2: [{ v: 4, w: 2 }],
  3: [{ v: 4, w: 4 }, { v: 5, w: 2 }],
  4: [{ v: 6, w: 1 }],
  5: [{ v: 6, w: 5 }],
  6: [{ v: 1, w: 1 }],
};
```

```
const codeLines = [
  { line: 1, text: 'function floyd_warshall(matrix, V
  { line: 2, text: '    dist = copy(matrix)' },
```

```

        activeCodeLine: 7,
      });
    }
  }
}

simulationSteps.push({
  k: 7, i: -1, j: -1, matrix: dist.map(r => [...r])
  log: ' Все пары отработаны. Алгоритм Флойда завершён.'
  activeCodeLine: 8,
});

setSteps(simulationSteps);
setCurrentStepIndex(0);
setIsPlaying(false);
}, []));

useEffect(() => {
  let timer: any = null;
  if (isPlaying && currentStepIndex < steps.length - 1) {
    timer = setTimeout(() => setCurrentStepIndex((p) => p + 1), 1000);
  } else if (currentStepIndex >= steps.length - 1) {
    setIsPlaying(false);
  }
  return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const currentStep = steps[currentStepIndex] || null;
if (!currentStep) return <div>Загрузка...</div>;

return (
  <div className="bg-slate-800/90 p-6 rounded-2xl border border-slate-700">
    <div className="flex flex-wrap items-center justify-center gap-4">
      <div className="flex items-center gap-3">
        <div className="p-2.5 bg-emerald-600 text-white rounded-md">
          <span className="text-2xl"> </span>
        </div>
      </div>
    </div>
  </div>
);

```

```

const vNode = nodesSvg.find((n) => n.id === v)
const isTarget = currentStep.i === u && currentStep.j === v
const isVia = (currentStep.i === u && currentStep.j === v)
let strokeColor = '#475569'; let markerEnd = null
if (isTarget) { strokeColor = '#10b981' }
else if (isVia) { strokeColor = '#818cf8' }
return (
  <g key={`$${u}-${edge.v}`}>
    <line x1={uNode.x} y1={uNode.y} x2={vNode.x} y2={vNode.y}
      markerEnd={markerEnd} strokeDasharray={isTarget ? [5, 5] : [1, 3]}
      strokeColor={strokeColor} strokeWidth={isTarget ? 2 : 1}>
    <circle cx={({uNode.x + vNode.x} / 2) cy={({uNode.y + vNode.y} / 2)}
      r={isVia ? 8 : 4} strokeColor={strokeColor} strokeDasharray={isTarget ? [5, 5] : [1, 3]}
      stroke={strokeColor} strokeWidth={isTarget ? 2 : 1}>
    <text x={({uNode.x + vNode.x} / 2) y={({uNode.y + vNode.y} / 2) + 8}
      fill={strokeColor} fontSize="10" fontWeight="bold" text={edge.v}>
    </text>
  </g>
);
});
}}}
{nodesSvg.map((node) => {
  const isK = currentStep.k === node.id;
  const isI = currentStep.i === node.id;
  const isJ = currentStep.j === node.id;
  let fill = '#1e293b'; let stroke = '#64748f';
  if (isK) { fill = '#4f46e5'; stroke = '#a8a2c0' }
  else if (isI) { fill = '#b45309'; stroke = '#c07040' }
  else if (isJ) { fill = '#047857'; stroke = '#2e8b57' }
  return (
    <g key={node.id} className="transition-colors">
      <circle cx={node.x} cy={node.y} r={isJ ? 4 : 2}
        className="transition-colors duration-200">
      <text x={node.x} y={node.y + 5} fill={stroke}
        font-size="10" font-weight="bold">
      <text x={node.x} y={node.y + 32} fill={stroke}
        font-size="10" font-weight="bold">
    </text>
  </g>
);
}}}
</svg>

```

```

<div className="flex flex-col lg:flex-row gap-6">
  {/* Матрица расстояний */}
  <div className="w-full lg:w-1/2 bg-rose-950/10">
    <div className="flex justify-between items-center">
      <h4 className="text-lg font-bold text-rose-950">
        Матрица расстояний
      </h4>
    </div>
    <div className="overflow-x-auto">
      <table className="w-full text-center border">
        <thead><tr className="bg-rose-950/40 text-rose-950">
          <th className="p-2 border-r border-rose-950">
            Шаг
          </th>
          {nodeNames.map((n, idx) => <th key={idx} className="p-2 border-r border-rose-950 font-bold" : ''>`>{n.split(' ')[0]} {idx + 1}</th>)}
        </tr></thead>
        <tbody className="text-xs font-mono">
          {currentStep.matrix.map((row, i) => (
            <tr key={i} className="border-b border-rose-950">
              <td className={`p-2 bg-rose-950/40 font-bold`} : ''>`>
                Шаг
              </td>
              {nodeNames[i].split(' ')[0]} {i + 1}
            </td>
            {row.map((val, j) => {
              const isUpd = currentStep.i === i
              const isVia = currentStep.k !== i
              const isDest = currentStep.j === j);
              let bg = 'bg-slate-900/40 text-slate-950';
              if (isUpd) bg = 'bg-emerald-900/80 text-emerald-950';
              else if (isVia) bg = 'bg-indigo-900/80 text-indigo-950';
              else if (i === j) bg = 'bg-slate-900/40 text-slate-950';
              return (<td key={j} className={bg} : ''>`>{val}</td>)});
            </td>
          </tr>)}
        </tbody>
      </table>
    </div>
  </div>
</div>

```



```

    { id: 'E', label: 'E', x: 150, y: 200 },
    { id: 'F', label: 'F', x: 250, y: 200 },
    { id: 'G', label: 'G', x: 350, y: 200 },
  ];

const edges: Edge[] = [
  { u: 'A', v: 'B' }, { u: 'A', v: 'C' },
  { u: 'B', v: 'D' }, { u: 'B', v: 'E' },
  { u: 'C', v: 'F' }, { u: 'C', v: 'G' },
];

const adj: Record<string, string[]> = {
  'A': ['B', 'C'],
  'B': ['A', 'D', 'E'],
  'C': ['A', 'F', 'G'],
  'D': ['B'],
  'E': ['B'],
  'F': ['C'],
  'G': ['C'],
};

type Action = {
  type: 'visit' | 'process' | 'backtrack';
  node: string;
  desc: string;
  queueOrStack?: string[];
  visitedNodes?: string[];
};

function generateDFSSteps(start: string) {
  const steps: Action[] = [];
  const vis = new Set<string>();
  const stack: string[] = []; // for visualization, j

  function dfs(v: string) {
    vis.add(v);
    stack.push(v);
    steps.push({ type: 'visit', node: v, desc: `Зависимости: ${vis.size} узла` });
  }
}

```

```

        q.push(u);
        steps.push({ type: 'visit', node: u, desc: `04. ${u}
        tack: [...q], visitedNodes: [...Array.from(
        }
    }
}

steps.push({ type: 'backtrack', node: '', desc: `04. ${u}
    om(vis)] });

return steps;
}

export function GraphTraversalViz() {
    const [mode, setMode] = useState<"dfs" | "bfs">("dfs");
    const [autoPlay, setAutoPlay] = useState(false);
    const [stepIdx, setStepIdx] = useState(0);

    const steps = useMemo(() => mode === 'dfs' ? generateSteps(
    graph, mode) : generateSteps(graph, mode), [mode]);

    useEffect(() => {
        setStepIdx(0);
        setAutoPlay(false);
    }, [mode]);

    useEffect(() => {
        if (autoPlay) {
            const t = setInterval(() => {
                setStepIdx(p => {
                    if (p >= steps.length - 1) {
                        setAutoPlay(false);
                        return p;
                    }
                    return p + 1;
                });
            }, 1200);
            return () => clearInterval(t);
        }
    }, [autoPlay, steps]);
}

```

```
500' : 'stroke-emerald-500') : 'stroke-slate-500'
    );
  }}}
}
```

```

{/* Nodes */}
{nodes.map(n => {
  const visited = isVisited(n)
  const current = isCurrent(n)

  let fillColor = "fill-slate-500"
  let strokeColor = "stroke-slate-500"
  let textColor = "fill-slate-500"
  let radius = 18;

  if (visited) {
    fillColor = mode === 'd' ? "fill-emerald-500" : "fill-slate-500"
    strokeColor = mode === 'd' ? "stroke-emerald-500" : "stroke-slate-500"
    textColor = "fill-white"
  }

  if (current && step.type !== 'd') {
    strokeColor = "stroke-amber-500"
    textColor = "fill-amber-500"
    radius = 22;
  }

  return (
    <g key={n.id} className={mode === 'd' ? 'node-current' : 'node-visited'}>
      <circle cx={n.x} cy={n.y} radius={radius}
        className={mode === 'd' ? 'stroke-amber-500' : 'stroke-slate-500'} />
      <text x={n.x} y={n.y} radius={radius}
        className={mode === 'd' ? 'fill-amber-500' : 'fill-slate-500'}>
        {n.label}
      </text>

      {/* Ripple for BFS */}
      {current && mode === 'd' && step.type !== 'd' &&
        <circle cx={n.x} cy={n.y} radius={radius + 2}
          className="stroke-slate-500" />
      }
    </g>
  )
})}
```

```

<div className="flex flex-col sm:flex-row gap-2" style={{width: '100%'}}
  { /* Controls */ }
  <div className="flex bg-slate-900 border border-slate-800 rounded-lg p-2 text-slate-400 hover:text-white hover:bg-slate-800" style={{width: '100%'}}
    <button onClick={() => {setAutoPlay(!autoPlay)}} style={{width: '50%'}}
      <Undo strokeWidth={3} size={16} />
    </button>
    <button onClick={() => setAutoPlay(!autoPlay)} style={{width: '50%'}}
      justify-center min-w-[80px] ${mode === 'default' ? 'text-slate-400' : 'text-white'}
    </button>
    {autoPlay ? <><Pause size={16} /> : <><Play size={16} />}
  </div>
  <div className="flex justify-between gap-2" style={{width: '100%'}}
    <button onClick={() => {setAutoPlay(!autoPlay)}} style={{width: '50%'}}
      = steps.length - 1} className="p-2 text-slate-400 hover:bg-transparent"
    <SkipForward strokeWidth={3} size={16} />
  </div>
  <div className="flex justify-between gap-2" style={{width: '100%'}}
    <button onClick={() => {setAutoPlay(!autoPlay)}} style={{width: '50%'}}
      g-slate-800 rounded-lg ml-2 border-l border-slate-800
    <RefreshCw strokeWidth={3} size={16} />
  </div>
</div>

{ /* Description Log */ }
<div className="flex-1 bg-slate-900/50 border border-slate-800 rounded-lg p-2" style={{width: '100%'}}
  <div className="text-[10px] text-slate-400" style={{width: '100%'}}
    {steps.length}</div>
    <div className="text-sm text-slate-400" style={{width: '100%'}}
      {step.desc}
    </div>
  </div>
</div>
</div>
);
}

```

```

        const r = 2 * idx + 2;
        if (l < n && a[l].priority > a[largest].priority) l = l;
        if (r < n && a[r].priority > a[largest].priority) l = r;
        if (largest !== idx) {
            [a[largest], a[idx]] = [a[idx], a[largest]];
            idx = largest;
        } else break;
    }
    return a;
}

function heapInsert(arr: Patient[], item: Patient): Patient {
    return siftUp([...arr, { ...item }]);
}

// Position in tree layout
function nodePos(index: number): { x: number; y: number } {
    const level = Math.floor(Math.log2(index + 1));
    const posInLevel = index - (Math.pow(2, level) - 1);
    const count = Math.pow(2, level);
    const x = ((posInLevel + 0.5) / count) * 100;
    const y = 10 + level * 24;
    return { x, y };
}

// Priority presets for realistic ER scenarios
const PATIENT_PRESETS = [
    { name: 'Иван (Инфаркт)', symptom: ' Боли в сердце', emoji: '👤🔴' },
    { name: 'Маша (Перелом)', symptom: ' Открытый перелом', emoji: '👤🔴' },
    { name: 'Кот Барсик (Укус)', symptom: ' Укусил шмеля', emoji: '🐈🔴' },
    { name: 'Семён (Температура)', symptom: ' Жар 39.5', emoji: '👤🔴' },
    { name: 'Оля (Порез)', symptom: ' Глубокий порез', emoji: '👤🔴' },
    { name: 'Пётр (Кашель)', symptom: ' Простуда', emoji: '👤🔴' },
];

const INITIAL_PATIENTS: Patient[] = (() => {
    let arr: Patient[] = [];
    const start = [

```

```

const finalEmoji = preset.emoji;
const finalSymptom = preset.symptom;

const newPatient: Patient = {
  id: `p${uid++}`,
  name: finalName,
  emoji: finalEmoji,
  symptom: finalSymptom,
  priority,
  animState: 'in',
};

const nextHeap = heapInsert(heap, newPatient).map(x => x);
// Mark specifically this new patient as 'in' in the heap
const target = nextHeap.find(x => x.priority === priority);
const marked = nextHeap.map(x => x.id === (target?.id));

setHeap(marked);
addLog(` Поступил пациент: ${finalName} с тяжестью ${priority}`);

setTimeout(() => {
  setHeap(nextHeap);
  setBusy(false);
}, 500);
}, [busy, heap]);

const handleRandomInsert = () => {
  const preset = PATIENT_PRESETS[Math.floor(Math.random() * PATIENT_PRESETS.length)];
  const randomShift = Math.floor(Math.random() * 10);
  const prio = Math.max(10, Math.min(99, preset.priority + randomShift));
  insertPatient(preset.name, prio);
};

const handleCustomInsert = (e: React.FormEvent) => {
  e.preventDefault();
  if (!customName.trim()) return;
  insertPatient(customName, customPriority);
  setCustomName('');
};

```

```

        setHealing(null);
        setBusy(false);
    }, 600);
    }, 800);

    }, 350);
    }, 650);
}, [busy, heap]);

const reset = () => {
    setHeap(INITIAL_PATIENTS.map(x => ({ ...x, animStat: 0 })));
    setLog([' База данных скорой помощи перезагружена.']);
    setHealing(null);
    setTimeout(() => setHeap(INITIAL_PATIENTS.map(x => ({ ...x, animStat: 0 } ))), 1000);
};

// --- Render binary tree lines ---
const edges = heap.flatMap((_, idx) => {
    const res: React.ReactElement[] = [];
    const p = nodePos(idx);
    const l = 2 * idx + 1;
    const r = 2 * idx + 2;
    if (l < heap.length) {
        const lp = nodePos(l);
        res.push(
            <line key={`el${idx}`}
                x1={` ${p.x}%`} y1={` ${p.y + 4}%`}
                x2={` ${lp.x}%`} y2={` ${lp.y - 4}%`}
                stroke="#475569" strokeWidth="2"
            />
        );
    }
    if (r < heap.length) {
        const rp = nodePos(r);
        res.push(
            <line key={`er${idx}`}
                x1={` ${p.x}%`} y1={` ${p.y + 4}%`}
                x2={` ${rp.x}%`} y2={` ${rp.y - 4}%`}
            />
        );
    }
});

```

```

    {/* Добавить кастомного */}
    <form onSubmit={handleCustomInsert} className=""
      <input
        type="text"
        required
        value={customName}
        onChange={e => setCustomName(e.target.value)}
        placeholder="Имя пациента"
        className="flex-1 bg-slate-800 border border-
er-emerald-500 transition-colors placeholder
/>
    <div className="flex flex-col items-center min
      <span className="text-[9px] font-mono text-
    <input
      type="range"
      min="10"
      max="99"
      value={customPriority}
      onChange={e => setCustomPriority(Number(e
      className="w-20 accent-emerald-500 cursor
    />
  </div>
  <button
    type="submit"
    disabled={busy || !customName.trim() || heap
    className="bg-teal-600 hover:bg-teal-500 di
    px-4 rounded-xl shadow-lg active:scale-95 t
  >
    Везти
  </button>
</form>

  {/* Забрать самого тяжелого */}
  <button
    onClick={extractMax}
    disabled={busy || heap.length === 0}
    className="flex-1 min-w-[170px] bg-gradient-t
    40 disabled:cursor-not-allowed text-white f

```



```

        ${patient.animState === 'in' ? 'анимация' : ''}
        ${patient.animState === 'winner' ? 'победитель' : ''}
        ${patient.animState === 'out' ? 'анимация' : ''}
        ${isRoot ? 'bg-rose-950/80 border-radius-10px' : ''}
      `}
    style={{
      left: `${pos.x}%`,
      top: `${pos.y}%`,
      width: isRoot ? 60 : 52,
      height: isRoot ? 60 : 52,
    }}
  >
    <span className="text-2xl leading-none font-medium">
      <span className="text-[10px] font-monospace">
        {patient.priority}%
      </span>
    </span>

    {/* Tooltip */}
    <div className="absolute bottom-full left-0 shadow-xl">
      <div className="bg-slate-950 border-1px border-slate-700 shadow-xl">
        <div className="font-bold text-white text-sm">
          <div className="text-emerald-400">
            {patient.priority}%
          </div>
        </div>
      </div>
    </div>
  </div>
);
}}
</div>

<div className="w-full h-3 bg-gradient-to-r from-rose-500 to-rose-900">
</div>

{/* ПРАВАЯ КОЛОНКА: ОПЕРАЦИОННЫЙ СТОЛ */}
<div className="lg:col-span-4 bg-slate-900 rounded-2xl">
  <div className="absolute top-4 left-4 text-[10px] font-monospace">
    Операционная койка
  </div>

```

```
    <div className="text-[10px] font-mono text-slate-500">
      {log.map((entry, idx) => (
        <div
          key={idx}
          className={`text-xs font-mono flex items-center justify-between gap-2`}
        >
          {idx === 0 ? <span className="text-emerald-500 font-weight-bold">
            <span>{entry}</span>
          </div>
        )}}
      </div>
    </div>
  );
};
```

ФАЙЛ 41/82: src/components/hints/demoKit.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { createContext, useContext, useEffect, useReducer } from 'react';

/** Общие примитивы для мини-визуализаций подсказок. */
export const W = 260;
export const H = 130;

/**
 * Общий множитель скорости. Кадры были слишком короткими,
 * заметить, что именно переехало, и анимация читалась
 */
export const SPEED = 2.1;

/** Длительность переезда узлов. Должна быть заметно меньше MOVE_MS
export const MOVE_MS = 800;
export const EASE = "cubic-bezier(.34,.01,.2,1)";
export const MOVE = `all ${MOVE_MS}ms ${EASE}`;
```

```

        {caption}
      </div>
    })
    <div className="text-[9px] text-slate-600 text-center"
      {paused ? "пауза – курсор на карточке" : "наведи"}
    </div>
  </div>
);
};

```

```

export const Node: React.FC<{
  x: number;
  y: number;
  r?: number;
  fill: string;
  label?: string | number;
  stroke?: string;
}> = ({ x, y, r = 13, fill, label, stroke }) => (
  <g style={{ transition: MOVE }}>
    <circle cx={x} cy={y} r={r} fill={fill} stroke={stroke} />
    {label !== undefined && (
      <text x={x} y={y} textAnchor="middle" dominantBaseline="middle">
        {label}
      </text>
    )}
  </g>
);

```

```

export const Edge: React.FC<{
  a: [number, number];
  b: [number, number];
  color?: string;
  width?: number;
  dashed?: boolean;
}> = ({ a, b, color = C.edge, width = 2, dashed }) => (
  <line
    x1={a[0]}
    y1={a[1]}
    x2={b[0]}
    y2={b[1]}
    stroke={color}
    stroke-width={width}
    stroke-dasharray={dashed ? [4, 4]} : []
  />
);

```

```

    <text x={130} y={112} textAnchor="middle" fontSize=
      9
    </text>
    <text x={72} y={54} textAnchor="middle" fontSize=
      3
    </text>
    <text x={188} y={54} textAnchor="middle" fontSize=
      4
    </text>

    <Node x={pos.i[0]} y={pos.i[1]} label="i" fill={C}
    <Node x={pos.k[0]} y={pos.k[1]} label="k" fill={v}
    <Node x={pos.j[0]} y={pos.j[1]} label="j" fill={r}
  </Svg>
);
};

```

/** Компоненты связности: несколько «островов», которые

```

export const ComponentsDemo: React.FC = () => {
  const comps: [string, string][][] = [
    ["A", "B"], ["B", "C"],
    ["D", "E"],
    ["F", "G"], ["G", "H"], ["F", "H"],
  ];
  const pos: Record<string, [number, number]> = {
    A: [26, 40], B: [62, 92], C: [26, 92],
    D: [120, 40], E: [120, 92],
    F: [190, 34], G: [232, 78], H: [178, 96],
  };
  const colors = [C.active, C.warn, C.done];
  const step = useLoop(comps.length + 1, 900);
  const compOf = (v: string) => comps.findIndex((e) =>

  return (
    <Svg caption={step === 0 ? "Граф распался на острова"
      {comps.flatMap((edges, ci) =>
        edges.map(([u, v], i) => (

```

```

        fontWeight="700"
        fill={C.warn}
      >
        {w}
      </text>
    )}
  </g>
);
}})
{Object.entries(pos).map(([k, [x, y]]) => {
  <Node key={k} x={x} y={y} label={k} fill={C.idle} />
})}
</Svg>
);
};

```

/** Сжатие координат: разрежённые числа → плотные индексы

```

export const CoordCompressDemo: React.FC = () => {
  const raw = [1000, 5, 74000, 5, 300];
  const sorted = [5, 300, 1000, 74000];
  const step = useLoop(3, 1200);
  const boxW = 46;
  const startX = (260 - raw.length * (boxW + 4)) / 2;

  return (
    <Svg
      caption={
        step === 0
          ? "Исходные координаты – огромные и с дырами"
          : step === 1
            ? "Сортируем уникальные значения"
            : "Каждое число заменяем его номером → плотные индексы"
      }
    >
      {raw.map((v, i) => {
        <g key={i}>
          <rect x={startX + i * (boxW + 4)} y={20} width={boxW} height={20} />
          <text x={startX + i * (boxW + 4) + boxW / 2} y={35} />
        </g>
      )}
    >

```

```

const f = frames[step];
const color = (id: string) =>
  id === "x" ? "#6366f1" : id === "p" ? "#0ea5e9" : id

return (
  <Svg caption={captions[step]}>
    {f.edges.map(([a, b], i) => (
      <line
        key={i}
        x1={f.pos[a][0]}
        y1={f.pos[a][1]}
        x2={f.pos[b][0]}
        y2={f.pos[b][1]}
        stroke={C.edge}
        strokeWidth={1.8}
        style={{ transition: MOVE }}
      />
    ))}
    {Object.entries(f.pos).map(([id, [x, y]]) => (
      <g key={id} style={{ transform: `translate(${x}, ${y})` }>
        <circle r={13} fill={color(id)} stroke={C.idle} />
        <text textAnchor="middle" dominantBaseline="bottom">
          {id}
        </text>
      </g>
    ))}
  </Svg>
);
};

```

```

/** Zig – один поворот, когда родитель уже корень. */
export const ZigDemo: React.FC = () => (
  <SplayFrames
    captions={["p – корень, x под ним", "Один поворот: "]}
    frames={[
      { pos: { p: [130, 32], x: [78, 92] }, edges: [{"p", "x"}]},
      { pos: { x: [130, 32], p: [182, 92] }, edges: [{"x", "p"}]}
    ]}
  />
);

```

```

};
const GRAPH_EDGES: [string, string][] = [
  ["A", "B"], ["A", "C"], ["B", "D"], ["C", "D"], ["D", "E"], ["E", "F"],
];

export const BfsDemo: React.FC = () => {
  const order = [["A"], ["B", "C"], ["D"], ["E", "F"]];
  const step = useLoop(order.length + 1, 850);
  const visited = new Set(order.slice(0, step).flat());
  const wave = step > 0 && step <= order.length ? order[step - 1] : null;
  return (
    <Svg caption={`Уровень ${Math.max(0, Math.min(step, order.length))}`} >
      {GRAPH_EDGES.map(([u, v], i) => {
        <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_POS[v]} />
      })}
      {Object.entries(GRAPH_POS).map(([k, [x, y]]) => {
        <Node key={k} x={x} y={y} label={k}
          fill={wave?.includes(k) ? C.active : visited.has(k) ? C.visited : C.inactive}
          stroke={wave?.includes(k) ? "#a5b4fc" : undefined} />
      })}
    </Svg>
  );
};

```

```

export const DfsDemo: React.FC = () => {
  const path = ["A", "B", "D", "E", "F", "C"];
  const step = useLoop(path.length + 1, 750);
  const visited = path.slice(0, step);
  const head = visited[visited.length - 1];
  return (
    <Svg caption={head ? `Идём вглубь: ${visited.join(" ")}` : ""} >
      {GRAPH_EDGES.map(([u, v], i) => {
        const iu = visited.indexOf(u);
        const iv = visited.indexOf(v);
        const onPath = iu >= 0 && iv >= 0 && Math.abs(iu - iv) <= 1;
        return <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_POS[v]}
          fill={onPath ? C.onPath : visited.has(u) || visited.has(v) ? C.visited : C.inactive}
          stroke={onPath ? "#a5b4fc" : undefined} />
      })}
      {Object.entries(GRAPH_POS).map(([k, [x, y]]) => {
        <Node key={k} x={x} y={y} label={k}
          fill={visited.has(k) ? C.visited : C.inactive}
          stroke={visited.has(k) ? "#a5b4fc" : undefined} />
      })}
    </Svg>
  );
};

```

```

    </Svg>
  );
};

export const RelaxDemo: React.FC = () => {
  const step = useLoop(3, 1100);
  const dV = [12, 12, 7][step];
  const improved = step === 2;
  return (
    <Svg caption={improved ? "12 > 3 + 4 → пишем 7. Путь  

      <Edge a={[60, 70]} b={[200, 70]} color={improved ? "red" : "black"} />  

      <text x={130} y={54} textAnchor="middle" fontSize=14 />  

      <Node x={60} y={70} r={20} fill={C.active} label="3" />  

      <Node x={200} y={70} r={20} fill={improved ? C.done : C.active} label="4" />  

      <text x={60} y={106} textAnchor="middle" fontSize=14 />  

      <text x={200} y={106} textAnchor="middle" fontSize=14 />  

      <text x={130} y={22} textAnchor="middle" fontSize=14 />  

    </Svg>
  );
};

export const BridgeDemo: React.FC = () => {
  const step = useLoop(2, 1300);
  const pos: Record<string, [number, number]> = {
    A: [40, 40], B: [40, 95], C: [95, 68], D: [170, 68]
  };
  const edges: [string, string][] = [
    ["A", "B"], ["A", "C"], ["B", "C"], ["C", "D"], ["D", "A"]
  ];
  const cut = step === 1;
  return (
    <Svg caption={cut ? "Убрали C-D → граф развалился на 2 компонента" : "Граф связен"} />
    {edges.map(([u, v], i) => {
      const isBridge = u === "C" && v === "D";
      if (isBridge && cut) return null;
      return <Edge key={i} a={pos[u]} b={pos[v]} color={isBridge ? "red" : "black"} />
    })}
    {Object.entries(pos).map(([k, [x, y]]) => (

```



```

const taken = new Set(chosen.slice(0, step));
return (
  <Svg caption={`Жадно берём самые лёгкие рёбра без ц
    {edges.map((e, i) => {
      const on = taken.has(`${e.u}-${e.v}`);
      return (
        <g key={i}>
          <Edge a={pos[e.u]} b={pos[e.v]} color={on ?
          <text x={({pos[e.u][0] + pos[e.v][0]} / 2} y=
            textAnchor="middle" fontSize={9} fontWeigh
          </g>
        );
      });
    })}
    {Object.entries(pos).map(([k, [x, y]]) => (<Node
  </Svg>
);
};

export const SccDemo: React.FC = () => {
  const pos: Record<string, [number, number]> = {
    A: [45, 35], B: [110, 35], C: [77, 95], D: [185, 45
  ];
  const edges: [string, string][] = [["A", "B"], ["B",
  const step = useLoop(2, 1400);
  const scc = ["A", "B", "C"];
  const inScc = (u: string, v: string) => step === 1 &&
  return (
    <Svg caption={step === 1 ? "{A,B,C} – цикл: из кажд
      {edges.map(([u, v], i) => {
        <Edge key={i} a={pos[u]} b={pos[v]} color={inSc
      })}
      {Object.entries(pos).map(([k, [x, y]]) => {
        <Node key={k} x={x} y={y} label={k} fill={step :
      })}
    </Svg>
  );
};

```



```

<text x={4} y={18} fontSize={8} fill={C.dim}>a</text>
{a.map((v, i) => (
  <g key={i}>
    <rect x={20 + i * 39} y={24} width={34} height={12}
      fill={show && i >= l && i <= r ? C.active : C.dim}>
    <text x={37 + i * 39} y={36} textAnchor="middle">{v}</text>
  </g>
)}}
<text x={4} y={72} fontSize={8} fill={C.dim}>P</text>
{pref.map((v, i) => (
  <g key={i}>
    <rect x={20 + i * 33} y={78} width={29} height={12}
      fill={show && i === l ? C.bad : show && i === r ? C.dim : C.idle}
      stroke={C.idleStroke} style={{ transition: "stroke 0.5s" }}>
    <text x={34 + i * 33} y={90} textAnchor="middle">{v}</text>
  </g>
)}}
</Svg>
);
};

```

```

export const SparseTableDemo: React.FC = () => {
  const step = useLoop(3, 1000);
  const len = [1, 2, 4][step];
  const count = 8 - len + 1;
  return (
    <Svg caption={`Уровень j=${step}: готовые ответы для`}>
      {Array.from({ length: 8 }).map((_, i) => (
        <g key={i}>
          <rect x={12 + i * 30} y={18} width={26} height={12}
            fill={C.dim}>
          <text x={25 + i * 30} y={29} textAnchor="middle">{i}</text>
        </g>
      ))}
      {Array.from({ length: count }).map((_, i) => (
        <rect key={i} x={12 + i * 30} y={52 + (i % 4) * 12}
          fill={C.active} opacity={i === 0 ? 1 : 0.4} stroke={C.dim}>

```

```

export const PrefixFunctionDemo: React.FC = () => {
  const s = "abacaba";
  const pi = [0, 0, 1, 0, 1, 2, 3];
  const step = useLoop(s.length + 1, 700);
  const i = Math.max(0, step - 1);
  const len = step > 0 ? pi[i] : 0;
  return (
    <Svg caption={step > 0 ? `π[${i}] = ${len}: префикс`
      {s.split("").map((ch, idx) => (
        <g key={idx}>
          <rect x={18 + idx * 33} y={26} width={28} height={44}
            fill={step > 0 && (idx < len || (idx > i - len)) ? C.fill : C.dim}
            style={{ transition: "fill .25s" }} />
          <text x={32 + idx * 33} y={40} textAnchor="middle">{ch}</text>
          <text x={32 + idx * 33} y={70} textAnchor="middle">π</text>
          <rect x={18 + idx * 33} y={70} width={28} height={44}
            fill={idx <= i && step > 0 ? C.warn : C.idle} />
        </g>
      ))}
    <text x={4} y={40} fontSize={8} fill={C.dim}>s</text>
    <text x={4} y={70} fontSize={8} fill={C.dim}>π</text>
    <text x={130} y={106} textAnchor="middle" fontSize={12}>
      π[i] — длина самого длинного «префикс = суффикс»
    </text>
  </Svg>
  );
};

```

```

export const DpDemo: React.FC = () => {
  const rows = 3, cols = 5;
  const step = useLoop(rows * cols + 1, 320);
  return (
    <Svg caption={`Заполняем таблицу подзадач: готово ${step}`
      {Array.from({ length: rows }).map((_, r) =>
        Array.from({ length: cols }).map((_, c) => {
          const idx = r * cols + c;
          const filled = idx < step;
          return (

```

```

-----
| "suffix-link" | "toposort" | "relax" | "prefix-sum"
| "segment-tree" | "bridge" | "articulation" | "mst"
| "prefix-function" | "dp" | "floyd" | "components" |
| "zig" | "zig-zig" | "zig-zag";

const REGISTRY: Record<DemoKind, React.FC> = {
  bfs: BfsDemo,
  dfs: DfsDemo,
  heap: HeapDemo,
  stack: StackDemo,
  queue: QueueDemo,
  dsu: DsuDemo,
  trie: TrieDemo,
  "trie-bfs": TrieBfsDemo,
  "suffix-link": SuffixLinkDemo,
  toposort: ToposortDemo,
  relax: RelaxDemo,
  "prefix-sum": PrefixSumDemo,
  "sparse-table": SparseTableDemo,
  "segment-tree": SegmentTreeDemo,
  bridge: BridgeDemo,
  articulation: ArticulationDemo,
  mst: MstDemo,
  amortized: AmortizedDemo,
  np: NpDemo,
  scc: SccDemo,
  "prefix-function": PrefixFunctionDemo,
  dp: DpDemo,
  floyd: FloydDemo,
  components: ComponentsDemo,
  graph: GraphBasicsDemo,
  "coord-compress": CoordCompressDemo,
  zig: ZigDemo,
  "zig-zig": ZigZigDemo,
  "zig-zag": ZigZagDemo,
};

export const MiniDemo: React.FC<{ kind: DemoKind }> = (

```

```

    };
    window.addEventListener("keydown", onKey);
    document.body.style.overflow = "hidden";
    return () => {
        window.removeEventListener("keydown", onKey);
        document.body.style.overflow = "";
    };
}, [vizId, onClose]);

const viz = getViz(vizId ?? undefined);
if (!vizId || !viz) return null;
const Component = viz.Component;

return createPortal(
    <div className="fixed inset-0 z-[110] flex items-end" >
        <div className="absolute inset-0 bg-slate-950/80" >
            <div className="relative w-full sm:max-w-5xl max-h-
                ded-2xl shadow-2xl flex flex-col" >
                <div className="flex items-center gap-3 px-4 py-4" >
                    <div className="min-w-0 flex-1" >
                        <h3 className="font-bold text-white text-sm" >
                            {viz.hint} && <p className="text-[11px] text-white" >
                                {viz.description}
                        </div>
                    <button
                        onClick={onClose}
                        className="p-2 rounded-lg bg-slate-800 hover:bg-slate-700"
                        aria-label="Заккрыть симулятор"
                    >
                        <X className="w-5 h-5" />
                    </button>
                </div>
                <div className="overflow-y-auto overscroll-contain" >
                    <Component />
                </div>
            </div>
        </div>,
    document.body
);

```

```

        return;
    }
    const vw = window.innerWidth;
    const vh = window.innerHeight;
    const width = Math.min(CARD_W, vw - 16);
    const height = cardRef.current?.offsetHeight ?? 300;

    let left = anchor.rect.left + anchor.rect.width / 2;
    left = Math.max(8, Math.min(left, vw - width - 8));

    const below = anchor.rect.top < height + GAP + 8;
    const top = below ? anchor.rect.bottom + GAP : anchor.rect.top - GAP;

    setPos({ top: Math.max(8, Math.min(top, vh - height)), left }, [anchor]);

    useEffect(() => {
        if (!anchor) return;
        const onKey = (e: KeyboardEvent) => {
            if (e.key === "Escape") onClose();
        };
        window.addEventListener("keydown", onKey);
        window.addEventListener("scroll", onClose, true);
        return () => {
            window.removeEventListener("keydown", onKey);
            window.removeEventListener("scroll", onClose, true);
        };
    }, [anchor, onClose]);

    if (!anchor) return null;
    const entry = GLOSSARY_BY_ID.get(anchor.termId);
    if (!entry) return null;
    const viz = getViz(entry.simulator);
    const simulatorId = entry.simulator;

    return createPortal(
        <div
            ref={cardRef}

```



```

    })

    {viz && simulatorId && (
      <button
        onClick={() => onOpenSimulator(simulatorId)}
        className="w-full flex items-center justify-center text-sm font-medium rounded-lg py-2 transition-colors"
      >
        <Play className="w-3.5 h-3.5" /> Показать
      </button>
    )}
  </div>
</div>,
document.body
);
};

```

ФАЙЛ 48/82: src/components/JohnsonViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

```

import { useState } from 'react';
import { RotateCcw, SkipForward, Undo } from 'lucide-react';

export function JohnsonViz() {
  const [step, setStep] = useState(0);
  const MAX_STEP = 7;

  const nodes = [
    { id: 'S', x: 200, y: 150, label: 'S' },
    { id: 'A', x: 200, y: 50, label: 'A' },
    { id: 'B', x: 320, y: 220, label: 'B' },
    { id: 'C', x: 80, y: 220, label: 'C' },
  ];
}

```

```

        if (step > 5) return "stroke-emerald-500";
    }
    if (e.id === 'CA') {
        if (step === 6) return "stroke-amber-400 stroke-emerald-500";
        if (step > 6) return "stroke-emerald-500";
    }

    if (step >= 4) return "stroke-slate-600"; // otherwise
    return "stroke-slate-500";
};

const getMarkerUrl = (e: any) => {
    if (e.u === 'S') {
        if (step === 0 || step === 7) return "";
        if (step === 1) return "url(#arrow-amber)";
        if (step === 2) return "url(#arrow-pink)";
        return "url(#arrow-slate-dim)";
    }
    if (e.id === 'AB' && step === 4) return "url(#arrow-amber)";
    if (e.id === 'AB' && step > 4) return "url(#arrow-slate-dim)";
    if (e.id === 'BC' && step === 5) return "url(#arrow-pink)";
    if (e.id === 'BC' && step > 5) return "url(#arrow-slate-dim)";
    if (e.id === 'CA' && step === 6) return "url(#arrow-emerald-500)";
    if (e.id === 'CA' && step > 6) return "url(#arrow-emerald-500)";

    if (step >= 4) return "url(#arrow-slate-dim)";
    return "url(#arrow-slate)";
};

const getEdgeContent = (e: any) => {
    if (e.u === 'S') return e.weight;

    if (step < 4) return e.weight;

    if (e.id === 'AB') {
        if (step === 4) return `${e.weight} + (${0})`;
        return e.newWeight;
    }

```

Добавляем фиктивную вершину S. Соединяем вершины S с остальными вершинами.

Это наша база для расчета потенциалов.

);
case 2: return (

<div>
Шаг 2: Ищем потенциалы $h(v)$.
Запускаем медленный, но мощный алгоритм для всех вершин:

...
</div>

);
case 3: return (

<div>
Шаг 3: Магическая формула.Шаг 3.1: Пересчет ребра A → B.
Всем ребрам графа назначаем новый вес.
<div className="text-center font-monospace">
Это повышает или понижает вес ребра.

</div>
);
case 4: return (

<div>
Шаг 3.1: Пересчет ребра A → B.
Старый вес: ...
Вычисляем: $-2 + 0 - (-2) =$...
</div>

);
case 5: return (

<div>
Шаг 3.2: Пересчет ребра B → C.
Старый вес: 1.

Вычисляем: $1 + (-2) - (-1) =$...
</div>

);
case 6: return (

<div>
Шаг 3.3: Пересчет ребра C → A.

```

        <path d="M 0 0 L 10 5 L 10 10" />
      </marker>
      <marker id="arrow-pink" viewBox="0 0 10 10" />
    tart-reverse">
      <path d="M 0 0 L 10 5 L 10 10" />
    </marker>
    <marker id="arrow-emerald" viewBox="0 0 10 10" />
  o-start-reverse">
    <path d="M 0 0 L 10 5 L 10 10" />
  </marker>
</defs>

{/* Edges */}
{edges.map((e, i) => {
  const u = nodes.find(n => n.id === e.u);
  const v = nodes.find(n => n.id === e.v);

  const isS = e.u === 'S';
  if (isS && (step === 0 || step === 1)) {
    const midX = (u.x + v.x) / 2;
    const midY = (u.y + v.y) / 2;

    const edgeColorClass = getEdgeColorClass(e);
    const textHigh = isEdgeTextHigh(e);
    const textEmer = isEdgeTextEmer(e);

    return (
      <g key={i}>
        <line
          x1={u.x} y1={u.y}
          className={`stroke-${edgeColorClass}`}
          markerEnd={getMarkerEnd(e)}
        />

        <text
          x={midX} y={midY}
          textAnchor="middle"
          fill={textHigh ? 'black' : 'white'}
          stroke={textEmer ? 'black' : 'white'}
          stroke-width={2}
        />
      </g>
    );
  }
});
}

```

```

                                {pot}
                                </text>
                                </g>
                                }}
                                </g>
                                )
                                }}}
                                </svg>
                                </div>

                                <div className="w-full md:w-[320px] flex
                                  {/* Log Box */}
                                  <div className="bg-[#0f172a] p-5 rounded
l justify-center leading-relaxed text-sm text-white
                                  {getDesc()}
                                  </div>

                                  {/* Controls */}
                                  <div className="flex bg-slate-900 border
                                  <button onClick={() => setStep(1)}
                                  className="p-3 bg-slate-800 border
-slate-400 transition-colors" title="Сбросить"
                                  <RotateCcw strokeWidth={3}
                                  </button>
                                  <button onClick={() => setStep(1)}
                                  className="p-3 bg-slate-800 border
-slate-400 transition-colors" title="Вар на
                                  <Undo strokeWidth={3} size={24}
                                  </button>
                                  <button onClick={() => setStep(1)}
                                  className={`flex-1 font-bold text-white
                                  ${step === MAX_STEP
                                    ? 'bg-emerald-600/50'
                                    : 'bg-indigo-600 hover:bg-indigo-500'}
                                  {step === MAX_STEP ? "Готово" : "В процессе"}
                                  {step !== MAX_STEP && <Skip
                                  </button>
                                  </div>

```

```

}

const initialNodes: GNode[] = [
  { id: 0, x: 150, y: 100, label: "0" },
  { id: 1, x: 300, y: 100, label: "1" },
  { id: 2, x: 225, y: 220, label: "2" }, // 0->1->2->0
  { id: 3, x: 450, y: 160, label: "3" },
  { id: 4, x: 600, y: 160, label: "4" }, // 3 <-> 4 (Se
];

const initialEdges: GEdge[] = [
  { u: 0, v: 1 },
  { u: 1, v: 2 },
  { u: 2, v: 0 },
  { u: 2, v: 3 }, // Connection edge
  { u: 3, v: 4 },
  { u: 4, v: 3 },
];

interface AlgoStep {
  phase: "init" | "dfs1" | "transpose" | "dfs2" | "fini
  desc: string;
  visited: Set<number>;
  currentNode: number | null;
  order: number[];
  transposed: boolean;
  sccMap: Record<number, number>; // node -> sccId
  currentSccId: number;
  activeEdges: GEdge[];
}

export const KosarajuViz: React.FC = () => {
  const [steps, setSteps] = useState<AlgoStep[]>([]);
  const [currentStepIdx, setCurrentStepIdx] = useState(
  const [isPlaying, setIsPlaying] = useState(false);

  useEffect(() => {
    // Generate step-by-step Kosaraju steps

```

```
// DFS 1: post-order list
const dfs1 = (u: number) => {
  visited.add(u);
  recordStep(
    "dfs1",
    `DFS 1: зашли в вершину ${u}, помечаем посещение`,
    u,
    false,
    -1,
  );

  for (const to of adj[u]) {
    if (!visited.has(to)) {
      dfs1(to);
    }
  }

  order.push(u);
  recordStep(
    "dfs1",
    `DFS 1: вершина ${u} полностью обработана. Записываем`,
    u,
    false,
    -1,
  );
};

// Run DFS1 on all unvisited
for (let i = 0; i < 5; i++) {
  if (!visited.has(i)) {
    dfs1(i);
  }
}

const topoOrder = [...order].reverse();
recordStep(
  "transpose",
```

```

        recordStep(
            "dfs2",
            `DFS 2: берем следующую непосещенную из стека
            u,
            true,
            sccId,
        );
        dfs2(u, sccId);
    } else {
        recordStep(
            "dfs2",
            `DFS 2: вершина ${u} из стека уже покрашена в
            u,
            true,
            sccId,
        );
    }
}

recordStep(
    "finished",
    `Алгоритм успешно завершен! Найдено ${sccId} компонент
    null,
    true,
    sccId,
);

setSteps(generatedSteps);
setCurrentStepIdx(0);
}, []);

const step = steps[currentStepIdx] || {
    phase: "init",
    desc: "Загрузка...",
    visited: new Set(),
    currentNode: null,
    order: [],
    transposed: false,

```



```

        <button
          onClick={togglePlay}
          className="flex items-center gap-2 px-3 py-
            d-500 text-white"
        >
          {isPlaying ? (
            <Pause className="w-4 h-4 ml-1" />
          ) : (
            <Play className="w-4 h-4 ml-1" />
          )}
          {isPlaying ? "Пауза " : "Старт "}
        </button>

        <button
          onClick={reset}
          className="flex items-center justify-center
            ors"
        >
          <RotateCcw className="w-4 h-4" />
        </button>
      </div>
    </div>

    <div className="grid grid-cols-1 lg:grid-cols-12" >
      {/* Playfield SVG */}
      <div className="lg:col-span-8 bg-[#0B1120] rela
        <svg className="w-full h-full max-w-[650px]" >
          {/* Markers for directed arrows */}
          <defs>
            <marker
              id="arrow"
              viewBox="0 0 10 10"
              refX="22"
              refY="5"
              markerWidth="6"
              markerHeight="6"
              orient="auto-start-reverse"
            >

```

```

        step.sccMap[edge.u] &&
        step.sccMap[edge.v] &&
        step.sccMap[edge.u] === step.sccMap[edge.v]
    let stroke = "#475569";
    let markerId = "arrow";

    if (step.transposed) {
        stroke = isSccEdge ? "#818cf8" : "#4f46e5";
        markerId = "arrow-transposed";
    } else {
        if {
            step.currentNode === edge.u ||
            step.currentNode === edge.v
        } {
            stroke = "#10b981";
            markerId = "arrow-active";
        }
    }

    // Adjust slightly for bidirectional link
    const isBidi =
        (edge.u === 3 && edge.v === 4) ||
        (edge.u === 4 && edge.v === 3);
    const dy = isBidi ? (edge.u === 3 ? -10 : 10) : 0;

    return (
        <line
            key={`edge-${idx}`}
            x1={x1}
            y1={y1 + dy}
            x2={x2}
            y2={y2 + dy}
            stroke={stroke}
            strokeWidth="2.5"
            markerEnd={`url(#${markerId})`}
            className="transition-all duration-300"
        />
    );

```

```

        {node.label}
      </text>

      {sccId && {
        <text
          x={node.x}
          y={node.y - 28}
          textAnchor="middle"
          fill="#10b981"
          fontSize="10px"
          fontWeight="bold"
        >
          SCC #{sccId}
        </text>
      }}
    </g>
  );
}}
</svg>

{step.transposed && {
  <div className="absolute top-4 left-4 bg-in
    1 rounded">
    Граф транспонирован (Ребра обратные)
  </div>
}}
</div>

{/* Logs & Stacks */}
<div className="lg:col-span-4 bg-slate-950 p-5
  00 overflow-y-auto">
  <h4 className="text-xs font-bold text-slate-4
    Порядок обхода и стек
  </h4>

  <div className="bg-slate-900 border border-sl
    <p className="text-xs text-white leading-re
      {step.desc}

```

```

                key={idx}
                className={`text-[11px] p-1.5 rounded-[4px]`}
            >
                {s.desc}
            </div>
        )))
    </div>
</div>
</div>

<div className="mt-4 pt-3 border-t border-slate-200">
    <span>
        Шаг {currentStepIdx + 1} из {steps.length}
    </span>
    <div className="flex gap-1.5">
        <button
            disabled={currentStepIdx === 0}
            onClick={() => setCurrentStepIdx((prev) => prev - 1)}
            className="bg-slate-900 border border-slate-700 px-2 py-1 rounded"
        >
            Назад
        </button>
        <button
            disabled={currentStepIdx >= steps.length - 1}
            onClick={() => setCurrentStepIdx((prev) => prev + 1)}
            className="bg-slate-900 border border-slate-700 px-2 py-1 rounded"
        >
            Вперед
        </button>
    </div>
</div>
</div>
</div>
</div>
);
};

```

```

-----
    { id: 4, label: 'E', x: 50, y: 50, color: 'none' },
    { id: 5, label: 'F', x: 82, y: 50, color: 'none' },
    { id: 6, label: 'G', x: 20, y: 80, color: 'none' },
    { id: 7, label: 'H', x: 50, y: 87, color: 'none' },
    { id: 8, label: 'I', x: 80, y: 80, color: 'none' },
  ];

// 12 Edges connecting the 9 vertices
const initialEdges: Edge[] = [
  { id: '0-1', u: 0, v: 1, weight: 2, status: 'pending' },
  { id: '3-4', u: 3, v: 4, weight: 3, status: 'pending' },
  { id: '1-2', u: 1, v: 2, weight: 4, status: 'pending' },
  { id: '0-3', u: 0, v: 3, weight: 5, status: 'pending' },
  { id: '1-4', u: 1, v: 4, weight: 6, status: 'pending' },
  { id: '4-5', u: 4, v: 5, weight: 7, status: 'pending' },
  { id: '3-6', u: 3, v: 6, weight: 8, status: 'pending' },
  { id: '2-5', u: 2, v: 5, weight: 9, status: 'pending' },
  { id: '6-7', u: 6, v: 7, weight: 10, status: 'pending' },
  { id: '4-7', u: 4, v: 7, weight: 11, status: 'pending' },
  { id: '7-8', u: 7, v: 8, weight: 12, status: 'pending' },
  { id: '5-8', u: 5, v: 8, weight: 13, status: 'pending' },
];

export const KruskalSimulator: React.FC = () => {
  const [activeAlgo, setActiveAlgo] = useState<'kruskal'>
  const [vertices, setVertices] = useState<Vertex[]>(in
  const [edges, setEdges] = useState<Edge[]>(initialEdg
  const [stepIndex, setStepIndex] = useState<number>(0)
  const [heapQueue, setHeapQueue] = useState<string[]>(
  const [activeCheatTab, setActiveCheatTab] = useState<

  const [logs, setLogs] = useState<StepLog[]>([
    {
      title: 'Введение в раскраску (Краскал)',
      description: '«Краскал – это раскраска?». Предста
        овы начинать!',
      type: 'info',
    },
  ]),

```

```

        description: 'Вершины D и E окрашены в синий цвет',
        type: 'success'
    }, ...prev]);
},
// Step 5: Inspect B-C (4)
() => {
    setEdges(prev => prev.map(e => e.id === '1-2' ? {
    setLogs(prev => [{
        title: 'Шаг 3: Берем ребро B-C (вес 4)',
        description: 'Оно соединяет красную вершину B и
        type: 'info'
    }, ...prev]));
},
// Step 6: Include B-C
() => {
    setVertices(prev => prev.map(v => v.id === 2 ? {
    setEdges(prev => prev.map(e => e.id === '1-2' ? {
    setLogs(prev => [{
        title: 'Успех: Ребро B-C в осто́ве',
        description: 'Вершина C перекрашена в красный цвет',
        type: 'success'
    }, ...prev]));
},
// Step 7: Inspect A-D (5)
() => {
    setEdges(prev => prev.map(e => e.id === '0-3' ? {
    setLogs(prev => [{
        title: 'Шаг 4: Берем ребро A-D (вес 5)',
        description: 'ВНИМАНИЕ! Ребро соединяет КРАСНУЮ
        ю – мы перекрашиваем всё в один цвет!"',
        type: 'warning'
    }, ...prev]));
},
// Step 8: Include A-D (Merge to purple)
() => {
    setVertices(prev => prev.map(v => v.color === 'blue' ? {
    setEdges(prev => prev.map(e => e.status === 'included' ? {
    setLogs(prev => [{

```

```

        description: 'Вершина F окрашена в фиолетовый ц
        type: 'success'
    }, ...prev]));
},
// Step 13: Inspect D-G (8)
() => {
    setEdges(prev => prev.map(e => e.id === '3-6' ? {
    setLogs(prev => [{
        title: 'Шаг 7: Берем ребро D-G (вес 8)',
        description: 'Оно соединяет фиолетовую вершину
        type: 'info'
    }, ...prev]));
},
// Step 14: Include D-G
() => {
    setVertices(prev => prev.map(v => v.id === 6 ? {
    setEdges(prev => prev.map(e => e.id === '3-6' ? {
    setLogs(prev => [{
        title: 'Успех: Ребро D-G в осто́ве',
        description: 'Вершина G окрашена в фиолетовый.'
        type: 'success'
    }, ...prev]));
},
// Step 15: Inspect C-F (9) - Cycle!
() => {
    setEdges(prev => prev.map(e => e.id === '2-5' ? {
    setLogs(prev => [{
        title: 'Шаг 8: Берем ребро C-F (вес 9)',
        description: 'Оно соединяет вершины C и F, кото
        type: 'warning'
    }, ...prev]));
},
// Step 16: Reject C-F
() => {
    setEdges(prev => prev.map(e => e.id === '2-5' ? {
    setLogs(prev => [{
        title: 'Отказ: Цикл обнаружен!',
        description: 'Выбрасываем ребро C-F.',

```

```

    }, ...prev]));
  },
  // Step 21: Inspect H-I (12)
  () => {
    setEdges(prev => prev.map(e => e.id === '7-8' ? {
    setLogs(prev => [{
      title: 'Шаг 11: Берем ребро H-I (вес 12)',
      description: 'Соединяет фиолетовую H и последнюю',
      type: 'info'
    }, ...prev]));
  },
  // Step 22: Include H-I
  () => {
    setVertices(prev => prev.map(v => v.id === 8 ? {
    setEdges(prev => prev.map(e => e.id === '7-8' ? {
    setLogs(prev => [{
      title: 'Успех: MST построено Краскалом!',
      description: 'Все 9 вершин окрашены в единый фиолетовый цвет. Суммарный вес: 3 + 4 + 5 + 7 + 8 + 10 + 12 = 51.',
      type: 'success'
    }, ...prev]));
  },
  ];

  // --- PRIM STEPS ---
  const primSteps = [
    // Step 1: Start at E
    () => {
      setVertices(prev => prev.map(v => v.id === 4 ? {
      setHeapQueue(['D-E (3)', 'B-E (6)', 'E-F (7)', 'E-G (8)'],
      setEdges(prev => prev.map(e => ['3-4', '1-4', '4-5', '4-6', '4-7', '4-8', '4-9']),
      setLogs(prev => [{
        title: 'Шаг 1: Плесень зарождается в Токио (Вершина E)',
        description: 'Мы выбрали вершину E как стартовую. Добавили в приоритетную очередь (Heap): D-E(3), B-E(6), E-F(7), E-G(8).',
        type: 'info'
      }, ...prev]));
    },
  ],

```



```

        description: 'Новое ребро A-B(2) добавлено в кучу',
        type: 'success'
    }, ...prev]);
},
// Step 6: Pop A-B (2)
() => {
    setEdges(prev => prev.map(e => e.id === '0-1' ? {
    setLogs(prev => [{
        title: 'Шаг 4: Куча выдает ребро A-B (вес 2)',
        description: 'Плесень стремительно бежит к вершине A.',
        type: 'info'
    }, ...prev]));
},
// Step 7: Include A-B, add B's edges
() => {
    setVertices(prev => prev.map(v => v.id === 1 ? {
    setEdges(prev => prev.map(e => e.id === '0-1' ? {
        eap' } : e));
    setHeapQueue(['B-C (4)', 'B-E (6 - цикл)', 'E-F (4)']);
    setLogs(prev => [{
        title: 'Успех: Вершина B захвачена плесенью',
        description: 'В кучу добавлено B-C(4). Ребро B-A(2) включено.',
        type: 'success'
    }, ...prev]));
},
// Step 8: Pop B-C (4)
() => {
    setEdges(prev => prev.map(e => e.id === '1-2' ? {
    setLogs(prev => [{
        title: 'Шаг 5: Куча выдает ребро B-C (вес 4)',
        description: 'Плесень тянется к вершине C.',
        type: 'info'
    }, ...prev]));
},
// Step 9: Include B-C, add C's edges
() => {
    setVertices(prev => prev.map(v => v.id === 2 ? {
    setEdges(prev => prev.map(e => e.id === '1-2' ? {

```

```

setVertices(prev => prev.map(v => v.id === 5 ? {
setEdges(prev => prev.map(e => e.id === '4-5' ? {
  eap' } : e));
setHeapQueue(['D-G (8)', 'C-F (9 - цикл)', 'E-H (
setLogs(prev => [{
  title: 'Успех: Вершина F захвачена',
  description: 'Ребро F-I(13) добавлено в кучу. P
  type: 'success'
}, ...prev]));
},
// Step 14: Pop D-G (8)
() => {
  setEdges(prev => prev.map(e => e.id === '3-6' ? {
setLogs(prev => [{
  title: 'Шаг 8: Куча выдает ребро D-G (вес 8)',
  description: 'Плесень расширяется вниз к G.',
  type: 'info'
}, ...prev]));
},
// Step 15: Include D-G, add G's edges
() => {
  setVertices(prev => prev.map(v => v.id === 6 ? {
setEdges(prev => prev.map(e => e.id === '3-6' ? {
  eap' } : e));
setHeapQueue(['C-F (9 - цикл)', 'G-H (10)', 'E-H
setLogs(prev => [{
  title: 'Успех: Вершина G захвачена',
  description: 'В кучу добавлено G-H(10).',
  type: 'success'
}, ...prev]));
},
// Step 16: Pop C-F (9) - Cycle!
() => {
  setEdges(prev => prev.map(e => e.id === '2-5' ? {
setLogs(prev => [{
  title: 'Шаг 9: Куча выдает ребро C-F (вес 9)',
  description: 'Вершины C и F уже заражены плесенью',
  type: 'warning'

```

```

        description: 'Обе вершины E и H уже в плесени.',
        type: 'warning'
    }, ...prev]);
},
// Step 21: Reject E-H
() => {
    setEdges(prev => prev.map(e => e.id === '4-7' ? {
    setHeapQueue(['H-I (12)', 'F-I (13)']));
    setLogs(prev => [{
        title: 'Отказ: Игнорируем E-H',
        description: 'Выбрасываем из кучи.',
        type: 'error'
    }, ...prev]));
},
// Step 22: Pop H-I (12)
() => {
    setEdges(prev => prev.map(e => e.id === '7-8' ? {
    setLogs(prev => [{
        title: 'Шаг 12: Куча выдает H-I (вес 12)',
        description: 'Плесень тянется к последней чистой',
        type: 'info'
    }, ...prev]));
},
// Step 23: Include H-I
() => {
    setVertices(prev => prev.map(v => v.id === 8 ? {
    setEdges(prev => prev.map(e => e.id === '7-8' ? {
    setHeapQueue(['F-I (13 - цикл)']));
    setLogs(prev => [{
        title: 'Успех: Плесень захватила весь граф',
        description: 'Все 9 вершин поглощены. Итоговый граф
            иной – мы разрастались из одной точки с помощью
        type: 'success'
    }, ...prev]));
},
},
];

// --- BORUVKA STEPS ---

```

```

    }
    return e;
  }));
setLogs(prev => [{
  title: 'Слияние: Деревни объединились в колхозы',
  description: 'Все выбранные дороги построены ра
    оз {D, E, F, G, H, I}.',
  type: 'success'
}, ...prev]));
},
// Step 3: Five-Year Plan 2 (Inspecting bridge betw
()) => {
  // Now both kolkhozes look at all outgoing roads
  // Outgoing roads between {A, B, C} and {D, E, F,
  // A-D (5), B-E (6), C-F (9).
  // Cheapest is A-D (5).
  setEdges(prev => prev.map(e => e.id === '0-3' ? {
  setLogs(prev => [{
    title: 'Вторая пятилетка: Колхозы выбирают мост
    description: 'Теперь Красный и Синий колхозы од
      -D с весом 5. Остальные мосты B-E(6) и C-F(9)
    type: 'warning'
  }, ...prev]));
},
// Step 4: Concurrently merge into single Kolkhoz
()) => {
  setVertices(prev => prev.map(v => ({ ...v, color:
  setEdges(prev => prev.map(e =>
    e.status === 'included' || e.id === '0-3' ? { .
  ));
  setLogs(prev => [{
    title: 'Объединение завершено: Один гигантский
    description: 'Оба колхоза объединились по мосту
    type: 'success'
  }, ...prev]));
},
// Step 5: Check remaining edges & mark as rejected
()) => {

```

```

        {
          title: 'Введение в Плесень (Прима)',
          description: 'Логика «Плесени»: выбираем стар
            едь (Heap) и захватываем соседей!',
          type: 'info',
        },
      ]));
    } else {
      setLogs([
        {
          title: 'Введение в Коллективизацию (Борувка)',
          description: 'Логика «Борувки»: Каждая из 9 д
            чтобы объединиться в колхоз.',
          type: 'info',
        },
      ]));
    }
  }
};

```

```

const getColorClass = (color: string, id: number) =>
  switch (color) {
    case 'red': return 'bg-red-500 border-red-300 tex
    case 'blue': return 'bg-blue-500 border-blue-300
    case 'purple': return 'bg-purple-600 border-purple
    case 'mold': return 'bg-emerald-500 border-emerald
    default:
      if (activeAlgo === 'prim' && id === 4 && stepIn
        return 'bg-slate-700 border-emerald-500 text-
      }
      return 'bg-slate-700 border-slate-500 text-slate
    }
  };

```

```

const getEdgeStroke = (status: string, color: string)
  if (status === 'inspecting') return '#f59e0b';
  if (status === 'rejected') return '#ef4444';
  if (status === 'in_heap') return '#0284c7';
  if (status === 'included') {

```

```

    }
  >
    <GitGraph className="w-3.5 h-3.5" />
    <span>Краскал (Билет 18)</span>
  </button>
  <button
    onClick={() => handleReset('prim')}
    className={
      "flex items-center gap-1.5 px-3 py-2
+
      (activeAlgo === 'prim'
        ? 'bg-emerald-600 text-white shadow
        : 'text-slate-400 hover:text-slate-:
      )
    }
  >
    <Layers className="w-3.5 h-3.5" />
    <span>Прима (Билет 19)</span>
  </button>
  <button
    onClick={() => handleReset('boruvka')}
    className={
      "flex items-center gap-1.5 px-3 py-2
+
      (activeAlgo === 'boruvka'
        ? 'bg-rose-600 text-white shadow-md
        : 'text-slate-400 hover:text-slate-:
      )
    }
  >
    <Users className="w-3.5 h-3.5" />
    <span>Борувка (Билет 20)</span>
  </button>
</div>

<button
  onClick={() => handleReset(activeAlgo)}
  className="flex items-center justify-cent
-200 font-semibold text-xs rounded-xl borde
>

```



```

        </text>
      </g>
    );
  }}}
</svg>

{/* HTML Overlay for Vertices */}
<div className="absolute inset-0 w-full h-full" >
  {vertices.map(vertex => {
    <div
      key={vertex.id}
      style={{ top: vertex.y + "%", left: vertex.x + "%" }}
      className={"absolute -translate-x-1/2 -translate-y-1/2"}
      font-bold text-base border-2 transition-all duration-300
      id={vertex.id}
    >
      <span>{vertex.label}</span>
      {vertex.id === 4 && activeAlgo === 'prim' && !isHighlighted(4)} &&
      <span className="text-[8px] block -mt-10px" >
        {vertex.id}
      </span>
    </div>
  )}}
</div>

{stepIndex >= currentSteps.length && (
  <div className="absolute inset-x-6 bottom-6"
    shadow-xl z-20">
    <p className={"font-bold text-base " +
      "text-rose-300"} >
      MST построено с помощью {activeAlgo}
    </p>
    <p className="text-slate-300 text-xs mt-2">
      Общий вес минимального остова: 51. Все
    </p>
  </div>
)}
</div>

```



```

      : log.type === 'error'
      ? 'bg-rose-950/40 border-rose-500'
      : 'bg-slate-900/60 border-slate-700'
    }
  >
    <div className="flex items-center gap-2">
      {log.type === 'success' && <CheckCircle className="text-emerald-400" />}
      {log.type === 'warning' && <HelpCircle className="text-amber-400" />}
      {log.type === 'error' && <XCircle className="text-red-400" />}
      {log.type === 'info' && <Play className="text-cyan-400" />}
      <h5 className="font-bold text-sm text-slate-300">{log.message}</h5>
    </div>
    <p className="text-xs text-slate-300">{log.timestamp}</p>
  </div>
  </div>
  </div>
</div>

```

```

/* Cheat Sheet: Complexity & Code for all 3 algo
<div className="bg-slate-900/80 border border-slate-700 p-2">
  <div className="flex flex-col sm:flex-row items-center gap-2">
    <div className="flex items-center gap-2.5">
      <BookOpen className="w-5 h-5 text-indigo-400" />
      <h4 className="text-xl font-bold text-white">Cheat Sheet</h4>
    </div>
    <div className="flex bg-slate-950 p-1 rounded">
      <button
        onClick={() => setActiveCheatTab('kruskal')}
        className={
          activeCheatTab === 'kruskal' ? "bg-indigo-600 text-white" : "bg-slate-900 text-slate-300"
        }
      >
        Краскал (Билет 18)
      </button>
      <button
        onClick={() => setActiveCheatTab('prim')}

```

```

        </div>
        <div className="lg:col-span-2">
            <div className="bg-slate-950 p-4 rounded-1">
                <p className="text-slate-400 text-xs font-mono">
                    <Code className="w-3.5 h-3.5 text-indigo-400">
                        </p>
                        <pre className="text-xs text-emerald-400 font-mono">
1      80 flex-1">
{`# 1. Инициализируем DSU
def find(i):
    if parent[i] == i: return i
    parent[i] = find(parent[i])
    return parent[i]

def union(i, j):
    r_i, r_j = find(i), find(j)
    if r_i != r_j:
        parent[r_i] = r_j
        return True
    return False

# 2. Жадный выбор ребер
edges.sort() # O(E log E)
mst = []
for w, u, v in edges:
    if union(u, v):
        mst.append((u, v, w))`}
        </pre>
        </div>
        </div>
        </div>
    })

    {activeCheatTab === 'prim' && (
        <div className="grid grid-cols-1 lg:grid-cols-2">
            <div className="lg:col-span-1 space-y-4">
                <div className="bg-slate-950 p-4 rounded-1">
                    <p className="text-slate-400 text-xs font-mono">

```

```

        for next_w, v in graph[u]:
            if not visited[v]:
                heapq.heappush(heap, (next_w, v, u))
    return mst` }

```

```

    </pre>

```

```

    </div>

```

```

    </div>

```

```

    </div>

```

```

})

```

```

{activeCheatTab === 'boruvka' && (

```

```

    <div className="grid grid-cols-1 lg:grid-cols-2" >

```

```

        <div className="lg:col-span-1 space-y-4">

```

```

            <div className="bg-slate-950 p-4 rounded-3xl">

```

```

                <p className="text-slate-400 text-xs font-medium">

```

```

                    <Clock className="w-3.5 h-3.5 text-red-500" />

```

```

                </p>

```

```

                <p className="text-2xl font-black text-white">

```

```

                <p className="text-slate-400 text-xs mt-1">

```

```

            </div>

```

```

        </div>

```

```

        <div className="bg-slate-950 p-4 rounded-3xl">

```

```

            <p className="text-slate-400 text-xs font-medium">

```

```

                <Award className="w-3.5 h-3.5 text-red-500" />

```

```

            </p>

```

```

            <p className="text-2xl font-black text-white">

```

```

            <p className="text-slate-400 text-xs mt-1">

```

```

        </div>

```

```

        <div className="bg-slate-950 p-4 rounded-3xl">

```

```

            <p className="text-slate-400 text-xs font-medium">

```

```

                ении:</p>

```

```

                <p className="text-xs text-slate-300 leading-relaxed">

```

```

                , организовав массовое распараллеленное слияние

```

```

            </div>

```

```

        </div>

```

```

    <div className="lg:col-span-2">

```

```

        <div className="bg-slate-950 p-4 rounded-3xl">

```

```
-----  
import bellmanImg from '../assets/bellman-ford-truck.jpg'  
import floydImg from '../assets/floyd-network.jpg';
```

```
const cards = [  
  {  
    img: dijkstraImg,  
    alt: 'Добрый робот Дейкстра',  
    title: '    Добрый (Дейкстра)',  
    desc: 'Быстрый, жадный, но работает только с',  
    highlight: 'положительными',  
    highlightColor: 'text-emerald-400',  
    rest: 'весами. Подсунь отрицательное ребро – сломает',  
    complexity: ' $O((V+E) \log V)$ ',  
    border: 'border-blue-500/30 hover:border-blue-400/60',  
    titleColor: 'text-blue-400',  
    badge: 'text-blue-400/70 bg-blue-950/40',  
  },  
  {  
    img: bellmanImg,  
    alt: 'Фура по болоту Форд-Беллман',  
    title: '    Фура по Болоту (Ф-Б)',  
    desc: 'Медленный, тяжёлый – зато тащит',  
    highlight: 'отрицательные',  
    highlightColor: 'text-rose-400',  
    rest: 'веса и детектит отрицательные циклы.',  
    complexity: ' $O(V \cdot E)$ ',  
    border: 'border-rose-500/30 hover:border-rose-400/60',  
    titleColor: 'text-rose-400',  
    badge: 'text-rose-400/70 bg-rose-950/40',  
  },  
  {  
    img: floydImg,  
    alt: 'Все-ко-Все Флойд',  
    title: '    Все ко Все (Флойд)',  
    desc: 'Матрица + три цикла (',  
    highlight: 'k, i, j',  
    highlightColor: 'text-emerald-400',  
    rest: ')). Ищет пути от всех ко всем.',
```

```
interface Props {
  selectedChapterId: string;
  setSelectedChapterId: (id: string) => void;
  /** Заголовок текущей темы — показываем в кнопке, что */
  currentTitle: string;
  index: number;
  total: number;
}

/**
 * Мобильное содержание: липкая панель снизу-сверху + в
 * На десктопе не рендерится (там обычный сайдбар).
 */
export const MobileToc: React.FC<Props> = ({ selectedCh
  const [open, setOpen] = useState(false);

  useEffect(() => {
    document.body.style.overflow = open ? "hidden" : ""
    return () => {
      document.body.style.overflow = "";
    };
  }, [open]);

  useEffect(() => {
    const onKey = (e: KeyboardEvent) => e.key === "Escap
    window.addEventListener("keydown", onKey);
    return () => window.removeEventListener("keydown",
  }, []));

  return (
    <>
      <div className="lg:hidden sticky top-[57px] z-40
        <button
          onClick={() => setOpen(true)}
          className="w-full flex items-center gap-2.5 p
          aria-label="Открыть содержание"
        >
```

```
    });  
  };
```

ФАЙЛ 53/82: src/components/Navbar.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useState } from 'react';  
import { BookOpen, Cpu, Sparkles, FileDown, FileText, Cloud } from 'lucide-react';  
import { chapters } from '../data/content';  
import { downloadBookHtml } from '../utils/exportHtml';
```

```
interface NavbarProps {  
  activeTab: 'guide' | 'simulator';  
  setActiveTab: (tab: 'guide' | 'simulator') => void;  
}
```

```
export const Navbar: React.FC<NavbarProps> = ({ activeTab, setActiveTab }) => {  
  const [busy, setBusy] = useState(false);
```

```
  const saveBook = async () => {  
    setBusy(true);  
    try {  
      await downloadBookHtml(chapters);  
    } finally {  
      setBusy(false);  
    }  
  };  
};
```

```
  return (  
    <header className="bg-slate-900 border-b border-slate-800">  
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">  
        <div className="flex items-center gap-3 w-full">  
          <div className="bg-gradient-to-tr from-indigo-500 to-purple-500 p-2 rounded-lg">  
            <div className="flex items-center gap-2">  
              <div></div>  
            </div>  
          </div>  
        </div>  
      </div>  
    </header>  
  );
```

```

        <a
            id="download-pdf-btn"
            href="/export/full_code_all_pages.pdf"
            download="full_code_all_pages.pdf"
            target="_blank"
            rel="noreferrer"
            className="flex items-center justify-center
over:bg-emerald-600/20 border border-emerald-500/20"
            title="Скачать полный PDF сборник всех страниц"
        >
            <FileDown className="w-4 h-4" /> Скачать PDF
        </a>
        <a
            id="download-txt-btn"
            href="/export/full_code_all_pages.txt"
            download="full_code_all_pages.txt"
            target="_blank"
            rel="noreferrer"
            className="flex items-center justify-center
over:bg-sky-600/20 border border-sky-500/30 transition"
            title="Скачать полный TXT файл со всем кодом"
        >
            <FileText className="w-4 h-4" /> TXT
        </a>
        <button
            id="download-html-btn"
            onClick={saveBook}
            disabled={busy}
            className="flex items-center justify-center
over:bg-amber-600/20 border border-amber-500/20"
            title="Скачать всё пособие одним автономным файлом"
        >
            {busy ? <Loader2 className="w-4 h-4 animate-spin" /> : "Скачать"}
        </button>
    </div>
</div>
</header>
);

```

```

        function ccw(ax:number,ay:number,bx:number,cx:number) {
            return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax)
        }
        return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)
        && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)
    }
    const crosses: number[] = [];
    for(let i=0;i<edges.length;i++) {
        for(let j=i+1;j<edges.length;j++) {
            if (intersect(edges[i][0],edges[i][1],edges[j][0],edges[j][1])) {
                if (!crosses.includes(i)) crosses.push(i)
                if (!crosses.includes(j)) crosses.push(j)
            }
        }
    }
    const lines = []; const circles = [];
    for(let i=0;i<edges.length;i++) {
        const p1=findPt(edges[i][0]),p2=findPt(edges[i][1]);
        const xing = crosses.includes(i);
        lines.push(<line key={`l${i}`} x1={p1.x} x2={p2.x} y1={p1.y} y2={p2.y}
            stroke={xing ? "#f43f5e" : "#4f46e5"} />);
    }
    const labels = ["A","B","C","D","E"];
    for(const p of pts) {
        circles.push(<g key={`c${p.id}`}>
            <circle cx={p.x} cy={p.y} r={8} fill="#4f46e5" />
            <text x={p.x} y={p.y+3} textAnchor="middle">{labels[p.id]}</text>
        </g>);
    }
    return <>{lines}{circles}</>;
    })()}
</svg>
<div className="absolute bottom-2 left-2 right-2 z-20">
    V=5, E=10 {'>'} 3V-6 = 9 {'-->'} {'\u274C'}
</div>
</div>

```



```

        stroke={xing?"#f43f5e":"#4f46e5"} strokeDasharray={dash}
      }
      const labels = ["\u04141", "\u04142", "\u04143"]
      for(const p of pts) {
        circles.push(<g key={`c${p.id}`}>
          <circle cx={p.x} cy={p.y} r={8} fill={color} />
          <text x={p.x} y={p.y+3} textAnchor="middle" />
        </g>);
      }
      return <>{lines}{circles}</>;
    })()
  </svg>
  <div className="absolute bottom-2 left-2 right-2 top-2 z-20">
    
$$V=6, E=9 \Rightarrow 2V-E=12-9=3 \text{ (двудольный)}$$

  </div>
</div>
</div>

<div className="bg-amber-950/40 border border-amber-950/40 p-10">
  ⚠ Запомни:  $K5$  и  $K3,3$  — два кита непланарности.
  гомеоморфный  $K5$  или  $K3,3$  — он непланарен. Теорема Кюри.
</div>
</div>
);
}

```

ФАЙЛ 55/82: src/components/QueueViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState, useCallback } from 'react';
import { Sparkles } from 'lucide-react';

```

```

interface Customer {
  id: string;

```

```

const addLog = (msg: string) => setLog(prev => [msg,

// ENQUEUE: Встать в конец очереди (хвост / Tail)
const enqueue = useCallback(() => {
  if (isServing) return;
  if (queue.length >= 6) {
    addLog('⚠ Хвост очереди уже на улице, повару тяже.
    setShake(true);
    setTimeout(() => setShake(false), 400);
    return;
  }

  const preset = PRESETS[counter % PRESETS.length];
  counter++;

  const newGuest: Customer = {
    id: Date.now().toString(),
    ...preset,
    animState: 'in',
  };

  setQueue(prev => [...prev, newGuest]);
  addLog(`- ENQUEUE: ${newGuest.name} ${newGuest.emoj

  setTimeout(() => {
    setQueue(prev => prev.map(c => c.id === newGuest.
  }, 450);
}, [queue, isServing]);

// DEQUEUE: Выдать суп первому (голова / Head, FIFO)
const dequeue = useCallback(() => {
  if (isServing) return;
  if (queue.length === 0) {
    setShake(true);
    addLog('⚠ DEQUEUE: В очереди никого нет! Повар ск
    setTimeout(() => setShake(false), 400);
    return;

```

```

    setQueue(prev => prev.map(c => ({ ...c, animState
  }, 500));
};

```

```

return (
  <div className="flex flex-col gap-6">
    {/* МНЕМОНИКА / ПРАВИЛО */}
    <div className="bg-indigo-950/40 border border-in
      <div className="text-3xl"> </div>
      <div>
        <h3 className="font-black text-indigo-400 tex
        <p className="text-xs text-slate-300 mt-1 lea
          В очереди за супом всё честно. Кто первый в
          Новые гости встают строго <b>в конец</b> оч
          Здесь нет обхода и блата – только строгий п
        </p>
      </div>
    </div>
  </div>

```

```

  {/* УПРАВЛЕНИЕ */}
  <div className="flex items-center gap-3 flex-wrap
    <button
      onClick={enqueue}
      disabled={isServing}
      className="flex-1 min-w-[150px] bg-gradient-t
        d:opacity-40 disabled:cursor-not-allowed te
        ive:scale-95 transition-all cursor-pointer
    >
      <span> Встать в очередь (ENQUEUE)</span>
    </button>

```

```

    <button
      onClick={dequeue}
      disabled={isServing || queue.length === 0}
      className="flex-1 min-w-[150px] bg-gradient-t
        opacity-40 disabled:cursor-not-allowed text
        :scale-95 transition-all cursor-pointer fle
    >

```

```
        ПОВАР
      </span>
    </div>
  </div>

  { /* РАЗДЕЛИТЕЛЬНАЯ СТРЕЛКА */ }
  <div className="flex flex-col items-center" >
    <span className="text-2xl animate-pulse" >
      <span className="text-[8px] font-mono" >
    </div>

  { /* СПИСОК ОЧЕРЕДИ */ }
  <div className="flex-1 flex items-end gap-4" >
    {queue.length === 0 ? (
      <div className="flex-1 flex flex-col" >
        <span className="text-4xl mb-1" > </
        <p className="text-xs font-bold" >
        <p className="text-[10px]" >Все сыты
      </div>
    ) : (
      queue.map((customer, idx) => {
        const isHead = idx === 0;
        const isTail = idx === queue.length - 1;
        return (
          <div
            key={customer.id}
            className={`
              flex flex-col items-center gap-4
              ${customer.animState === 'in' ? 'animate-in' : ''}
              ${customer.animState === 'eat' ? 'animate-eat' : ''}
              ${customer.animState === 'out' ? 'animate-out' : ''}
            `
          >
            { /* Облачко мыслей */ }
            <div className={`
              absolute -top-16 left-1/2 -transform translate-x-50%
              text-center text-slate-200 leading-tight
              ${isHead ? 'opacity-100 border' : ''}
            `}
          >

```

```

    { /* ПРАВАЯ КОЛОНКА: СЫТЫЕ И ДОВОЛЬНЫЕ */ }
    <div className="md:col-span-3 bg-slate-900 rounded-3xl" >
      <div className="absolute top-4 left-4 text-[11px] font-mono" >
        Сытые гости (Архив FIFO)
      </div>

      <div className="absolute top-4 right-4 flex items-center justify-between" >
        <div className="flex flex-col gap-1" >
          <div className="flex items-center gap-1" >
            <div className="order-emerald-800/80 text-[9px] font-mono" >
              <Sparkles className="w-3.5 h-3.5 animate-pulse" />
              <span>СЫТЫ </span>
            </div>
          </div>
        </div>
      </div>

      <div className="flex-1 flex flex-col justify-between" >
        {servedHistory.length === 0 ? (
          <div className="flex-1 flex flex-col items-center justify-center" >
            <span className="text-5xl mb-2" > </span>
            <p className="text-xs font-bold font-mono" >
              <p className="text-[10px] mt-1" >Здесь 6
            </p>
          </div>
        ) : (
          <div className="flex flex-col gap-1.5 w-full" >
            {servedHistory.map((item, idx) => (
              <div
                key={idx}
                className="w-full py-1.5 px-3 rounded-3xl" >
                0 flex items-center gap-2 text-[11px] text-
              >
                <span>{item}</span>
              </div>
            )
          )
        )
      </div>

    { /* Полн */ }
    <div className="w-full h-3 bg-gradient-to-r from-slate-900 to-slate-800" >
  </div>

```

```

    x?: number;
    y?: number;
}

export const SalmonAutomatonWidget: React.FC = () => {
  const [words, setWords] = useState<string[]>(['CAR',
  const [newWord, setNewWord] = useState('');
  const [nodes, setNodes] = useState<{ [key: number]: T
  const [simulationStep, setSimulationStep] = useState<
  const [bfsQueue, setBfsQueue] = useState<number[]>([]
  const [currentNode, setCurrentNode] = useState<number
  const [speechText, setSpeechText] = useState<string>('
    'Привет! Я Эхо-Лосось Сеня! Введи слова в словарь и
      буль!'
  );
  const [isTalking, setIsTalking] = useState<boolean>(fa
  const [isBlinking, setIsBlinking] = useState<boolean>

  // Регулярное моргание рыбы
  useEffect(() => {
    const blinkInterval = setInterval(() => {
      setIsBlinking(true);
      setTimeout(() => setIsBlinking(false), 200);
    }, 4000);
    return () => clearInterval(blinkInterval);
  }, []);

  // Эффект разговора при смене текста
  useEffect(() => {
    setIsTalking(true);
    const timer = setTimeout(() => setIsTalking(false),
    return () => clearTimeout(timer);
  }, [speechText]);

  // Построение начального бора (Trie) без fail-ссылок
  const buildInitialTrie = (wordList: string[]) => {
    const newNodes: { [key: number]: TrieNode } = {
      0: { id: 0, char: 'ROOT', parent: 0, children: {}

```

```

    if (childKeys.length === 0) {
      newNodes[u].x = 40 + currentX * paddingX;
      currentX++;
    } else {
      let leftX = -1;
      let rightX = -1;
      childKeys.forEach(v => {
        layoutDFS(v, depth + 1);
        if (leftX === -1) leftX = newNodes[v].x!;
        rightX = newNodes[v].x!;
      });
      newNodes[u].x = leftX === -1 ? 40 + currentX * paddingX : (leftX + rightX) / 2;
      if (leftX === -1) currentX++;
    }
  };
};

```

```

layoutDFS(0, 0);

```

```

setNodes(newNodes);
setSimulationStep(0);
setBfsQueue([]);
setCurrentNode(null);
setSpeechText(
  `Отлично! Я построил базовое префиксное дерево (БПД) для слова S, чтобы расставить fail-ссылки!`
);
};

```

```

useEffect(() => {
  buildInitialTrie(words);
}, []);

```

```

const handleAddWord = (e: React.FormEvent) => {
  e.preventDefault();
  const cleanWord = newWord.trim().toUpperCase().replace(' ', '');
  if (!cleanWord) return;
  if (words.includes(cleanWord)) {
    setSpeechText(`Слово «${cleanWord}» уже есть в слове S`);
  } else {
    words.push(cleanWord);
    buildInitialTrie(words);
  }
};

```

```

const stepBFS = () => {
  if (bfsQueue.length === 0) {
    setSimulationStep(2);
    setCurrentNode(null);
    setSpeechText(
      'Ура! Очередь пуста! Мы успешно построили полный  

        ь полюбоваться таблицей переходов!'
    );
    return;
  }

  const r = bfsQueue[0];
  const newQueue = bfsQueue.slice(1);
  const updatedNodes = { ...nodes };
  const rNode = updatedNodes[r];
  const childrenIds = Object.values(rNode.children);

  let logs = `Рассматриваем вершину #${r} (${rNode.chi

  for (const [char, u] of Object.entries(rNode.children)) {
    newQueue.push(u);
    let v = rNode.fail;
    while (v !== 0 && updatedNodes[v].children[char] !== undefined) {
      v = updatedNodes[v].fail;
    }
    const failTarget = updatedNodes[v].children[char];
    updatedNodes[u].fail = failTarget;

    // Рассчитываем терминальную ссылку (term_link)
    if (updatedNodes[failTarget].is_terminal) {
      updatedNodes[u].term_link = failTarget;
    } else {
      updatedNodes[u].term_link = updatedNodes[failTa
    }

    logs += `Для #${u} ('${char}') fail = #${failTarg
  }

```



```

<div className="grid grid-cols-1 md:grid-cols-3 g
  {/* Аватар Лосося (Сеня) с анимацией моргания и
  <div className="flex flex-col items-center just
    <div className="w-40 h-40 relative flex items
      full border border-blue-500/30 shadow-inner
      {/* SVG Лосося */}
      <svg
        viewBox="0 0 200 200"
        className={`w-36 h-36 transition-transform
          isTalking ? 'scale-105 drop-shadow-[0_0_
        ]`}
      >
        {/* Тело лосося */}
        <ellipse cx="100" cy="100" rx="70" ry="45"
        <path d="M 40 85 Q 100 55 160 85 Q 100 115
          40 85" fill="#4682b4"
          className="origin-[35px_100px] animate-
        />

        {/* Плавники */}
        <path d="M 90 55 L 70 30 L 110 55 Z" fill="#4682b4"
        <path d="M 90 145 L 70 170 L 110 145 Z" fill="#4682b4"
        <path
          d="M 120 115 L 100 135 L 130 130 Z"
          fill="#f87171"
          className="origin-[120px_115px] animate-
        />

        {/* Глаз (с морганием) */}
        <circle cx="145" cy="85" r="12" fill="#ff69b4"
        {isBlinking ? (
          <line x1="133" y1="85" x2="157" y2="85"
        ) : (
          <

```

```
<p className="text-slate-200 text-base leading-relaxed"
  {speechText}
</p>
```

```
<div className="mt-4 pt-3 border-t border-slate-200"
  <div className="text-xs text-slate-400 flex justify-between"
    <Info className="w-3.5 h-3.5 text-blue-400"
      Статус: {simulationStep === 0 ? 'Борется' : 'Победа'}
    </div>
  </div>
```

```
<div className="flex gap-2">
  {simulationStep === 0 && (
    <button
      onClick={startBFS}
      className="bg-emerald-600 hover:bg-emerald-500 text-white px-4 py-2 shadow-lg shadow-emerald-900/40 transition-colors"
    >
      <Play className="w-4 h-4" /> Запустить
    </button>
  )}
  {simulationStep === 1 && (
    <button
      onClick={stepBFS}
      className="bg-blue-600 hover:bg-blue-500 text-white px-4 py-2 shadow-lg shadow-blue-900/40 transition-colors"
    >
      <span>Шаг BFS</span>
      <span className="bg-blue-800 px-1.5 py-0.5 text-white text-xs" />
    </button>
  )}
  {simulationStep === 2 && (
    <button
      onClick={() => buildInitialTrie(words)}
      className="bg-indigo-600 hover:bg-indigo-500 text-white px-4 py-2 shadow-lg shadow-indigo-900/40 transition-transform active:scale-95"
    >
      <RotateCcw className="w-4 h-4" /> Перезагрузить
    </button>
  )}
</div>
```

```

        if (!parent.x || !parent.y || !node.x || !node.y) return;

        const isCurrent = currentNode === node;

        return (
          <g key={`edge-${node.id}`}>
            <line
              x1={parent.x}
              y1={parent.y + 16}
              x2={node.x}
              y2={node.y - 16}
              stroke={isCurrent ? '#60a5fa' : '#ccc'}
              strokeWidth="2"
              markerEnd="url(#arrowhead)"
            />
            <text
              x={({parent.x + node.x} / 2 - 10)}
              y={({parent.y + node.y} / 2)}
              fill="#94a3b8"
              fontSize="12"
              fontWeight="bold"
            >
              {node.char}
            </text>
          </g>
        );
      });
    });

    /* Draw Fail Links */
    if (simulationStep > 0 && Object.values(nodes).some(
      node => node.fail !== 0
    )) {
      // Wait, all nodes get fail link. Let's draw it at simulation
      // step >= 1 and computed it
      // To avoid too many lines, let's draw only one
      const target = nodes[nodes.fail];
      if (!target || !target.x || !target.y || !target.char) return;
    }
  }
}

```

```

        return (
            <g key={`node-${node.id}`}>
                <circle
                    cx={node.x}
                    cy={node.y}
                    r="16"
                    fill={fill}
                    stroke={stroke}
                    strokeWidth="2"
                />
                <text
                    x={node.x}
                    y={node.y + 4}
                    fill="white"
                    fontSize={isRoot ? "10" : "12"}
                    fontWeight="bold"
                    textAnchor="middle"
                >
                    {isRoot ? 'R' : node.char}
                </text>
                {!isRoot && (
                    <text
                        x={node.x + 12}
                        y={node.y - 12}
                        fill="#94a3b8"
                        fontSize="9"
                    >
                        {node.id}
                    </text>
                )}
            </g>
        );
    }
}
</svg>
);
})();
</div>

```

```

        type="submit"
        className="bg-slate-700 hover:bg-slate-600 transition-colors"
      >
        <Plus className="w-3.5 h-3.5" /> Добавить
      </button>
    </form>
  </div>

```

```

  { /* Таблица Вершин и суффиксных ссылок */ }
  <div className="lg:col-span-2 bg-slate-900/50 p-4" >
    <h5 className="text-white font-bold mb-3 flex" >
      <span className="flex items-center gap-1.5" >
        <span> </span> Таблица переходов и fail-ссылок
      </span>
      <span className="text-xs text-slate-400 font-mono" >
        Всего вершин: {Object.keys(nodes).length}
      </span>
    </h5>

```

```

    <div className="max-h-[220px] overflow-y-auto" >
      <table className="w-full text-left border-collapse" >
        <thead>
          <tr className="border-b border-slate-700" >
            <th className="py-2 px-3" >ID Вершины</th>
            <th className="py-2 px-3" >Символ</th>
            <th className="py-2 px-3" >Родитель</th>
            <th className="py-2 px-3" >fail-ссылка</th>
            <th className="py-2 px-3" >term_link</th>
            <th className="py-2 px-3" >Дети</th>
            <th className="py-2 px-3" >Слова (dict)
          </tr>
        </thead>
        <tbody className="divide-y divide-slate-800" >
          {Object.values(nodes).map((node) => {
            const isCurrent = currentNode === node
            const isInQueue = bfsQueue.includes(n

```

```

        <span className="bg-indigo-950"
          #{node.fail}
        </span>
      )}
    </td>
    <td className="py-2.5 px-3">
      {node.id === 0 || node.term_link
        <span className="text-slate-500"
          ) : {
            <span className="bg-rose-950"
0 transition-colors cursor-help" title={`Ск
          #{node.term_link}
        </span>
      )}
    </td>
    <td className="py-2.5 px-3 text-slate-500">
      {Object.keys(node.children).length
        ? '0'
        : Object.entries(node.children)
          .map(([c, id]) => `${c}→#`
            .join(', '))}
    </td>
    <td className="py-2.5 px-3">
      {node.words.length === 0 ? {
        <span className="text-slate-600"
      ) : {
        <span className="bg-emerald-950"
          {node.words.join(', ')}
        </span>
      )}
    </td>
  </tr>
</tbody>
</table>
</div>
</div>

```

```
    return tree;
}
```

```
// ===== STEP GENERATORS =====
```

```
// 1. Build steps (recursive top-down)
```

```
function genBuildSteps(arr: number[]): Step[] {
    const n = arr.length;
    const steps: Step[] = [];
    let id = 0;
    const tree: Record<number, number> = {};
```

```
    const CODE = [
        "build(v, l, r) {", // 1
        "    if (l === r) {", // 2
        "        tree[v] = A[l]; return", // 3
        "    }", // 4
        "    mid = (l + r) >> 1", // 5
        "    build(2v, l, mid)", // 6
        "    build(2v+1, mid+1, r)", // 7
        "    tree[v] = tree[2v]+tree[2v+1]", // 8
        "}", // 9
    ];
```

```
};
void CODE;
```

```
function go(v: number, l: number, r: number) {
    if (l === r) {
        steps.push({ id: id++, desc: `build(${v}, ${l}, ${r}`,
            , highlightNodes: [v], arrayHighlight: [l, r] });
        tree[v] = arr[l];
        steps.push({ id: id++, desc: `tree[${v}] = ${arr[l]}`,
            , highlightNodes: [v], arrayHighlight: [l, r] });
        return;
    }
    const m = (l + r) >> 1;
    steps.push({ id: id++, desc: `build(${v}, ${l}, ${r}`,
        { ...tree }, highlightNodes: [v], arrayHighlight: [l, r] });
    go(2 * v, l, m);
    go(2 * v + 1, m, r);
}
```

```

        rgeChildren: [2 * v, 2 * v + 1], arrayHighlight:
    return res;
}
const ans = go(1, 0, n - 1, qL, qR);
steps.push({ id: id++, desc: `Запрос sum([${qL}..${qR}], ${qL}, ${qR})`
    highlightNodes: [1], arrayHighlight: [qL, qR], result: ans});
return steps;
}

```

// 3. Bottom-Up Query steps (iterative on implicit tree)

```

function genQueryBottomUpSteps(arr: number[], qL: number, qR: number) {
    const n = arr.length;
    // Build iterative tree: leaves at [n..2n-1]
    const tree: Record<number, number> = {};
    for (let i = 0; i < n; i++) tree[n + i] = arr[i];
    for (let i = n - 1; i > 0; --i) tree[i] = (tree[2 * i] + tree[2 * i + 1]) / 2;

    const steps: Step[] = [];
    let id = 0;
    let l = qL + n;
    let r = qR + n + 1; // half-open [l, r)
    let res = 0;

    steps.push({ id: id++, desc: `Старт: l = qL + n = ${qL + n}, r = qR + n + 1 = ${qR + n + 1}`
        , codeLine: 3, treeState: tree, highlightNodes: [l, r]});

    while (l < r) {
        const highlights: number[] = [];
        if (l & 1) {
            res += tree[l] ?? 0;
            steps.push({ id: id++, desc: `l=${l} нечётный → l++`
                , codeLine: 5, treeState: tree, highlightNodes: [l]});
            highlights.push(l);
            l++;
        }
        if (r & 1) {
            r--;
            res += tree[r] ?? 0;
            steps.push({ id: id++, desc: `r=${r} нечётный → r--`
                , codeLine: 6, treeState: tree, highlightNodes: [r]});
            highlights.push(r);
        }
        l = (l + 1) / 2;
        r = (r - 1) / 2;
        if (l === r) break;
        const h = (l + r) / 2;
        highlights.push(h);
        steps.push({ id: id++, desc: `Сумма = ${res}. l=${l}, r=${r}, h=${h}`
            , codeLine: 7, treeState: tree, highlightNodes: [h]});
    }
    return steps;
}

```



```
const QUERY_TD_CODE = [
  "query(v, l, r, ql, qr) {",
  "  if (ql > r || qr < l) return 0;",
  "  // -----",
  "  if (ql <= l && r <= qr) return tree[v];",
  "  // -----",
  "  mid = (l + r) >> 1;",
  "  // спускаемся в обоих детей",
  "  return query(left) + query(right);",
  "}"
```

```
];
const QUERY_BU_CODE = [
  "queryBU(ql, qr) {",
  "  res = 0;",
  "  l = ql + n; r = qr + n + 1;",
  "  // -----",
  "  while (l < r) {",
  "    if (l & 1) res += tree[l++];",
  "    // -----",
  "    if (r & 1) res += tree[--r];",
  "    l >>= 1; r >>= 1;",
  "  }",
  "  return res; }"
```

```
];
```

```
// ===== PLAYER COMPONENT =====
```

```
function Player({ steps, color }: { steps: Step[]; color: string }) {
  const [idx, setIdx] = useState(0);
  const [playing, setPlaying] = useState(false);
  const [speed, setSpeed] = useState(700);
```

```
  useEffect(() => { setIdx(0); setPlaying(false); }, [steps]);
```

```
  useEffect(() => {
    if (!playing || idx >= steps.length - 1) { if (idx < steps.length - 1) {
      const t = setTimeout(() => setIdx(i => i + 1), speed);
      return () => clearTimeout(t);
    }
  }, [playing, idx, steps, speed]);
```

```

    <span className="text-[7px] opacity-60 font-mono">[nd.l]
    <span className="text-[9px] font-bold">[{nd.l}
    <span className="text-xs font-extrabold font-mono">[nd.l]
  </div>
  {(nd.left || nd.right) && (
    <div className="flex flex-col items-center mt-2">
      <div className="w-px h-2 bg-slate-700" />
      <div className="flex justify-around w-full">
        {nd.left && renderNode(nd.left)}
        {nd.right && renderNode(nd.right)}
      </div>
    </div>
  )}
</div>
);
}, [step, clr]);

return (
  <div className={`rounded-2xl border overflow-hidden
    ? 'border-emerald-500/30' : 'border-amber-500/30'
  } /* Header */>
    <div className={`px-4 py-2.5 flex items-center justify-between
      -emerald-950/50' : 'bg-amber-950/50'} border-b border-slate-500">
      <h4 className="font-bold text-sm text-white flex-1">
        <span className="text-[10px] font-mono text-slate-500">
      </div>

      { /* Array strip */}
      <div className="px-3 py-2 bg-slate-950 border-b border-slate-500">
        {arr.map((v, i) => {
          const inRange = step.arrayHighlight && i >= step.start
          return (
            <div key={i} className={`flex flex-col item
              go' ? 'bg-indigo-950 border-indigo-500' : 'bg-slate-900
              er-500'} -translate-y-0.5` : 'bg-slate-900'>
              <span className="text-[7px] text-slate-500">
              <span className="text-[11px] font-bold font-mono">
            </div>
          )
        })}
      </div>
    </div>
  )
);

```

```

    { /* Controls */
    <div className="px-3 py-2.5 bg-slate-950 border-t-1 border-l-1 border-r-1 border-b-1" >
      <div className="flex items-center bg-slate-900 rounded" >
        <button onClick={() => { setPlaying(false); setPlayingTime(0); }} className="p-1 text-slate-400 hover:text-white hover:bg-slate-800 rounded" title="Сброс"><RotateCcw className="w-3 h-3" /></button>
        <button onClick={() => { setPlaying(false); setPlayingTime(0); }} className="p-1 text-slate-400 hover:text-white hover:bg-slate-800 rounded" title="Пауза"><Pause className="w-3 h-3" /></button>
        <button onClick={() => setPlaying(!playing)} className="p-1 text-white transition-all ${playing ? 'bg-rose-500' : 'bg-slate-800'} rounded" title={playing ? 'Пауза' : 'Воспроизведение'}></button>
        <button onClick={() => { setPlaying(false); setPlayingTime(0); }} className="p-1 text-slate-400 hover:text-white hover:bg-slate-800 rounded" title="Остановить"><RotateCcw className="w-3 h-3" /></button>
      </div>
      <select value={speed} onChange={e => setSpeed(Number(e.target.value))} className="w-30 text-[9px] font-bold rounded px-1.5 py-1 outline-1" >
        <option value={1200}>Медл.</option>
        <option value={700}>Норма</option>
        <option value={350}>Быстро</option>
        <option value={120}>Турбо</option>
      </select>
    </div>

    { /* Progress bar */
    <div className="h-1 bg-slate-950"><div className="bg-rose-500" style={{ width: ((currentTime + 1) / total) * 100 : 0}%` }} /></div>
    </div>
  );
}

// =====
// MAIN EXPORTED COMPONENT
// =====
export function SegmentTreeVisualizer() {
  const [arr, setArr] = useState<number[]>([5, 8, 3, 12]);
  const [customInput, setCustomInput] = useState('5, 8, 3, 12');
  const [qL, setQL] = useState(1);

```

```

        <label className="text-[10px] font-bold up
label>
        <button type="button" onClick={randomize}
"><RefreshCw className="w-3.5 h-3.5" /></bu
</div>
<div className="flex gap-2">
    <input
        value={customInput}
        onChange={e => setCustomInput(e.target.v
        className="flex-1 bg-slate-900 border b
one focus:border-indigo-500"
        placeholder="5, 8, 3, 12, 7, 2"
    />
    <button type="submit" className="bg-indigo
ition-colors">Применить</button>
</div>
<div className="flex flex-wrap gap-1.5 pt-1
    {arr.map((v, i) => {
        <span key={i} className="bg-slate-900 b
        <span className="text-slate-500">A[{i
        </span>
    }}}
    <span className="text-xs text-slate-500 f
</div>
</form>

```

```

{/* Query range */}
<div className="bg-slate-950 p-4 rounded-xl b
    <label className="text-[10px] font-bold upp
    <div className="flex items-center gap-2">
        <div className="flex items-center gap-1">
            <span className="text-xs text-slate-400
            <input
                type="number"
                min={0}
                max={arr.length - 1}
                value={qL}
                onChange={e => setQL(Math.min(Number(

```

```

        te-200'}``}>
        <Info className="w-3.5 h-3.5" /> Почему / Сравн
    </button>
</div>

```

```

{ /* ---- TAB CONTENT ---- */
{view === 'build' && {
    <SimPanel
        key={`build-${arr.join('-')}`}
        title="Построение дерева (рекурсия)"
        icon=" "
        steps={buildSteps}
        code={BUILD_CODE}
        color="indigo"
        arr={arr}
    />
}}

```

```

{view === 'queries' && {
    <div className="grid grid-cols-1 xl:grid-cols-2"
        <SimPanel
            key={`qtd-${arr.join('-')}-${qL}-${qR}`}
            title={`Запрос sum([${qL}..${qR}]) – Сверху`
            icon="↑ "
            steps={queryTDSteps}
            code={QUERY_TD_CODE}
            color="emerald"
            arr={arr}
        />
        <SimPanel
            key={`qbu-${arr.join('-')}-${qL}-${qR}`}
            title={`Запрос sum([${qL}..${qR}]) – Снизу`
            icon="↑ "
            steps={queryBUSteps}
            code={QUERY_BU_CODE}
            color="amber"
            arr={arr}
        />
    }
}

```

```
<div className="flex justify-between border border-slate-900 p-5 rounded-xl">
  <span className="text-rose-400 font-mono">0(
    <div className="flex justify-between border border-slate-900 p-5 rounded-xl">
      <span className="text-rose-400 font-mono"><div className="flex justify-between border border-slate-900 p-5 rounded-xl">
        <span className="text-emerald-400 font-mono">д-400 font-bold">✓ легко</span></div>
      </div>
    </div>
  </div>
  <div className="bg-slate-950 p-5 rounded-xl border border-amber-500 shadow-md">
    <div className="absolute -top-3 left-4 bg-white text-black font-weight-normal">
      † Запрос Снизу-вверх (Итерация)
    </div>
    <p className="text-xs text-slate-300 mb-3">
      Стартуем с двух указателей (l и r) на левом и правом краях массива. Ищем индекс первого элемента, который больше или равен элементу, на котором мы находимся сейчас. Поднимаемся пока l {".
    </p>
    <div className="space-y-1.5 text-xs">
      <div className="flex justify-between border border-slate-900 p-5 rounded-xl">
        <span className="text-emerald-400 font-mono">д-400 font-bold">x сложно</span></div>
      </div>
    </div>
  </div>
</div>

{/* Verdict */}
<div className="bg-slate-950 p-5 rounded-xl border border-amber-500 shadow-md">
  <h4 className="font-bold text-blue-400 mb-2">Вердикт
  <p className="text-sm text-slate-300 leading-relaxed">
    Итеративный подход работает для большинства случаев. Но если нужны рекурсивный (сверху-вниз)<.
  </p>
</div>
</div>
```

```

return (
  <aside className="w-full bg-slate-900/80 lg:bg-tran
    <div className="p-4 pb-3 shrink-0">
      <h3 className="text-xs font-bold text-slate-400">
        <Compass className="w-4 h-4 text-indigo-400" />
      </h3>
      <div className="relative">
        <Search className="w-4 h-4 text-slate-500 abs
        <input
          value={query}
          onChange={(e) => setQuery(e.target.value)}
          placeholder="Поиск по темам.."
          className="w-full bg-slate-800/70 border bo
          500 focus:outline-none focus:border-indigo-!
        />
        {query && (
          <button
            onClick={() => setQuery("")}
            className="absolute right-2 top-1/2 -tran
            aria-label="Очистить поиск"
          >
            <X className="w-3.5 h-3.5" />
          </button>
        )}
      </div>
    </div>

    <div className="flex-1 overflow-y-auto px-3 pb-4">
      {groups.length === 0 && <p className="text-sm t

      {groups.map(([category, items]) => (
        <div key={category}>
          <div className="text-[10px] font-bold upperc
          <div className="space-y-1">
            {items.map((chapter) => {
              const isSelected = chapter.id === selec
              const hasViz = Boolean(VIZ_REGISTRY[cha
              return (

```

```
      <p className="leading-relaxed">
        Подчёркнутые термины в тексте раскрываются по
        симулятора.
      </p>
      <p className="text-slate-400 italic">Стрелки ←
    </div>
  </aside>
);
};
```

ФАЙЛ 59/82: src/components/Simulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useState } from 'react';
import { Layers, AlignLeft, Award, Plus, ArrowRight, Ar

interface Plate {
  id: string;
  name: string;
  icon: string;
}

interface Person {
  id: string;
  name: string;
  icon: string;
}

interface Hypothesis {
  id: string;
  text: string;
  probability: number;
}

export const Simulator: React.FC = () => {
```



```

    const newPlate = {
      id: Date.now().toString(),
      name: randomPreset.name,
      icon: randomPreset.icon
    };
    setStack(prev => [...prev, newPlate]);
    setPopMessage(` Положили поверх стопки: ${newPlate.name}`);
  };

  const popPlate = () => {
    if (stack.length === 0) {
      setPopMessage("⚠ Стопка пуста! Все тарелки вымыты");
      return;
    }
    const topPlate = stack[stack.length - 1];
    setStack(prev => prev.slice(0, prev.length - 1));
    setPopMessage(` Вымыта верхняя тарелка (LIFO): ${topPlate.name}`);
  };

  const resetStack = () => {
    setStack([
      { id: '1', name: 'Тарелка из-под пасты', icon: '🍝' },
      { id: '2', name: 'Тарелка из-под пиццы', icon: '🍕' },
      { id: '3', name: 'Блюдец от торта', icon: '🍰' },
    ]);
    setPopMessage(" Стопка тарелок восстановлена.");
  };

  // Queue handlers
  const addPerson = () => {
    const presets = [
      { name: 'Любитель борща', icon: '🥘' },
      { name: 'Курьер доставки', icon: '📦' },
      { name: 'Усталый админ', icon: '🤖' },
      { name: 'Турист с рюкзаком', icon: '🎒' },
    ];
    const randomPreset = presets[Math.floor(Math.random() * presets.length)];
    const newPerson = {

```

```

    setHeapMessage(`    Добавлена гипотеза с вероятностью
  });

const popHeap = () => {
  if (heap.length === 0) {
    setHeapMessage("⚠ Куча пуста! Нет кандидатов для
    return;
  }
  // Find highest probability
  let maxIdx = 0;
  for (let i = 1; i < heap.length; i++) {
    if (heap[i].probability > heap[maxIdx].probability) {
      maxIdx = i;
    }
  }
  const best = heap[maxIdx];
  setHeap(prev => prev.filter((_, idx) => idx !== maxIdx));
  setHeapMessage(`    Выбран самый "тяжелый" кандидат:
  });

const resetHeap = () => {
  setHeap([
    { id: '1', text: 'Кандидат: "Привет, я искусственный интеллект"' },
    { id: '2', text: 'Кандидат: "Привет, я испек вкусный хлеб"' },
    { id: '3', text: 'Кандидат: "Привет, я испанский язык"' },
  ]);
  setHeapMessage("    Гипотезы восстановлены.");
};

// Sorted heap for display (Max-Heap property view)
const sortedHeap = [...heap].sort((a, b) => b.probability - a.probability);

return (
  <div className="bg-slate-900/90 border border-slate-800 p-6 mb-6">
    <div className="border-b border-slate-800 pb-6 mb-6">
      <h2 className="text-2xl font-bold text-transparent bg-clip-text bg-gradient-to-r from-slate-400 to-slate-600">
        <span> Интерактивный симулятор структур данных

```

```

        <div className="text-left">
          <div className="font-bold text-sm text-wh
            <div className="text-xs opacity-80">FIFO
          </div>
        </div>
        <span className="text-xs bg-indigo-500/20 tex
          BFS
        </span>
      </button>

    <button
      onClick={() => setActiveTab('heap')}
      className={`flex items-center justify-between
        activeTab === 'heap'
          ? 'bg-emerald-600/20 border-emerald-500 t
            : 'bg-slate-800/60 border-slate-700/60 te
      `}
    >
      <div className="flex items-center gap-3">
        <Award className={`w-5 h-5 ${activeTab ===
          <div className="text-left">
            <div className="font-bold text-sm text-wh
              <div className="text-xs opacity-80">Приор
            </div>
          </div>
        </div>
        <span className="text-xs bg-emerald-500/20 te
          Beam Search
        </span>
      </button>
    </div>

    { /* STACK TAB */ }
    { activeTab === 'stack' && {
      <div className="space-y-6">
        <div className="bg-slate-800/80 p-5 rounded-x
          tify-between gap-4">
          <div>
            <h3 className="text-lg font-bold text-whi

```

```

    </div>
  })

  {/* Visualization */}
  <div className="bg-slate-950/60 p-6 rounded-2
    >
    {stack.length === 0 ? (
      <div className="text-center py-12 text-sl
        <div className="text-5xl mb-3">    </div>
        <p className="text-base font-bold">Стор
        <p className="text-xs mt-1">Добавьте гр
      </div>
    ) : (
      <div className="w-full max-w-sm flex flex
        <div className="absolute top-0 text-xs
full flex items-center gap-1">
        <span>ВЕРШИНА СТЕКА (TOP)</span>
        <ArrowDown className="w-3.5 h-3.5 anim
      </div>

      {/* Reversed map so top of stack is vis
      {stack.slice().reverse().map((plate, ind
        const isTop = index === 0;
        return (
          <div
            key={plate.id}
            className={`w-full flex items-cen
              isTop
                ? 'bg-gradient-to-r from-blue
le-102'
                : 'bg-slate-900/90 border-sla
            }`}
          >
            <div className="flex items-center
              <span className="text-2xl">{pla
              <span className={`font-bold tex
                {plate.name}
              </span>

```

```

        >
        <Plus className="w-4 h-4" /> Встать в очередь
    </button>
    <button
      onClick={servePerson}
      className="flex-1 md:flex-none flex items-center border-indigo-500/40 px-4 py-2.5 rounded"
    >
      Налить суп первому
    </button>
    <button
      onClick={resetQueue}
      title="Сбросить"
      className="p-2.5 bg-slate-800 hover:bg-tan-500 transition-colors cursor-pointer"
    >
      <RotateCcw className="w-5 h-5" />
    </button>
  </div>
</div>

{dequeueMessage && {
  <div className="bg-indigo-950/60 border border-indigo-950/60 p-3 animate-fade-in">
    <span className="inline-block w-2 h-2 rounded" style={dequeueMessage} />
  </div>
}}

{/* Visualization */}
<div className="bg-slate-950/60 p-6 rounded-2xl overflow-x-auto">
  {queue.length === 0 ? (
    <div className="text-center py-12 text-slate-400">
      <div className="text-5xl mb-3">0</div>
      <p className="text-base font-bold">Очередь пуста
      <p className="text-xs mt-1">Добавьте человека в очередь
    </div>
  ) : (
    <div className="flex flex-wrap gap-2">
      {queue.map((person, index) => (
        <div key={index} className="flex items-center gap-2">
          <div className="w-8 h-8 rounded bg-indigo-500/40 border border-indigo-500/60 flex items-center justify-center text-white font-size-8">{person.id}</div>
          <div>{person.name}</div>
        </div>
      ))}
    </div>
  )}
</div>

```

```

        </span>
      </div>
    );
  }}}

  <div className="flex flex-col items-center
-600 min-w-[120px] h-[120px]">
    <Plus className="w-6 h-6 mb-1" />
    <span className="text-xs font-mono up">
  </div>
</div>
})
<div className="mt-6 text-xs text-slate-500"
  ПЕРВЫМ ПРИШЕЛ → ПЕРВЫМ ПОЛУЧИЛ (FIFO) • И
</div>
</div>
</div>
})

{/* HEAP TAB */}
{activeTab === 'heap' && {
  <div className="space-y-6">
    <div className="bg-slate-800/80 p-5 rounded-x
tify-between gap-4">
      <div>
        <h3 className="text-lg font-bold text-whi
          <span> Приоритетная очередь / Куча</sp
          <span className="text-xs font-mono bg-e
x-Heap</span>
        </h3>
        <p className="text-slate-400 text-xs mt-1
          Здесь не важно, кто пришел первым. Важн
        </p>
      </div>
    <div className="flex items-center gap-3 w-f
      <button
        onClick={popHeap}
        className="flex-1 md:flex-none flex ite

```



```
        <div className="hidden sm:block"
          <div
            className={`h-full rounded-
              style={{ width: `${percent}%
            ></div>
          </div>
          <span className={`font-mono font-
            isTop ? 'text-emerald-400 tex
          }`}>
            {percent}%
          </span>
        </div>
      </div>
    );
  }
}
</div>
)}
<div className="mt-6 text-xs text-slate-500
  НЕ ВАЖНО, КТО ПРИШЕЛ ПЕРВЫМ • ВАЖЕН САМЫЙ
</div>
</div>
</div>
)}
</div>
);
};
```

ФАЙЛ 60/82: src/components/SimulatorHub.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useMemo, useState } from "react";
import { BookOpen, Play, Search, X } from "lucide-react";
import { VIZ_REGISTRY } from "../vizRegistry";
import { chapters } from "../data/content";
```


* Здесь ничего не подсказывается: дано настоящее дерево.
* поднять отмеченный узел в корень. Клик по узлу = один шаг.
* Порядок поворотов выбираешь сам; правильный ли он по сути.
* только после того, как узел окажется наверху (или по сути).
*/

```
export interface TNode {  
  key: number;  
  left: TNode | null;  
  right: TNode | null;  
}
```

```
const leaf = (key: number): TNode => ({ key, left: null, right: null })
```

```
const clone = (t: TNode | null): TNode | null =>  
  t ? { key: t.key, left: clone(t.left), right: clone(t.right) } : null
```

/** Заготовки: линия (zig-zig), змейка (zig-zag) и просеивание

```
export const PRESETS: { name: string; hintCase: string; target: number; build: () => TNode } =
```

```
{  
  {  
    name: "Линия влево",  
    hintCase: "Zig-Zig",  
    target: 1,  
    build: () => {  
      const n1 = leaf(1);  
      const n2: TNode = { key: 2, left: n1, right: leaf(3) };  
      const n4: TNode = { key: 4, left: n2, right: leaf(5) };  
      return { key: 6, left: n4, right: leaf(7) };  
    },  
  },  
  {  
    name: "Змейка",  
    hintCase: "Zig-Zag",  
    target: 3,  
    build: () => {  
      const n3 = leaf(3);  
      const n2: TNode = { key: 2, left: leaf(1), right: n3 };  
      const n4: TNode = { key: 4, left: n2, right: leaf(5) };  
      return { key: 6, left: n4, right: leaf(7) };  
    },  
  },  
}
```

```
{  
  name: "Змейка",  
  hintCase: "Zig-Zag",  
  target: 3,  
  build: () => {  
    const n3 = leaf(3);  
    const n2: TNode = { key: 2, left: leaf(1), right: n3 };  
    const n4: TNode = { key: 4, left: n2, right: leaf(5) };  
    return { key: 6, left: n4, right: leaf(7) };  
  },  
}
```

```

    if (path.length < 2) return root;

    const x = path[path.length - 1];
    const p = path[path.length - 2];
    const gg = path.length >= 3 ? path[path.length - 3] : null;

    if (p.left === x) {
        p.left = x.right;
        x.right = p;
    } else {
        p.right = x.left;
        x.left = p;
    }

    if (!gg) return x;
    if (gg.left === p) gg.left = x;
    else gg.right = x;
    return root;
}

/** Какой поворот сделал бы настоящий splay из текущего
export function canonicalNext(root: TNode, key: number) {
    const path = findPath(root, key);
    if (path.length < 2) return null;
    const x = path[path.length - 1];
    const p = path[path.length - 2];
    const g = path.length >= 3 ? path[path.length - 3] : null;

    if (!g) return { node: x.key, kind: "Zig" };

    const xLeft = p.left === x;
    const pLeft = g.left === p;
    if (xLeft === pLeft) return { node: p.key, kind: "Zig" };
    return { node: x.key, kind: "Zig-Zag (сначала нижняя поворот)" };
}

/* ----- раскладка -----

```

```

const preset = PRESETS[presetIdx];

const [tree, setTree] = useState<TNode>(() => preset.t
const [history, setHistory] = useState<TNode[]>([]);
const [moves, setMoves] = useState<{ node: number; co
const [showAnswer, setShowAnswer] = useState(false);

const target = preset.target;
const solved = tree.key === target;

const reset = useCallback(
  (idx = presetIdx) => {
    setPresetIdx(idx);
    setTree(PRESETS[idx].build());
    setHistory([]);
    setMoves([]);
    setShowAnswer(false);
  },
  [presetIdx]
);

const { nodes, edges, width, height } = useMemo(() =>
const path = useMemo(() => new Set(findPath(tree, tar
const expected = useMemo(() => (solved ? null : canon

const clickNode = (key: number) => {
  if (solved) return;
  const p = findPath(tree, key);
  if (p.length < 2) return; // корень крутить не с ке

  const want = canonicalNext(tree, target);
  setHistory((h) => [...h, clone(tree) as TNode]);
  setMoves((m) => [...m, { node: key, correct: want?.
  setTree((t) => rotateUp(clone(t) as TNode, key));
};

const undo = () => {
  if (history.length === 0) return;

```

```

        </button>
      )})
    </div>

    <div className="bg-slate-900 rounded-xl border bo
      <svg
        viewBox={`0 0 ${width} ${height}`}
        className="h-auto block mx-auto"
        style={{ minWidth: Math.min(width * 1.5, 420)
        role="img"
      >
        {edges.map(([a, b], i) => {
          const na = nodes.find((n) => n.key === a);
          const nb = nodes.find((n) => n.key === b);
          if (!na || !nb) return null;
          const onPath = path.has(a) && path.has(b);
          return (
            <line
              key={i}
              x1={na.x}
              y1={na.y}
              x2={nb.x}
              y2={nb.y}
              stroke={onPath ? "#6366f1" : "#475569"}
              strokeWidth={onPath ? 3 : 1.8}
              style={{ transition: "all 700ms cubic-b
            />
          );
        })}
      )})

    {nodes.map((n) => {
      const isTarget = n.key === target;
      const clickable = !solved && n.depth > 0;
      return (
        <g
          key={n.key}
          onClick={() => clickable && clickNode(n)
          style={{

```

```

</button>
<button
  onClick={() => setShowAnswer((s) => !s)}
  className={`flex items-center gap-1.5 px-3 py-2
    showAnswer
      ? "bg-amber-600/20 border-amber-500 text-slate-500"
      : "bg-slate-800 border-slate-700 text-slate-500"
    }`>
  >
    {showAnswer ? <EyeOff className="w-3.5 h-3.5" /> : "Скрыть разбор"}
  </button>

  <span className="text-[11px] text-slate-500 ml-2">
    поворотов: {moves.length} • минимум: {minimalMoves}
  </span>
</div>

{showAnswer && !solved && expected && (
  <div className="mt-3 text-[12px] bg-amber-950/40 p-3">
    Сейчас splay крутил бы узел{" "}
    <span className="font-mono font-bold">{expected}
  </div>
)}

{solved && (
  <div
    className={`mt-3 rounded-lg border px-3 py-2.5
      wrong === 0
        ? "bg-emerald-950/40 border-emerald-700/60 text-emerald-500"
        : "bg-rose-950/30 border-rose-800/60 text-rose-500"
    }`}
  >
    <div className="flex items-center gap-2 font-mono">
      {wrong === 0 ? <CheckCircle2 className="w-4 h-4" /> : <X className="w-4 h-4" />}
      {wrong === 0
        ? `Узел ${target} в корне за ${moves.length} ходов`
        : `Узел ${target} в корне за ${moves.length} ходов`}
    </div>
  </div>
)}

```

```
*   * под деревом печатается обход слева направо: он ОД
*   это и есть доказательство, что поворот ничего не
*   * кадр «прицел» ничего не двигает, только подсвечива
*   * на кадре «результат» остаются призраки – пунктирн
*   местах и стрелки «откуда → куда»;
*   * по умолчанию анимация НЕ крутится сама: пользоват
*   идёт в своём темпе. Автопрокрутка включается кноп
*/
```

```
type NodeId = "x" | "p" | "g" | "A" | "B" | "C" | "D";
type Pos = Partial<Record<NodeId, [number, number]>>;
```

```
interface Frame {
  pos: Pos;
  edges: [NodeId, NodeId][];
  kind: "start" | "aim" | "result";
  title: string;
  text: string;
  /** Пара, которую крутим на этом шаге: кадр-«прицел».
  focus?: [NodeId, NodeId];
  /** Кто переехал на этом кадре – рисуем призрак и стр
  moved?: NodeId[];
  ms: number;
}
```

```
interface Case {
  key: "zig" | "zig-zig" | "zig-zag";
  label: string;
  when: string;
  rotations: string;
  /** Номера узлов: роль → число. */
  keys: Partial<Record<NodeId, number>>;
  /** Обход слева направо – одинаковый на всех кадрах.
  inorder: string[];
  /** Что означают поддеревья A..D. */
  ranges: string;
  frames: Frame[];
}
```

```

        edges: [{"x", "A"}, {"x", "p"}, {"p", "B"}, {"p",
kind: "result",
title: "Стало: 20 – корень",
text: "40 стало правым сыном двадцатки. Поддеревов
    ева от 40 – это одно и то же место.",
moved: [{"x", "p", "B"}],
ms: RESULT_MS,
    },
],
};

```

/* ----- Zig-Zig -----

```

const ZZ_S0 = {
    pos: { g: [255, 40], D: [325, 112], p: [160, 112], C:
    edges: [{"g", "p"}, {"g", "D"}, {"p", "x"}, {"p", "C"}
} as unknown as Pick<Frame, "pos" | "edges">;

```

```

const ZZ_S1 = {
    pos: { p: [165, 40], x: [85, 112], C: [212, 112], g:
    edges: [{"p", "x"}, {"p", "g"}, {"x", "A"}, {"x", "B"}
} as unknown as Pick<Frame, "pos" | "edges">;

```

```

const ZIGZIG: Case = {
    key: "zig-zig",
    label: "Zig-Zig",
    when: "x и p – дети с ОДНОЙ стороны (оба слева или оба
    rotations: "2 поворота, первым – верхний",
    keys: { x: 20, p: 40, g: 60 },
    inorder: ["A", "20", "B", "40", "C", "60", "D"],
    ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
    frames: [
        {
            ...ZZ_S0,
            kind: "start",
            title: "Было: прямая линия 60 → 40 → 20",
            text: "Три узла выстроились в «бамбук»: каждый сл
                зглаживает.",
            ms: RESULT_MS,

```



```

/* ----- Zig-Zag -----
const ZG_S0 = {
  pos: { g: [258, 40], D: [328, 112], p: [148, 112], A: [148, 112],
  edges: [{"g", "p"}, {"g", "D"}, {"p", "A"}, {"p", "x"}],
} as unknown as Pick<Frame, "pos" | "edges">;

const ZG_S1 = {
  pos: { g: [258, 40], D: [328, 112], x: [148, 112], p: [148, 112],
  edges: [{"g", "x"}, {"g", "D"}, {"x", "p"}, {"x", "C"}],
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIGZAG: Case = {
  key: "zig-zag",
  label: "Zig-Zag",
  when: "x и p – дети с РАЗНЫХ сторон (змейка)",
  rotations: "2 поворота, первым – нижний",
  keys: { x: 40, p: 20, g: 60 },
  inorder: ["A", "20", "B", "40", "C", "60", "D"],
  ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
  frames: [
    {
      ...ZG_S0,
      kind: "start",
      title: "Было: 60 → влево на 20 → вправо на 40",
      text: "Узлы идут «змейкой»: сначала шаг влево, по",
      ms: RESULT_MS,
    },
    {
      ...ZG_S0,
      kind: "aim",
      title: "Прицел 1: НИЖНЕЕ ребро 20 – 40",
      text: "Дед (60) стоит на месте и в этом повороте",
      focus: ["p", "x"],
      ms: AIM_MS,
    },
  ],
}

```

```

x: "x — ищем его",
p: "p — родитель",
g: "g — дед",
};

const isSubtree = (id: NodeId) => id === "A" || id ===

const Tree: React.FC<{ frame: Frame; prev: Pos; keys: C
const { pos, edges, focus, moved } = frame;
const dim = (id: NodeId) => (focus ? !focus.includes(
const ghosts = frame.kind === "result" ? (moved ?? [])

return (
  <svg viewBox={`0 0 ${VB.w} ${VB.h}`} className="w-f
    <defs>
      <marker id="splay-arrow" viewBox="0 0 10 10" re
        <path d="M0,0 L10,5 L0,10 z" fill="#a5b4fc" />
      </marker>
    </defs>

    {/* призраки: где узел стоял до поворота */}
    {ghosts.map((id) => {
      const from = prev[id]!;
      const to = pos[id]!;
      const r = isSubtree(id) ? 15 : 20;
      const dx = to[0] - from[0];
      const dy = to[1] - from[1];
      const len = Math.hypot(dx, dy) || 1;
      const pad = r + 8;
      return (
        <g key={`ghost-${id}`} opacity={0.55}>
          <circle cx={from[0]} cy={from[1]} r={r} fil
            {len > pad * 2 + 6 && (
              <line
                x1={from[0] + (dx / len) * pad}
                y1={from[1] + (dy / len) * pad}
                x2={to[0] - (dx / len) * pad}
                y2={to[1] - (dy / len) * pad}

```

```

    <g
      key={id}
      style={{
        transform: `translate(${p[0]}px, ${p[1]}px)`,
        transition: MOVE,
        opacity: dim(id) ? 0.28 : 1,
      }}
    >
      {hot && <circle r={r + 7} fill="none" stroke={c.stroke} />
      <circle r={r} fill={c.fill} stroke={c.stroke} />
      <text
        textAnchor="middle"
        dominantBaseline="central"
        fontSize={sub ? 12 : 15}
        fontWeight="700"
        fill={sub ? "#94a3b8" : "#fff"}
      >
        {sub ? id : keys[id]}
      </text>
      {!sub && (
        <text textAnchor="middle" y={-r - 7} font-size={12} font-weight="bold">
          {id}
        </text>
      )}
    </g>
  );
}
</svg>
);
};

```

```

export const SplayRotationsViz: React.FC = () => {
  const [caseIdx, setCaseIdx] = useState(0);
  const [frame, setFrame] = useState(0);
  const [playing, setPlaying] = useState(false);
  const [slow, setSlow] = useState(false);

  const active = CASES[caseIdx];

```

```

    { /* шаг + пояснение стоят НАД картинкой: сначала
    <div className="mb-3 rounded-xl border border-slate-500"
      <div className="flex items-center gap-2 mb-1 flex-wrap"
        <span
          className={`text-[10px] font-bold uppercase
            current.kind === "aim"
              ? "bg-amber-500/20 text-amber-300"
              : current.kind === "start"
                ? "bg-slate-700 text-slate-300"
                : "bg-indigo-500/20 text-indigo-300"
            }`
          >
            {current.kind === "aim" ? "прицел" : current.title}
          </span>
          <span className="text-sm font-bold text-white"
            >Шаг {idx + 1} из {total}. {current.title}</span>
        </div>
        <p className="text-[13px] text-slate-400 leading-relaxed">
      </div>

    <div className="bg-slate-900 rounded-xl border border-slate-500"
      <Tree frame={current} prev={prevPos} keys={activeKeys}>

    { /* обход слева направо – одинаковый на всех карточках
    <div className="mt-2 pt-2 border-t border-slate-500"
      <div className="flex items-center gap-1.5 flex-wrap"
        <span className="text-slate-500 mr-1">Обход
          {active.inorder.map((t, i) => (
            <span
              key={i}
              className={`px-1.5 py-0.5 rounded font-mono text-sm
                /\d/.test(t) ? "bg-emerald-500/15 text-emerald-300"
                : "bg-slate-700 text-slate-300"
              }`
            >
              {t}
            </span>
          ))}
        </div>
      </div>
    )}
  )}
}

```

```

>
  {last ? "⏮ Сначала" : "Дальше →"}
</button>
<button
  onClick={() => setPlaying((p) => !p)}
  className="flex items-center gap-1.5 px-3 py-3
    ext-xs font-bold transition-colors"
>
  {playing ? <Pause className="w-3.5 h-3.5" />
    {playing ? "Стоп" : "Авто"}}
</button>
<button
  onClick={() => setSlow((s) => !s)}
  title="Автопрокрутка ещё медленнее"
  className={`flex items-center gap-1.5 px-3 py-3
    slow
    ? "bg-emerald-600/20 border-emerald-500 text-emerald-500"
    : "bg-slate-800 border-slate-700 text-slate-200"}
  `}
>
  <Gauge className="w-3.5 h-3.5" /> {slow ? "0.5x" : "1x"}
</button>
<button
  onClick={() => {
    setFrame(0);
    setPlaying(false);
  }}
  className="flex items-center gap-1.5 px-3 py-3
    ext-xs font-bold transition-colors"
>
  <RotateCcw className="w-3.5 h-3.5" /> Сброс
</button>

<div className="flex items-center gap-3 ml-auto"
  {[["x", "p", "g"] as const].map((id) =>
    active.keys[id] === undefined ? null : (
      <span key={id} className="flex items-center gap-1"
        <span className="w-3 h-3 rounded-full"

```

```

    root.left = insertBST(root.left, key);
  } else if (key > root.key) {
    root.right = insertBST(root.right, key);
  }
  return root;
};

// Right Rotation (Zig / Right Rotate)
const rightRotate = (x: SplayNode): SplayNode => {
  const y = x.left;
  if (!y) return x;
  x.left = y.right;
  y.right = x;
  return y;
};

// Left Rotation (Zag / Left Rotate)
const leftRotate = (x: SplayNode): SplayNode => {
  const y = x.right;
  if (!y) return x;
  x.right = y.left;
  y.left = x;
  return y;
};

// Clone tree helper
const cloneTree = (node: SplayNode | null): SplayNode | null => {
  if (!node) return null;
  return {
    key: node.key,
    left: cloneTree(node.left),
    right: cloneTree(node.right),
    x: node.x,
    y: node.y,
  };
};

export const SplayTreeViz: React.FC = () => {

```

```

    } else if (key < current.key) {
      path.push({ node: current, dir: "left" });
      current = current.left;
    } else {
      path.push({ node: current, dir: "right" });
      current = current.right;
    }
  }

const targetKey = current ? key : lastNode.key;
const isFound = !!current;

if (!isFound) {
  steps.push(`Поиск ${key}: элемент отсутствует в д...`);
  steps.push(
    `Поиск споткнулся на узле [${lastNode.key}]. И...`
  );
} else {
  steps.push(
    `Поиск ${key}: элемент найден! Запускаем подъем...`
  );
}

// Now let's implement standard Splay logic recursively
const splayAction = (
  node: SplayNode | null,
  k: number,
): SplayNode | null => {
  if (!node || node.key === k) return node;

  if (k < node.key) {
    if (!node.left) return node;

    if (k < node.left.key) {
      // Zig-Zig (Left-Left)
      node.left.left = splayAction(node.left.left,
        node = rightRotate(node);
      steps.push(

```

```

        });
        return leftRotate(node);
    }
};

const finalRoot = splayAction(cloneTree(root), target);
return { newRoot: finalRoot, steps };
};

const handleSplay = (key: number) => {
    setHighlightedNode(key);
    const { newRoot, steps } = splayStepByStep(tree, key);
    if (newRoot) {
        setTree(newRoot);
    }
    setLog(steps);
};

// @ts-expect-error зарезервировано под кнопку вставки
const handleInsert = (e: React.FormEvent) => {
    e.preventDefault();
    const val = parseInt(inputValue);
    if (isNaN(val)) return;

    // Standard BST insert, then splay!
    let newTree = cloneTree(tree);
    newTree = insertBST(newTree, val);
    const { newRoot, steps } = splayStepByStep(newTree, val);
    if (newRoot) {
        setTree(newRoot);
    }
    setLog([`Вставили вершину [${val}].`, ...steps]);
    setInputValue("");
    setHighlightedNode(val);
};

const handleFind = (e: React.FormEvent) => {
    e.preventDefault();

```



```

        lines.push(
          <line
            key={`l-${node.key}-${node.left.key}`}
            x1={node.x}
            y1={node.y}
            x2={node.left.x}
            y2={node.left.y}
            stroke="#475569"
            strokeWidth="2"
          />,
        );
        lines = lines.concat(renderLines(node.left));
      }
      if (node.right && node.right.x && node.right.y) {
        lines.push(
          <line
            key={`l-${node.key}-${node.right.key}`}
            x1={node.x}
            y1={node.y}
            x2={node.right.x}
            y2={node.right.y}
            stroke="#475569"
            strokeWidth="2"
          />,
        );
        lines = lines.concat(renderLines(node.right));
      }
      return lines;
    };

    const renderNodes = (node: SplayNode | null): React.R
      if (!node) return [];
      let circles: React.ReactNode[] = [];
      const isHighlighted = highlightedNode === node.key;

      circles.push(
        <g
          key={`n-${node.key}`}

```

Интерактивное Splay-дерево

```

</h3>
<p className="text-xs text-slate-400">
  Кликайте на вершины дерева, чтобы «вытащить»
  Splay.
</p>
</div>

<div className="grid grid-cols-1 lg:grid-cols-12" >
  {/* Playfield */}
  <div className="lg:col-span-7 bg-[#0f172a] p-4" >
    <svg
      className="w-full h-full max-w-[600px] max-h-[320px]"
      viewBox="0 0 600 320"
    >
      {renderLines(drawTree)}
      {renderNodes(drawTree)}
    </svg>

    {/* Quick interactive controls */}
    <div className="absolute bottom-4 left-4 right-4" >
      <form onSubmit={handleFind} className="flex" >
        <input
          type="number"
          placeholder="Поиск..."
          value={inputValue}
          onChange={(e) => setInputValue(e.target.value)}
          className="w-24 bg-slate-950 text-white"
        />
        <button
          type="submit"
          onClick={handleFind}
          className="bg-indigo-600 hover:bg-indigo-500"
        >
          Поиск / Splay
        </button>
      </form>
    </div>
  </div>
  <div className="lg:col-span-5" >
    {/* ... */}
  </div>
</div>

```

```

        <div
          key={idx}
          className={`text-xs p-2.5 rounded trans
            idx === log.length - 1
              ? "border-l-2 border-indigo-500 bg-
              : "text-slate-400 bg-slate-900/50"
            }`
        >
          {step}
        </div>
      )}
    </div>

    <div className="border-t border-slate-800 pt-2">
      <p>
        <span className="font-semibold text-slate-400">
          у самого корня. Делаем 1 вращение.
        </span>
      </p>
      <p>
        <span className="font-semibold text-slate-400">
          оба левые или оба правые. Вращаем деда, по
        </span>
      </p>
      <p>
        <span className="font-semibold text-slate-400">
          колено. Вращаем узел в разные стороны.
        </span>
      </p>
    </div>
  </div>
</div>

/* Answers Section inside the visualizer widget
<div className="bg-slate-950/80 p-5 border-t border-slate-800">
  <h4 className="text-indigo-400 font-bold text-slate-400">
    <HelpCircle className="w-4 h-4 text-indigo-400"/>
    Разбор экзаменационных вопросов (Без нитья):
  </h4>

  <div className="grid grid-cols-1 md:grid-cols-3">

```



```

const [showSoap, setShowSoap] = useState(false);
const [shakeEmpty, setShake] = useState(false);
const [log, setLog] = useState<string[]>()

const addLog = (msg: string) => setLog(prev => [msg,

// PUSH: Положить грязную тарелку сверху стопки
const pushPlate = useCallback(() => {
  if (isWashing) return;
  if (dirtyStack.length >= 7) {
    addLog('⚠ Стопка слишком высокая, тарелки могут р
    setShake(true);
    setTimeout(() => setShake(false), 400);
    return;
  }

  const preset = PLATE_PRESETS[counter % PLATE_PRESET
  counter++;

  const newPlate: Plate = {
    id: Date.now().toString(),
    ...preset,
    isDirty: true,
    animState: 'in',
  };

  setDirtyStack(prev => [...prev, newPlate]);
  addLog(` PUSH: Положили ${newPlate.emoji} сверху с

  setTimeout(() => {
    setDirtyStack(prev => prev.map(p => p.id === newP
  }, 500);
}, [dirtyStack, isWashing]);

// POP: Помыть верхнюю тарелку (LIFO)
const popPlate = useCallback(() => {
  if (isWashing) return;
  if (dirtyStack.length === 0) {

```

```

        { id: 'r3', name: 'Блюдец от торта', emoji:
          mState: 'in' },
    });
    setCleanRack([]);
    setIsWashing(false);
    setShowSoap(false);
    setLog([' Стол сброшен. Посуда снова грязная!']);
  });

  const topIndex = dirtyStack.length - 1;

  return (
    <div className="flex flex-col gap-6">
      { /* МНЕМОНИКА / ПРАВИЛО */ }
      <div className="bg-blue-950/40 border border-blue-
        <div className="text-3xl"> </div>
        <div>
          <h3 className="font-black text-blue-400 text-
          <p className="text-xs text-slate-300 mt-1 lea
            Ты складываешь грязные тарелки друг на друг
            А когда приходит время мыть, ты берёшь именн
            Попытаешься вытащить нижнюю – всё рухнет!
          </p>
        </div>
      </div>

      { /* УПРАВЛЕНИЕ */ }
      <div className="flex items-center gap-3 flex-wrap
        <button
          onClick={pushPlate}
          disabled={isWashing}
          className="flex-1 min-w-[150px] bg-gradient-t
            acity-40 disabled:cursor-not-allowed text-w
            ale-95 transition-all cursor-pointer flex i
          >
            <span> Положить грязную (PUSH)</span>
          </button>

```



```

    </div>

    <div className="flex-1 flex flex-col justify-center"
      {cleanRack.length === 0 ? {
        <div className="flex-1 flex flex-col item-center"
          <span className="text-5xl mb-2">  </span>
          <p className="text-xs font-bold font-mono">
            <p className="text-[10px] mt-1">Здесь 6
          </p>
        </div>
      } : {
        <div className="flex flex-col gap-2 w-full"
          {cleanRack.map(plate => {
            <div
              key={plate.id}
              className="w-full h-8 rounded-full flex
items-center justify-center gap-2 opacity-100"
            >
              <span className="text-sm"> {plate.name}
              <span className="text-[10px] text-end">
                </div>
            </div>
          })}
        </div>
      }}
    </div>

    {/* Столешница сушилки */}
    <div className="w-full h-4 bg-gradient-to-r from-slate-900 to-slate-800"
    </div>
  </div>

  {/* ЛОГ ДЕЙСТВИЙ */}
  <div className="bg-slate-900 border border-slate-800 p-4"
    <div className="text-[10px] font-mono text-slate-200"
      {log.map((entry, idx) => {
        <div
          key={idx}
          className={`text-xs font-mono flex items-center justify-between`}
        >

```



```

        if (s[i] === s[j]) {
            steps.push({ i, j, pi: [...pi], highlight:
                ска увеличивается.` });
            j++;
        } else {
            steps.push({ i, j, pi: [...pi], highlight:
                овпадений нет.` });
        }

        pi[i] = j;
        steps.push({ i, j, pi: [...pi], desc: `Записывае

        if (j === pattern.length) {
            steps.push({ i, j, pi: [...pi], found: true
        }
    }
    return { s, steps, patternLength: pattern.length };
}

```

```

function generateZSteps(pattern: string, text: string)
    if (!pattern) pattern = " ";
    const s = pattern + "#" + text;
    const n = s.length;
    const z = new Array(n).fill(0);
    const steps: any[] = [];

    let l = 0, r = 0;
    steps.push({ i: 0, l, r, z: [...z], desc: `Начало.

    for (let i = 1; i < n; i++) {
        steps.push({ i, l, r, z: [...z], desc: `Шаг i=$

        if (i <= r) {
            steps.push({ i, l, r, z: [...z], desc: `Вни
                !` });
            z[i] = Math.min(r - i + 1, z[i - l]);
            steps.push({ i, l, r, z: [...z], desc: `Пре

```

```

const safePattern = pattern || " ";
const safeText = text || " ";

const { s, steps, patternLength } = useMemo(() => {
  if (mode === "kmp") return generateKmpSteps(safePattern, safeText);
  else return generateZSteps(safePattern, safeText);
}, [mode, safePattern, safeText]);

const [autoPlay, setAutoPlay] = useState(false);
const [stepIdx, setStepIdx] = useState(0);
const timer = useRef<NodeJS.Timeout | null>(null);

// Reset when inputs or mode change
useEffect(() => {
  setStepIdx(0);
  setAutoPlay(false);
}, [s, mode]);

// Timer loop for autoPlay
useEffect(() => {
  if (autoPlay) {
    timer.current = setInterval(() => {
      setStepIdx(prev => {
        if (prev >= steps.length - 1) { setStepIdx(0); }
        return prev + 1;
      });
    }, 1200);
  }
  return () => { if (timer.current) clearInterval(timer.current); };
}, [autoPlay, steps.length]);

const playPause = () => setAutoPlay(!autoPlay);
const nextStep = () => { setAutoPlay(false); setStepIdx(stepIdx + 1); };
const prevStep = () => { setAutoPlay(false); setStepIdx(stepIdx - 1); };
const reset = () => { setAutoPlay(false); setStepIdx(0); };

const step = steps[stepIdx] || steps[0];

```

```

        let bgColor = "bg-slate-900";
        let textColor = isSeparator ? "text-white" : "text-slate-900";

        // KMP Logic
        if (mode === "kmp") {
            if (step.highlight?.include) {
                borderColor = "border-amber-500";
                bgColor = step.match === "K" ? "bg-amber-500" : "bg-slate-900/40";
            }
        }

        // Z Logic
        if (mode === "z") {
            if (index >= step.l && index <= step.r) {
                bgColor = "bg-emerald-900";
                borderColor = "border-emerald-500";
            }
            if (step.zTarget === index) {
                borderColor = "border-amber-500";
                bgColor = step.match === "K" ? "bg-amber-500" : "bg-slate-900/40";
            }
        }

        return (
            <div key={index} className={
                /* L/R markers for Z */
                {mode === "z" && step.l <= index <= step.r
                    ? "border-2 border-emerald-500"
                    : ""}
            >
                <div className="absolute left-0 right-0 top-0 bottom-0 flex items-center justify-center"
                    {mode === "z" && step.l <= index <= step.r
                        ? "border-2 border-amber-500"
                        : ""}
                >
                    {/* Index */}
                    <span className={`text-${textColor} font-mono font-weight-normal`}

```

```

        {mode === "z" && step.z}
        <div className="abs
            <span className
        </div>
    }}
    {mode === "z" && step.z}
        <div className="abs
            <span className
        </div>
    }}

    {/* Found flash effect
    {step.found && mode ===
        <div className="abs
ulse z-0 bg-emerald-500/20 shadow-[0_0_15px
    }}
    {step.found && mode ===
        <div className="abs
ulse z-0 bg-emerald-500/20 shadow-[0_0_15px
    }}
    </div>
    )
    }}}}
    </div>
</div>

<div className="flex flex-col sm:flex-row g
    {/* Controls */}
    <div className="flex bg-slate-900 borde
        <button onClick={prevStep} disabled:
rounded-lg disabled:opacity-30 disabled:hov
        <Undo strokeWidth={3} size={16}
    </button>
        <button onClick={playPause} classNa
x items-center justify-center min-w-[80px]"
        {autoplay ? <><Pause size={16}
    </button>
        <button onClick={nextStep} disabled:

```

```

        <div className="bg-slate-800 px-4 py-2 b
late-400 uppercase">
            <span>Код C++ ({mode === 'kmp' ? 'КМП
            </div>
            <pre className="p-4 text-xs font-mono te
{mode === 'kmp' ? `vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
    int j = pi[i-1];
    while (j > 0 && s[i] != s[j]) j = pi[j-1];
    if (s[i] == s[j]) j++;
    pi[i] = j;
}` : `int l = 0, r = 0;
for (int i = 1; i < n; i++) {
    if (i <= r) z[i] = min(r - i + 1, z[i - l]);
    while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
    if (i + z[i] - 1 > r) {
        l = i;
        r = i + z[i] - 1;
    }
}`}`
            </pre>
        </div>
    </div>
    );
}

```

ФАЙЛ 66/82: src/components/TopologicalSortViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```

import React, { useState, useEffect } from "react";
import {
    Play,
    Pause,
    RotateCcw,
    Clock,

```

```

}

export const TopologicalSortViz: React.FC = () => {
  const [steps, setSteps] = useState<TStep[]>([]);
  const [currentStepIdx, setCurrentStepIdx] = useState(0);
  const [isPlaying, setIsPlaying] = useState(false);

  useEffect(() => {
    const listSteps: TStep[] = [];
    const visited = new Set<number>();
    const fullyProcessed = new Set<number>();
    const topoList: number[] = [];
    const dfsStack: number[] = [];

    const recordStep = (desc: string, currentNode: number) => {
      listSteps.push({
        desc,
        visited: new Set(visited),
        fullyProcessed: new Set(fullyProcessed),
        currentNode,
        topoList: [...topoList],
        dfsStack: [...dfsStack],
      });
    };

    recordStep(
      "Начало топологической сортировки. Используем кла",
      null,
    );

    // Adj list
    const adj: number[][] = Array.from({ length: 5 }, () => []);
    dagEdges.forEach((e) => adj[e.u].push(e.v));

    const dfs = (u: number) => {
      visited.add(u);
      dfsStack.push(u);
      recordStep(

```

```

        "Топологическая сортировка завершена! Полученный
        null,
    );
    setSteps(listSteps);
    setCurrentStepIdx(0);
}, []);

const step = steps[currentStepIdx] || {
  desc: "Заныск...",
  visited: new Set(),
  fullyProcessed: new Set(),
  currentNode: null,
  topoList: [],
  dfsStack: [],
};

useEffect(() => {
  let timer: NodeJS.Timeout;
  if (isPlaying && currentStepIdx < steps.length - 1)
    timer = setInterval(() => {
      setCurrentStepIdx((prev) => prev + 1);
    }, 1600);
  } else {
    setIsPlaying(false);
  }
  return () => clearInterval(timer);
}, [isPlaying, currentStepIdx, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
  setIsPlaying(false);
  setCurrentStepIdx(0);
};

const getTopoListString = () => {
  return [...step.topoList]
    .reverse()
    .map((id) => dagNodes[id].name)

```

```

<svg className="w-full h-full max-w-[650px]"
  <defs>
    <marker
      id="dag-arrow"
      viewBox="0 0 10 10"
      refX="24"
      refY="5"
      markerWidth="6"
      markerHeight="6"
      orient="auto-start-reverse"
    >
      <path d="M 0 1 L 10 5 L 0 9 z" fill="#4
    </marker>
    <marker
      id="dag-arrow-active"
      viewBox="0 0 10 10"
      refX="24"
      refY="5"
      markerWidth="6"
      markerHeight="6"
      orient="auto-start-reverse"
    >
      <path d="M 0 1 L 10 5 L 0 9 z" fill="#8
    </marker>
  </defs>

  {/* Edges */}
  {dagEdges.map((edge, idx) => {
    const u = dagNodes.find((n) => n.id === e
    const v = dagNodes.find((n) => n.id === e

    const isActive =
      (step.currentNode === edge.u || step.cu
      step.visited.has(edge.v));

    return (
      <line
        key={`edge-${idx}`}

```



```

        <g key={node.id}>
          <rect
            x={node.x - 55}
            y={node.y - 22}
            width="110"
            height="44"
            rx="8"
            fill={fill}
            stroke={stroke}
            strokeWidth="2"
            className="transition-all duration-1
          />
          <text
            x={node.x}
            y={node.y - 4}
            textAnchor="middle"
            fill="white"
            fontSize="11px"
            fontWeight="bold"
            className="pointer-events-none"
          >
            {{node.label}} {node.name.split(" "
          </text>
          <text
            x={node.x}
            y={node.y + 12}
            textAnchor="middle"
            fill="#94a3b8"
            fontSize="9px"
            className="pointer-events-none"
          >
            {node.name.split(" ").slice(1).join
          </text>
        </g>
      );
    }}}
  </svg>
</div>

```

```

        <span className="w-3.5 h-3.5 rounded bg-
        <span className="text-slate-400 text-[1
        Серые (зашли)
        </span>
    </div>
    <div className="flex items-center gap-2 t
        <span className="w-3.5 h-3.5 rounded bg-
        <span className="text-slate-400 text-[1
        Черные (готово)
        </span>
    </div>
</div>
</div>
</div>

<div className="mt-4 pt-3 border-t border-sla
    <span>
        Шаг {currentStepIdx + 1} из {steps.length}
    </span>
    <div className="flex gap-1">
        <button
            disabled={currentStepIdx === 0}
            onClick={() => setCurrentStepIdx((prev)
            className="bg-slate-900 border border-s
t-white"
        >
            Назад
        </button>
        <button
            disabled={currentStepIdx >= steps.length}
            onClick={() => setCurrentStepIdx((prev)
            className="bg-slate-900 border border-s
t-white"
        >
            Вперед
        </button>
    </div>
</div>
</div>
</div>

```

```

    y: number;
    parentKey: number | null;
}
interface PathHit {
    atKey: number;
    cmp: string;
    dir: "L" | "R";
}
interface Step {
    tree: Placed[];
    edges: { from: number; to: number; fresh: boolean }[];
    candidate: Pair | null;
    candidateAt: { x: number; y: number } | null;
    pathKeys: number[];
    justPlacedKey: number | null;
    doneKeys: number[];
    log: string;
    codeLine: number;
}

const HIRES: Pair[] = [
    { key: 45, grade: 8 },
    { key: 3, grade: 6 },
    { key: 3, grade: 5 },
    { key: 6, grade: 5 },
    { key: 2, grade: 4 },
    { key: 9, grade: 3 },
    { key: 7, grade: 2 },
    { key: 1, grade: -4 },
];

const CODE_LINES = [
    { line: 1, text: "root = None" },
    { line: 2, text: "for key, pri in pairs_sorted_by_pri" },
    { line: 3, text: "    cur, parent = root, None" },
    { line: 4, text: "    while cur:" },
    { line: 5, text: "        parent = cur" },
    { line: 6, text: "        cur = cur.left if key <= cu" }
];

```

```

        justPlacedKey: null,
        doneKeys: [...doneKeys],
        log,
        codeLine,
        ...extra,
    });
};

snap("  Корпорация пуста. Приказ о найме: всех выстрои

for (const hire of HIRES) {
    const label = `№${hire.key} (грейд ${hire.grade})`;
    let cur = root;
    let parent: TNode | null = null;
    let side: "L" | "R" | null = null;
    const pathHits: PathHit[] = [];
    const pathKeys: number[] = [];

    // кандидат «идёт» слева от входа, пока не начало с
    const candidateFloat = { x: 70, y: 40 };
    steps.push({
        ...snapToLists(),
        candidate: hire,
        candidateAt: candidateFloat,
        log: `  Вызывают кандидата ${label}. Вход в иерар
        codeLine: 2,
    });

    while (cur) {
        const atNode = cur;
        const dir: "L" | "R" = hire.key <= atNode.key ? "
        const cmp = `${hire.key} ${dir === "L" ? "≤" : ">
        pathHits.push({ atKey: atNode.key, cmp, dir });
        pathKeys.push(atNode.key);
        const { placed, edges } = layout(root);
        const at = placed.find((p) => p.key === atNode.ke
        steps.push({
            tree: placed,

```

```

snap("  Штат укомплектован: BST по табельным №, куча

function snapToLists(): Step {
  const { placed, edges } = layout(root);
  return {
    tree: placed,
    edges: edges.map((e) => ({ ...e, fresh: false })),
    candidate: null,
    candidateAt: null,
    pathKeys: [],
    justPlacedKey: null,
    doneKeys: [...doneKeys],
    log: "",
    codeLine: 3,
  };
}

return steps;
}

export const TreapBuildViz = () => {
  const steps = useMemo(buildSteps, []);
  const [idx, setIdx] = useState(0);
  const [playing, setPlaying] = useState(false);

  useEffect(() => {
    let timer: ReturnType<typeof setTimeout> | null = null;
    if (playing && idx < steps.length - 1) {
      timer = setTimeout(() => setIdx((v) => v + 1), 1400);
    } else if (idx >= steps.length - 1) {
      setPlaying(false);
    }
    return () => {
      if (timer) clearTimeout(timer);
    };
  }, [playing, idx, steps.length]);

  const step = steps[idx];

```

```

        setPlaying(false);
        if (idx < steps.length - 1) setIdx(idx + 1)
    }}
    disabled={idx >= steps.length - 1}
    className="flex items-center gap-1 bg-indigo-500
    px-3 py-2 rounded-lg text-sm font-medium tr
    >
    ⏮
  </button>
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6 m
  {/* Дерево */}
  <div className="w-full lg:w-2/3 bg-emerald-950/50
    justify-center min-h-[400px]">
    <div className="absolute top-3 left-3 bg-emerald-950/50
      0 font-bold shadow-sm">
      ⏮ {idx + 1} из {steps.length}
    </div>
    {step.candidate && {
      <div className="absolute top-3 right-3 bg-indigo-500
        font-bold shadow">
        Кандидат: №{step.candidate.key} • гр. {step.candidate.group}
      </div>
    }}
  </div>
  <div className="w-full h-auto max-h-[440px] m
    {step.edges.map((e) => {
      const a = posByKey.get(e.from);
      const b = posByKey.get(e.to);
      if (!a || !b) return null;
      const onPath = step.pathKeys.includes(e.from);
      const isFresh = freshEdge && freshEdge.to === e.to;
      return (
        <line
          key={` ${e.from} - ${e.to} `}
          x1={a.x}

```

```

        {/* Кандидат «парит» над узлом, с которым с
        {step.candidate && step.candidateAt && !step
        <g transform={`translate(${step.candidate
        <circle r={17} fill="#4f46e5" stroke="#
        <text y={-2} fill="#ffffff" fontSize="1
        {step.candidate.key}
        </text>
        <text y={10} fill="#c7d2fe" fontSize="9
        rp.{step.candidate.grade}
        </text>
        </g>
        }}
    </svg>

    <div className="w-full bg-slate-950/80 p-4 ro
        <p className="font-semibold text-amber-400
        <p className="min-h-[40px] flex items-cente
    </div>
</div>

{/* Очередь найма */}
<div className="w-full lg:w-1/3 bg-emerald-950/
    <h4 className="text-lg font-bold text-emerald
    <div className="space-y-2.5 flex-1 overflow-y
        {HIRES.map((h) => {
            const isCurrent = step.candidate?.key ===
            const isDone = doneSet.has(h.key);
            return (
                <div
                    key={` ${h.key} - ${h.grade} `}
                    className={`p-3 rounded-lg border tra
                        isCurrent
                            ? "bg-indigo-900/70 border-indigo
                            : isDone
                                ? "bg-emerald-950/30 border-eme
                                : "bg-slate-800/40 border-slate
                    }` }
                >

```

```

                №{h.key}
            </td>
            <td className="py-2.5 px-3 font-monospace">
            <td className="py-2.5 px-3 font-monospace">
            <td className="py-2.5 px-3 text-slate-600">
            -"}</td>
            <td className="py-2.5 px-3 text-slate-600">
            entKey)?.x ?? placed.x) ? "← левый" : "правый"
            </tr>
        );
    }}}
</tbody>
</table>
</div>

{/* Код */}
<div className="w-full lg:w-1/2 bg-slate-900/60">
  <h4 className="text-lg font-bold text-slate-200">
  <div className="bg-slate-950/80 rounded-lg p-4">
    {CODE_LINES.map((item) => {
      const isActive = step.codeLine === item.line;
      return (
        <div
          key={item.line}
          className={`py-1.5 px-3 rounded flex items-center justify-between`}
          style={isActive ? "bg-indigo-900/80 text-white" : ""}
        >
          <span className="text-slate-600 select-none">{item.line}</span>
          <span className="whitespace-pre flex-grow-1">{item.code}</span>
          {isActive && <span className="text-white font-mono font-size-0.8em">
            </span>
          }
        </div>
      );
    })}
  </div>
</div>
</div>
</div>

```



```
-----
import { Simulator } from "../Simulator";
import { PrefixSumViz } from "../visualizers/PrefixSumViz";
import { SparseTableViz } from "../visualizers/SparseTableViz";
import { TreapBuildViz } from "../TreapBuildViz";

/** Разбор поворота и песочница всегда идут парой: посмотри
**
* Анимация и песочница идут друг под другом, а не в две
* колонке дерево сжималось до нечитаемого размера, а по
* компактности. Между ними – подпись, объясняющая перемену
*/
const SplayRotationsPair: React.FC = () => (
  <div className="space-y-4">
    <SplayRotationsViz />
    <p className="flex items-start gap-2 text-[12px] leading-
      3 py-2">
      <span aria-hidden="true"> </span>
      <span>
        Разобрался – проверь себя: ниже то же самое дерево
        не решишь.
      </span>
    </p>
    <SplayRotationSandbox />
  </div>
);

export interface VizEntry {
  /** Заголовок панели/модалки. */
  title: string;
  /** Короткое пояснение, что здесь можно потыкать. */
  hint?: string;
  Component: React.FC;
}

/**
* Единый реестр интерактивных демонстраций.
* Используется и для панели под главой, и для всплывающей
* и для модального окна «Открыть симулятор».
```

```

"prefix-sums-2d": {
  title: "Префиксные суммы (1D и 2D)",
  hint: "Наведите на число — увидите, из чего оно состоит",
  Component: PrefixSumViz,
},
"dynamic-programming": {
  title: "Динамическое программирование",
  hint: "Таблица подзадач заполняется на глазах.",
  Component: DPVisualizer,
},
"splay-tree": {
  title: "Splay-дерево",
  hint: "Покадровый разбор трёх поворотов, песочница",
  Component: () => (
    <div className="space-y-10">
      <SplayRotationsPair />
      <SplayTreeViz />
    </div>
  ),
},
"splay-rotations": {
  title: "Splay: Zig, Zig-Zig и Zig-Zag по шагам",
  hint: "Сверху — покадровый разбор поворота, снизу —",
  Component: () => <SplayRotationsPair />,
},
"graph-dfs-bfs": { title: "DFS и BFS на графе", Component: },
"top-sort": { title: "Топологическая сортировка", Component: },
"scc-kosaraju": { title: "Компоненты сильной связности", Component: },
"graph-articulation": { title: "Точки сочленения", Component: },
"bridges-code": { title: "Мосты: DFS + tin/low", Component: },
"euler-path-vs-cycle": { title: "Эйлеров путь и цикл", Component: },
"planarity-euler-formula": { title: "Планарность: KS", Component: },
dijkstra: {
  title: "Дейкстра",
  hint: "Жадно забираем ближайшую вершину и релаксируем",
  Component: () => (
    <>
    <ChapterImage vizType="dijkstra" />
  )
}

```

```
-----
intro: { title: "Мнемокарточки", Component: MnemonicC
"everyday-basics": {
  title: "Бытовой тренажёр: стек, очередь, куча",
  hint: "Тарелки, очередь в столовой и мешок гипотез",
  Component: Simulator,
},
};
```

```
export const getViz = (id?: string): VizEntry | undefined
```

```
-----
ФАЙЛ 69/82: src/components/WaterfallAnimationWidget.tsx
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО
```

```
-----
import React, { useState } from 'react';
import { ArrowRight, CheckCircle2, AlertCircle, Lightbulb } from 'lucide-react';

// Структура заранее подготовленного красивого бора для
// Словарь: ["HE", "SHE", "HIS", "HERS"]
interface WNode {
  id: number;
  label: string; // путь от корня
  char: string;
  x: number; // Координаты для SVG водопада
  y: number;
  depth: number;
  fail: number;
  term_link: number;
  is_terminal: boolean;
  words: string[];
}
```

```
const WATERFALL_NODES: { [key: number]: WNode } = {
  0: { id: 0, label: 'ROOT', char: 'ø', x: 400, y: 60,
    // Ветка H -> E -> R -> S
  1: { id: 1, label: 'H', char: 'H', x: 280, y: 160, de
```

```

    });
    const [jumpPath, setJumpPath] = useState<{ from: number; to: number; }>();
    const [foundMatches, setFoundMatches] = useState<{ index: number; match: string; }[]>();
    const [activeSearchCodeLine, setActiveSearchCodeLine] = useState<number>(1);

    // === СОСТОЯНИЯ РЕЖИМА ОБУЧЕНИЯ (TRAINING) ===
    // Шаги обучения от 0 до 8 (по числу вершин в боре, и т.д.)
    const [trainStep, setTrainStep] = useState<number>(0);

    const handleTextChange = (e: React.ChangeEvent<HTMLInputElement>) => {
        const clean = e.target.value.toUpperCase().replace(/ /g, '');
        setTextToScan(clean);
        resetSearchSimulation();
    };

    const resetSearchSimulation = () => {
        setCurrentIndex(0);
        setCurrentNode(0);
        setIsSearchDone(false);
        setSearchLog('Симуляция сброшена. Лосось вернулся на старт');
        setJumpPath(null);
        setFoundMatches([]);
        setActiveSearchCodeLine(1);
    };

    const stepSearchSimulation = () => {
        if (currentIndex >= textToScan.length) {
            setIsSearchDone(true);
            setSearchLog('Мы проплыли весь текст! Лосось доволен!');
            setActiveSearchCodeLine(4);
            return;
        }

        const char = textToScan[currentIndex];
        let curr = currentNode;
        let logMsg = `[Символ '${char}']`;

        let jumpType: 'normal' | 'fail' | null = null;
    
```

```

                matchedWordsForLog.push(word);
            });
        }
        temp = WATERFALL_NODES[temp].term_link;
    }

    if (matchedWordsForLog.length > 0) {
        logMsg += `    БИНГО! Найдено слово: ${matchedWords}
        setActiveSearchCodeLine(4);
    }

    setCurrentNode(curr);
    setCurrentIndex(currentIndex + 1);
    setSearchLog(logMsg);
    if (currentMatches.length > 0) {
        setFoundMatches((prev) => [...prev, ...currentMat
    }
};

// ПОЛНАЯ ПОШАГОВАЯ ИНФОРМАЦИЯ О ПОСТРОЕНИИ ДЕРЕВА (В
const TRAINING_STEPS_INFO = [
    {
        targetNodeId: 0,
        title: 'Шаг 0 (Depth 0): Вершина #0 (ROOT)',
        desc: 'Начало алгоритма BFS. Корень (ROOT) не обра
            рние вершины сразу инициализируются с fail = RO
        codeLine: 1,
        scoutPos: 0,
        inspectedParentFail: 0,
        checkTargetChar: 'ø',
        checkingBranchesFrom: null,
    },
    {
        targetNodeId: 1,
        title: 'Шаг 1 (Depth 1): Вершина #1 (H)',
        desc: 'Мы обрабатываем первую вершину из очереди -
            просто не существует. fail[H] автоматически нап
        codeLine: 5,
    },

```

```

{
  targetNodeId: 8,
  title: 'Шаг 5 (Depth 2): Вершина #8 (SH) – КАК СТИТЬ?',
  desc: 'ВАЖНЫЙ ВОПРОС! Обрабатываем #8 (SH) на Depth 2. Родитель: #7 (S). Рыба плавает в ROOT и ищет ветку "H". У корня суффикс "H"!',
  codeLine: 4,
  scoutPos: 0,
  inspectedParentFail: 0,
  checkTargetChar: 'H',
  checkingBranchesFrom: 0,
},
{
  targetNodeId: 3,
  title: 'Шаг 6 (Depth 3): Вершина #3 (HER)',
  desc: 'Переходим на Depth 3 к #3 (HER). Родитель: #2 (R). Но не подходят. fail[HER] = ROOT.',
  codeLine: 3,
  scoutPos: 0,
  inspectedParentFail: 0,
  checkTargetChar: 'R',
  checkingBranchesFrom: 0,
},
{
  targetNodeId: 6,
  title: 'Шаг 7 (Depth 3): Вершина #6 (HIS)',
  desc: 'Обрабатываем #6 (HIS) на Depth 3. Родитель: #5 (H). Но ветка "S" → #7 (S) совпадает! fail[HIS] = #7 (S).',
  codeLine: 4,
  scoutPos: 0,
  inspectedParentFail: 0,
  checkTargetChar: 'S',
  checkingBranchesFrom: 0,
},
{
  targetNodeId: 9,
  title: 'Шаг 8 (Depth 3): Вершина #9 (SHE) – КАК СТИТЬ?',
  desc: 'МАГИЯ АХО-КОРАСИК! Обрабатываем #9 (SHE) на Depth 3. Родитель: #8 (SH). Но не подходит. fail[SHE] = ROOT.',
  codeLine: 3,
  scoutPos: 0,
  inspectedParentFail: 0,
  checkTargetChar: 'S',
  checkingBranchesFrom: 0,
},

```

```

    </div>

    {/* Переключатель режимов */}
    <div className="bg-slate-900 p-1.5 rounded-xl b
      <button
        onClick={() => setActiveMode('training')}
        className={`flex items-center space-x-1.5 p
          activeMode === 'training'
            ? 'bg-indigo-600 text-white shadow-lg s
            : 'text-slate-400 hover:text-white'
          }`
        >
        <GitBranch className="w-4 h-4" />
        <span>Режим 1: Полное обучение (BFS от корн
      </button>
      <button
        onClick={() => setActiveMode('search')}
        className={`flex items-center space-x-1.5 p
          activeMode === 'search'
            ? 'bg-blue-600 text-white shadow-lg sha
            : 'text-slate-400 hover:text-white'
          }`
        >
        <Play className="w-4 h-4" />
        <span>Режим 2: Поиск в тексте</span>
      </button>
    </div>
  </div>

```

```

{/* ПАНЕЛЬ УПРАВЛЕНИЯ В ЗАВИСИМОСТИ ОТ РЕЖИМА */}
{activeMode === 'training' ? {
  /* ПАНЕЛЬ РЕЖИМА ОБУЧЕНИЯ */
  <div className="grid grid-cols-1 lg:grid-cols-3
    {/* Управление шагами обучения */}
    <div className="bg-slate-900/90 p-5 rounded-x
      <div>
        <h5 className="text-indigo-300 font-bold
        <span> </span> Полный обход в ширину (В

```

```

        >
        {trainStep === TRAINING_STEPS_INFO.leng
    </button>
    </div>
</div>

{/* Блок с кодом построения fail-ссылки и опи
<div className="lg:col-span-2 bg-slate-900/90
    >
    <div>
        <h5 className="text-white font-bold mb-3
            <span className="flex items-center gap-
                <span>❌</span> Код BFS построения fail
            </span>
            <span className="text-xs bg-indigo-950 l
                Вершина: #{targetTrainingNode.id} ({t
            </span>
        </h5>
        <div className="bg-slate-950 p-4 rounded-
            <div className={`p-1.5 rounded flex ite
-l-4 border-indigo-500 text-indigo-300 font
                <span>1. let u = trie[r][char]; // Те
                {currentTrainInfo.codeLine === 1 && <
            тут</span>}
        </div>
        <div className={`p-1.5 rounded flex ite
-l-4 border-indigo-400 text-indigo-200 font
                <span>2. let v = fail[r]; // Подъем к
            ainInfo.inspectedParentFail].label}}</span>
                {currentTrainInfo.codeLine === 2 && <
            к родителю</span>}
        </div>
        <div className={`p-1.5 rounded flex ite
l-4 border-amber-500 text-amber-300 font-bo
                <span>3. while (v !== root && trie[v]
                {currentTrainInfo.codeLine === 3 && <
            т, возврат в корень</span>}
        </div>

```



```

    <input
      type="text"
      value={textToScan}
      onChange={handleTextChange}
      maxLength={15}
      placeholder="ВВЕДИТЕ ТЕКСТ..."
      className="w-full bg-slate-800 border border-blue-500 focus:outline-none focus:border-blue-500"
    />
  </div>

  <div>
    <div className="mb-4 flex flex-wrap gap-1">
      <span className="text-xs text-slate-400">
        {textToScan.split('').map(({char, idx}) =>
          <span
            key={idx}
            className={`w-7 h-8 flex items-center justify-center
              ${idx === currentIndex ? 'bg-blue-600 text-white shadow' :
                idx < currentIndex ? 'bg-emerald-950/60 text-emerald-500' :
                'bg-slate-800 text-slate-500'}
            `}
          >
            {char}
          </span>
        )}
      </span>
    </div>
    {currentIndex >= textToScan.length && (
      <span className="bg-emerald-600 text-white text-sm font-medium">
        ГОТОВО ✓
      </span>
    )}
  </div>

  <button
    onClick={stepSearchSimulation}
    disabled={isSearchDone || textToScan.length === 0}
  >
    {isSearchDone ? 'ОК' : 'СЛЕДУЮЩИЙ'}
  </button>

```

```

        l-4 border-emerald-500 text-emerald-300 font-
        <span>4. if (is_terminal[curr]) report
        {activeSearchCodeLine == 4 && <span>
овпадение!</span>}
        </div>
    </div>
</div>

<div>
    <h5 className="text-slate-300 font-bold m-0">
        <AlertCircle className="w-3.5 h-3.5 text-slate-300"/>
    </h5>
    <div className="bg-slate-800 p-3.5 rounded-lg flex items-center">
        {searchLog}
    </div>
</div>
</div>
</div>
))

```

```

{/* SVG Анимация водопада */}
<div className="bg-gradient-to-b from-blue-950 via-blue-900 to-transparent pointer-events-none">
    <div className="absolute inset-0 opacity-10 bg-blue-900 to-transparent pointer-events-none"></div>
    <div className="absolute top-0 left-1/2 -translate-x-1/2">
        <div className="w-8 bg-blue-400 h-full animate-pulse"></div>
        <div className="w-12 bg-blue-300 h-full animate-pulse"></div>
        <div className="w-6 bg-blue-500 h-full animate-pulse"></div>
    </div>

    <div className="flex items-center justify-between">
        <h5 className="text-white font-bold text-base">
            <span> </span> Ветвящийся синий водопад (Борис)
        </h5>
        <div className="flex flex-wrap gap-4 text-xs">

```

```

    }}}

    { /* Рендер прямых переходов (потоков воды) */
    Object.entries(TRIE_TRANSITIONS).map(([fromNode, toNode]) => {
      const fromNode = WATERFALL_NODES[Number(fromNode)];
      return Object.entries(children).map(([character, toId]) => {
        const toNode = WATERFALL_NODES[toId];

        // Подсветка в режиме поиска
        let isHighlighted = activeMode === 'search' ?
        type === 'normal';

        // Подсветка в режиме обучения (когда ищем)
        let isTrainingExamined = activeMode === 'train' ?
        let isMatchBranch = isTrainingExamined && character === toNode.character;

        return (
          <g key={`edge-${fromNode.id}-${toNode.id}`}>
            { /* Линия течения воды */ }
            <line
              x1={fromNode.x}
              y1={fromNode.y}
              x2={toNode.x}
              y2={toNode.y}
              stroke={isHighlighted ? '#3b82f6' : '#ccc'}
              strokeWidth={isHighlighted || isTrainingExamined ? 2 : 1}
              className={isHighlighted || isTrainingExamined ? 'highlighted' : ''}
            />
            { /* Символ перехода */ }
            <circle cx={{(fromNode.x + toNode.x) / 2}} cy={{(fromNode.y + toNode.y) / 2}}
              r={10}
              fill={isTrainingExamined ? (isMatchBranch ? '#10b981' : '#f43f5f') : '#ccc'}
              stroke={isHighlighted ? '#3b82f6' : '#ccc'}
              strokeWidth={isHighlighted || isTrainingExamined ? 2 : 1}
            />
            <text
              x={{(fromNode.x + toNode.x) / 2}}
              y={{(fromNode.y + toNode.y) / 2 + 15}}
              fill={isTrainingExamined ? (isMatchBranch ? '#10b981' : '#f43f5f') : '#ccc'}
              fontSize="12"
              fontWeight="bold"
              textAnchor="middle"
            />
          </g>
        );
      });
    });
  }
}

```

```

const cx = node.x + dx / 2 + (node.id % 2) * dx;
const cy = node.y + dy / 2 - 30;

return (
  <g key={`fail-${node.id}`}>
    <path
      d={`M ${node.x} ${node.y} Q ${cx} ${cy} ${node.x} ${node.y}`}
      fill="none"
      stroke={isHighlighted ? '#f43f5e' : '#10b98f'}
      strokeWidth={isHighlighted || isJustEstablished ? 6 : 2}
      strokeDasharray="6,6"
      className={isHighlighted || isJustEstablished ? 'highlighted' : ''}
      opacity={isHighlighted || isJustEstablished ? 1 : 0.5}
    />
    {isJustEstablished && (
      <text x={cx} y={cy - 10} fill="#10b98f">
        fail[{node.label}] = {targetNode.label}
      </text>
    )}
  </g>
);
}}
}}

```

```

{/* Рендер вершин (бассейнов водопада) */}
Object.values(WATERFALL_NODES).map((node) => {
  const isCurrent = activeFishPosNode.id === node.id;
  const isTargetChild = activeMode === 'target' && node.id === targetNode.id;
  const hasWords = node.words.length > 0;

```

```

return (
  <g key={`node-${node.id}`} className={isCurrent ? 'current' : ''}>
    {/* Внешнее свечение для активной вершины */}
    {isCurrent && (
      <circle cx={node.x} cy={node.y} r="100" fill="none" stroke="red" stroke-width="2" />
    )}
    {isTargetChild && (
      <circle cx={node.x} cy={node.y} r="100" fill="none" stroke="blue" stroke-width="2" />
    )}

```

```

        <circle cx="0" cy="0" r="9" fill=
        <text x="0" y="3" fontSize="10" f
            *
        </text>
    </g>
    })
</g>
);
}}}
```

{/* ОДИН ПЛАВНЫЙ АНИМИРОВАННЫЙ ЭХО-ЛОСОСЬ */}

```

<g>
    { /* Круг-подложка под рыбу */ }
    <circle
        cx={activeFishPosNode.x + 15}
        cy={activeFishPosNode.y - 25}
        r="18"
        fill={activeMode === 'training' ? '#8b5'
        stroke="#ffffff"
        strokeWidth="2"
        style={{ transition: 'cx 0.8s cubic-bez
        className="shadow-lg"
    />
    { /* Сама рыба */ }
    <text
        x={activeFishPosNode.x + 15}
        y={activeFishPosNode.y - 19}
        fontSize="18"
        textAnchor="middle"
        style={{ transition: 'x 0.8s cubic-bezi
        className="animate-float select-none"
    >
        {activeMode === 'training' ? ' ♂ ' :
    </text>
</g>
```

```

</svg>
</div>
```

ФАЙЛ 70/82: `src/components/visualizers/PrefixSumViz.tsx`

КАТЕГОРИЯ: Визуализаторы (вложенные) | СТРОК: 330 | РАЗМ:

```
import React, { useMemo, useState } from "react";

/**
 * Префиксные суммы: 1D и 2D.
 *
 * Главная фишка – наведение на число:
 * * в 1D наводим на P[i] – подсвечивается кусок массива
 * * в 1D наводим на a[i] – видно, в какие префиксы он входит
 * * в 2D наводим на ячейку – подсвечивается прямоугольник
 * а при выбранном запросе показывается формула включения
 */

const A1 = [3, 1, 4, 1, 5, 9, 2, 6];

const A2 = [
  [1, 2, 3, 4],
  [5, 6, 7, 8],
  [9, 1, 2, 3],
  [4, 5, 6, 7],
];

const cellBase =
  "flex items-center justify-center rounded-md font-mono text-sm" +
  "w-[clamp(30px,9vw,46px)] h-[clamp(30px,9vw,46px)] text-center";

export const PrefixSumViz: React.FC = () => {
  const [mode, setMode] = useState<"1d" | "2d">("1d");
  return (
    <div className="w-full bg-slate-950 rounded-2xl border border-slate-800">
      <div className="flex gap-2 mb-4 overflow-x-auto">
        {
          mode === "1d" ? (
            <div>1D</div>
          ) : (
            <div>2D</div>
          )
        }
      </div>
      <div>
        {
          mode === "1d" ? (
            <div>1D</div>
          ) : (
            <div>2D</div>
          )
        }
      </div>
    </div>
  );
};
```

```

const sum = pref[range.r + 1] - pref[range.l];

return (
  <div className="space-y-5">
    <div className="overflow-x-auto pb-1">
      <div className="inline-block min-w-full">
        {/* индексы */}
        <div className="flex gap-1.5 mb-1 pl-[52px]">
          {A1.map((_, i) => (
            <div key={i} className={`${cellBase} !h-5
              {i}
            </div>
          ))}
        </div>

        {/* массив a */}
        <div className="flex gap-1.5 items-center mb-1">
          <div className="w-[46px] shrink-0 text-right">
            {A1.map((v, i) => {
              const inCover = covered && i >= covered.f
              const inRange = i >= range.l && i <= range.r
              return (
                <div
                  key={i}
                  onMouseEnter={() => setHover({ kind: 'cover' })}
                  onMouseLeave={() => setHover(null)}
                  onClick={() => setRange((r) => (i < r ? i : r))}
                  className={`${cellBase} cursor-pointer
                    ${inCover ? 'bg-indigo-500 text-white ring-1 ring-indigo-500' : inRange
                      ? 'bg-indigo-900/70 text-indigo-300' : 'bg-slate-800 text-slate-300'}
                  `}
                  title={`a[${i}] = ${v}`}
                >
                  {v}

```

```

    {hover?.kind === "pref" ? {
      hover.i === 0 ? {
        <p className="text-slate-300">
          <b className="text-emerald-400">P[0] = 0</b>
          отрезка, начинающегося с нуля.
        </p>
      } : {
        <p className="text-slate-300">
          <b className="text-emerald-400">
            P[{hover.i}] = {pref[hover.i]}
          </b>{" "}
          – это сумма всего, что набежало от начала
          <span className="font-mono text-indigo-300">
            a[0..{hover.i - 1}] = {A1.slice(0, hove
          </span>
        </p>
      }
    } : hover?.kind === "a" ? {
      <p className="text-slate-300">
        <b className="text-indigo-400">
          a[{hover.i}] = {A1[hover.i]}
        </b>{" "}
        входит во все префиксы, начиная с{" "}
        <span className="font-mono text-emerald-300">
          подвинуть границу запроса.
        </span>
      </p>
    } : {
      <p className="text-slate-500">
        Наведите курсор на любое число: у <b classN
        у <b className="text-indigo-400">a</b> – ку
      </p>
    }
  })
</div>

{/* Запрос */}
<div className="bg-slate-900 border border-indigo
  <div className="text-xs text-slate-400 mb-2">
    Запрос суммы на отрезке (кликайте по массиву

```



```

const cornerOf = (i: number, j: number) => {
  if (i === rect.r2 + 1 && j === rect.c2 + 1) return "A";
  if (i === rect.r1 && j === rect.c2 + 1) return "B";
  if (i === rect.r2 + 1 && j === rect.c1) return "C";
  if (i === rect.r1 && j === rect.c1) return "D";
  return null;
};

return (
  <div className="space-y-5">
    <div className="grid gap-5 lg:grid-cols-2">
      {/* Исходная матрица */}
      <div className="overflow-x-auto">
        <div className="text-xs text-slate-400 mb-2">
          <div className="inline-block">
            {A2.map((row, i) => {
              <div key={i} className="flex gap-1.5 mb-1">
                {row.map((v, j) => {
                  const inRect = i >= rect.r1 && i <= rect.r2 &&
                    j >= rect.c1 && j <= rect.c2;
                  return (
                    <div
                      key={j}
                      onClick={() => setRect((r) => {
                        i < r.r1 || j < r.c1 ? { r1: i, r2: i, c1: j, c2: j } : r
                      })}
                      className={` ${cellBase} cursor-pointer ${
                        inRect ? "bg-indigo-500 text-white" : ""
                      }`}
                    />
                  )
                })}
              </div>
            {v}
            </div>
          )};
        </div>
      </div>
    </div>
  )
);

```

```

        }}}
      </div>
    }}}
  </div>
</div>
</div>

<div className="min-h-[54px] bg-slate-900 border border-indigo-500"
  {hover ? {
    <p className="text-slate-300">
      <b className="text-amber-400">
         $P[\{hover.i\}][\{hover.j\}] = \{P[hover.i][hover.j] +$ 
      </b>{" "}
      – сумма всего прямоугольника от левого верхнего угла до
    </p>
  } : {
    <p className="text-slate-500">Наведите курсор на элемент
  }}
</div>

<div className="bg-slate-900 border border-indigo-500"
  <p className="font-mono text-sm sm:text-base text-slate-300">
    сумма = <span className="text-emerald-400">A</span> + <span
    <span className="text-rose-400">C</span> + <span
    <span className="text-indigo-300 font-bold">{
  </p>
  <p className="text-[11px] text-slate-500 mt-1">
    Вычли два «хвоста» и вернули дважды вычитенный
  </p>
</div>
</div>
);
};

```



```
const Viz1D: React.FC = () => {
  const [step, setStep] = useState(0);
  const [hover, setHover] = useState<{ i: number; j: number }>({ i: 0, j: 0 });

  // Total steps: we animate computing each element for each j
  const computeSteps: { i: number; j: number }[] = [];
  for (let j = 1; j < LOG; j++) {
    for (let i = 0; i + (1 << j) <= N; i++) {
      computeSteps.push({ i, j });
    }
  }

  const maxStep = computeSteps.length;
  const covered = hover ? { from: hover.i, to: hover.i + (1 << hover.j) } : null;

  return (
    <div className="flex flex-col gap-5">
      <div className="flex items-center justify-between">
        <div>
          <h3 className="text-lg font-bold text-indigo-600">
            Построение Sparse Table (Min)
          </h3>
          <p className="text-sm text-slate-400">
            Формула:  $ST[i][j] = \min(ST[i][j-1], ST[i + (1 \ll (j-1))][j-1])$ 
          </p>
        </div>
        <div className="flex gap-2">
          <button
            onClick={() => setStep(Math.max(0, step - 1))}
            className="p-2 bg-slate-800 hover:bg-slate-700 disabled:opacity-50"
            disabled={step === 0}
          >
            <ChevronLeft size={20} />
          </button>
          <button
            onClick={() => setStep(Math.min(maxStep, step + 1))}
            className="p-2 bg-indigo-600 hover:bg-indigo-700 disabled:opacity-50"
            disabled={step === maxStep}
          >
            <ChevronRight size={20} />
          </button>
        </div>
      </div>
      <div>
        <table>
          <caption>Sparse Table (Min)</caption>
          <thead>
            <tr>
              <th>i \ j</th>
              <th>1</th>
              <th>2</th>
              <th>3</th>
              <th>4</th>
            </tr>
          </thead>
          <tbody>
            <tr>
              <th>0</th>
              <td>0</td>
              <td>0</td>
              <td>0</td>
              <td>0</td>
            </tr>
            <tr>
              <th>1</th>
              <td>0</td>
              <td>1</td>
              <td>1</td>
              <td>1</td>
            </tr>
            <tr>
              <th>2</th>
              <td>0</td>
              <td>1</td>
              <td>2</td>
              <td>2</td>
            </tr>
            <tr>
              <th>3</th>
              <td>0</td>
              <td>1</td>
              <td>2</td>
              <td>3</td>
            </tr>
          </tbody>
        </table>
      </div>
    </div>
  );
}
```

```

        </b>{ " " }
        – это минимум на отрезке{ " " }
        <span className="font-mono text-sky-300">
            [{covered.from}, {covered.to}]
        </span>{ " " }
        длины  $2^{\{hover.j\}} = \{1 \ll hover.j\} : \{ " " \}$ 
        <span className="font-mono text-slate-400">
            min({arr1D.slice(covered.from, covered.to)})
        </span>
    </p>
    ) : (
        <p className="text-slate-500">
            Наведите курсор (или тапните) на любое число
        </p>
    )}
</div>
</div>

<div className="overflow-x-auto pb-2">
    <div className="inline-block min-w-full bg-slate-50">
        <div className="grid grid-cols-[auto_repeat(4,1fr)]">
            { /* Headers */ }
            <div className="text-slate-500 font-mono text-sm">
                i \ j
            </div>
            <div className="text-center font-bold text-slate-500">
                 $2^0$  (L=1)
            </div>
            <div className="text-center font-bold text-slate-500">
                 $2^1$  (L=2)
            </div>
            <div className="text-center font-bold text-slate-500">
                 $2^2$  (L=4)
            </div>
            <div className="text-center font-bold text-slate-500">
                 $2^3$  (L=8)
            </div>
        </div>
    </div>
</div>

```

```

        if (currentStep.i + (1 << (current
            isSrc2 = true;
        }
    }

    return (
      <div key={j} className="relative fl
        {!isValid ? (
          <div className="w-[clamp(30px,9
        ) : (
          <motion.div
            onMouseEnter={() => isVisible
            onMouseLeave={() => setHover(
            onClick={() => isVisible && s
            initial={{ opacity: 0, scale:
            animate={{
              opacity: isVisible ? 1 : 0,
              scale: isVisible ? 1 : 0.8,
            }}
            className={`w-[clamp(30px,9vw
clamp(11px,3vw,17px)] font-bold transition-
            hover && hover.i === i && h
              ? "bg-sky-500 text-white
              : isActive
              ? "bg-indigo-500 text-whi
              : isSrc1
              ? "bg-emerald-500 text-r
              : isSrc2
              ? "bg-amber-500 text-r
              : isVisible
              ? "bg-slate-800 tex
              : "bg-transparent t
            }`}
          >
            {st1D[i][j]}
          </motion.div>
        )}
    )}

```

```

        <p className="text-lg text-slate-300">
          <span className="text-white font-bold">
            ST[{c.i}][{c.j}]
          </span>{" "}
          = min(
            <span className="text-emerald-400 font-b"
              ST[{c.i}][{c.j - 1}]
            </span>
            ,
            <span className="text-amber-400 font-bo"
              ST[{c.i + half}][{c.j - 1}]
            </span>
          ) &rarr; min(
            <span className="text-emerald-400">{st1D
            <span className="text-amber-400">
              {st1D[c.i + half][c.j - 1]}
            </span>
          ) ={" "}
            <span className="text-indigo-400 font-b"
              {st1D[c.i][c.j]}
            </span>
          </p>
        );
      ))()
    ) : (
      <p className="text-slate-500 italic">
        Нажмите далее, чтобы увидеть, как блоки сво
      </p>
    )}
  </div>
</div>
);
};

```

```

const Viz2DBuild: React.FC = () => {
  const [k, setK] = useState(1);
  const [hover, setHover] = useState<{ r: number; c: num
  const [locked, setLocked] = useState<{ r: number; c: num

```

```

<div className={`flex flex-col items-c
  <h4 className={`font-bold mb-3 flex
}>
  <span>Матрица блоков {stepL}x{step
  <span className={`text-[10px] font
rder-indigo-800/40 text-indigo-300' : 'bg-s
</h4>
  <div className="max-w-full overflow-
    <div className={`grid gap-[2px] p-
go-900/40 shadow-[0_0_25px_rgba(99,102,241,
eColumns: `repeat(${stepSize}, max-content)
    {Array.from({length: stepSize * st
      const r = Math.floor(idx / step
      const c = idx % stepSize;

      let highlightClass = isCurr ? '
/50 cursor-pointer' : 'bg-slate-800/50 text
      let zIndex = 'z-0';

      const isActive = !!activeCell &
ll.c + L) && ((r - activeCell.r) % (1 << st

      if (isActive) {
        if (isCurr) {
          highlightClass = `bg-indigo
{locked && locked.r === r && locked.c === c
          zIndex = 'z-10';
        } else {
          const prevL = 1 << (k - 1);
          const isTop = r < activeCel
          const isLeft = c < activeCe

          if (isTop && isLeft) {
            highlightClass = 'bg-ros
md:scale-[1.08]';
          } else if (isTop && !isLeft
            highlightClass = 'bg-blur
] md:scale-[1.08]';

```



```

        <div className="flex item
            <span className="flex
shadow-[0_0_8px_#f59e0b]"></div> ST[{active
            <span className="text
[activeCell.r + prevL][activeCell.c + prevL
        </div>
    </>
    )
    }}{}}
</div>

    <div className="pt-3 mt-3 border-t
    ) = <span className="text-white
0">{ST2D[activeCell.r][activeCell.c][k][k]}
    </div>
    <p className="mt-4 text-[12px] font
список матриц (слева) вниз, чтобы увидеть,
    </div>
    ) : {
        <div className="bg-slate-950/40 border
mt-2 flex items-center justify-center anim
        Наведите курсор на матрицу {L}x{L
        </div>
    }}
</div>
    })
</div>
</div>
</div>
);
};

const Viz2DQuery: React.FC = () => {
    const [r1, setR1] = useState(1);
    const [c1, setC1] = useState(1);
    const [r2, setR2] = useState(5);
    const [c2, setC2] = useState(6);

```

```

top: `${(r1 / 8) * 100}%`,
left: `${(c1 / 8) * 100}%`,
marginTop: '8px',
marginLeft: '8px',
height: 'calc(' + (H/8*100) +
width: 'calc(' + (W/8*100) +
  }}
/>

{/* Показываем 4 угла в самой матрице */}
<div className="absolute pointer-events-none shadow-[0_0_10px_#f43f5e]" style={{ top: `calc(
  `${(Ly / 8) * 100}% - 16px}`, height: `calc(
    <div className="absolute pointer-events-none shadow-[0_0_10px_#3b82f6]" style={{ top: `
    width: `calc(
      `${(Ly / 8) * 100}% - 16px}`,
    <div className="absolute pointer-events-none shadow-[0_0_10px_#10b981]" style={{ top: `
    8px}`, width: `calc(
      `${(Ly / 8) * 100}% - 16px}`,
    <div className="absolute pointer-events-none shadow-[0_0_10px_#f59e0b]" style={{ top: `
    00}% + 8px}`, width: `calc(
      `${(Ly / 8) * 100}% + 8px}`,
    </div>

    <div className="bg-slate-950 p-4 rounded-lg shadow-[inset_0_2px_10px_rgba(0,0,0,0.5)]" style={{
    <label className="flex items-center justify-between" style={{ width: 100%; }}>
      <span className="text-slate-400">Регулировка размера</span>
      <div className="flex gap-2">
        <input type="number" min={1} max={10} value={1} />
        14 bg-slate-900 border border-slate-700 py-2 px-2
        <input type="number" min={1} max={10} value={1} />
        14 bg-slate-900 border border-slate-700 py-2 px-2
      </div>
    </label>
    <label className="flex items-center justify-between" style={{ width: 100%; }}>
      <span className="text-slate-400">Регулировка цвета</span>
      <div className="flex gap-2">

```

```

        &nbsp;&nbsp;&nbsp;<span className="text-slate-400">
В запроса)</span><br/>
        &nbsp;&nbsp;&nbsp;<span className="text-rose-900">
r1</span>][<span className="text-white bg-rose-900">

        &nbsp;&nbsp;&nbsp;<span className="text-slate-400">
во, чтобы правый край уперся в c2)</span><br/>
        &nbsp;&nbsp;&nbsp;<span className="text-blue-600">
r1</span>][<span className="text-white bg-blue-600">

        &nbsp;&nbsp;&nbsp;<span className="text-slate-400">
px, чтобы нижний край уперся в r2)</span><br/>
        &nbsp;&nbsp;&nbsp;<span className="text-emerald-500">
nded">r2 - Lx + 1</span>][<span className="text-white bg-emerald-500">

        &nbsp;&nbsp;&nbsp;<span className="text-slate-400">
и влево от c2)</span><br/>
        &nbsp;&nbsp;&nbsp;<span className="text-amber-500">
">r2 - Lx + 1</span>][<span className="text-white bg-amber-500">
        {'}}''));
</div>

<div className="w-full flex justify-center">
  <div className="flex flex-col items-center">
    <h4 className="text-slate-400 font-bold">
      Откуда берутся эти 4 числа: матрица
border-slate-700 shadow-sm">ST[][][{{kx}}][{{ky}}]
    </h4>
    <div className="grid gap-[2px] p-2.5">
Columns: `repeat(${matCols}, 1fr)`}}>
      {Array.from({length: matRows * matCols}, (v, idx) => {
        const r = Math.floor(idx / matCols);
        const c = idx % matCols;
        let isTL = r === r1 && c === c1;
        let isTR = r === r1 && c === c2;
        let isBL = r === r2 - Lx + 1 && c === c1;
        let isBR = r === r2 - Lx + 1 && c === c2;
      })}
    </div>
  </div>
</div>

```

```

            <span className="ml-3 text-emerald-500">
+ 1][c1][kx][ky], ST2D[r2 - Lx + 1][c2 - Ly]
            </div>
        </div>
    </div>
</div>
)
}
```

=====

РАЗДЕЛ: HTML-РАЗМЕТКА (LAYOUT)

=====

ФАЙЛ 72/82: src/layout/footer.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРОВОК: 137 | РАЗМЕР: 1.5 КБ

```

        </div>
    </main>
</div>

<script>
    function setMode(mode) {
        document.getElementById('btn-mode-normal').className =
            mode === 'normal' ? 'px-3 py-1 rounded text-sm font-bold text-emerald-500'
            : 'px-3 py-1 rounded text-sm font-bold text-gray-500';

        document.getElementById('btn-mode-wtf').className =
            mode === 'wtf' ? 'px-3 py-1 rounded text-sm font-bold text-emerald-500'
            : 'px-3 py-1 rounded text-sm font-bold text-gray-500';

        const aside = document.querySelector('aside');
        const headerMain = document.querySelector('h1');
        const questionsContainer = document.getElementById('questions-container');
    }
</script>
```

```

        n-colors';
        document.getElementById('btn-theme-light').
        ext-white transition-colors' : 'px-3 py-1 r
        ion-colors';
        document.getElementById('btn-theme-notebook
        -600 text-white transition-colors' : 'px-3 p
        ransition-colors';
    }

    // Initialize theme and mode
    setTheme('dark');

    // Setup intersection observer for TOC highlight
    document.addEventListener('DOMContentLoaded', {
        const mobileMenuBtn = document.getElementById('mobile-menu');
        const mobileDrawer = document.getElementById('mobile-drawer');
        const closeDrawerBtn = document.getElementById('close-drawer');
        const drawerOverlay = document.getElementById('drawer-overlay');

        function toggleDrawer() {
            const isClosed = mobileDrawer.classList.contains('closed');
            if (isClosed) {
                mobileDrawer.classList.remove("-transl");
                if (drawerOverlay) {
                    drawerOverlay.classList.remove("op");
                    // Timeout allows display:block
                    setTimeout(() => drawerOverlay.
                }
                document.body.style.overflow = "hidden";
            } else {
                mobileDrawer.classList.add("-transl");
                if (drawerOverlay) {
                    drawerOverlay.classList.add("op");
                    setTimeout(() => drawerOverlay.
                }
                document.body.style.overflow = "";
            }
        }
    });

```

```
        }
      });
    }, observerOptions);

    document.querySelectorAll('section[id^="que
      observer.observe(section);
    });
  });
</script>
</body>
</html>
```

ФАЙЛ 73/82: src/layout/header.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРОК: 1169 | РАЗМЕР

```
<!DOCTYPE html>
<html lang="ru" class="scroll-smooth">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
  <title>Пособие по алгебре: От пиццы к предикатам</t
  <script src="https://cdn.tailwindcss.com"></script>
  <script src="https://polyfill.io/v3/polyfill.min.js
  <script id="MathJax-script" async src="https://cdn.
  <script>
    window.MathJax = {
      tex: {
        inlineMath: [['$', '$'], ['\\(', '\\)']]
        displayMath: [['$$', '$$'], ['\\[', '\\
      }
    };
  </script>
  <style>
    @import url('https://fonts.googleapis.com/css2?
    @reference "../src/index.css";
```

```

        -ms-overflow-style: none;
        -webkit-overflow-scrolling: touch;
    }
    mjx-container[display="true"]::-webkit-scrollbar {
        display: none;
    }

    @media (max-width: 640px) {
        html { font-size: 13px; }

        .uno-card { width: 40px; height: 60px; font-size: 12px; }
        .uno-card::before, .uno-card::after { font-size: 12px; }
        .uno-card::before { top: 1px; left: 2px; }
        .uno-card::after { bottom: 1px; right: 2px; }

        .uno-card.mini { width: 28px !important; height: 40px !important; }
        .uno-card.mini::before, .uno-card.mini::after { font-size: 10px; }

        .uno-equation .text-2xl { font-size: 1.2rem; }
        .uno-equation.gap-4 { gap: 0.5rem !important; }

        /* Allow horizontal scroll for equations in containers */
        .formula-scroll { flex-wrap: nowrap !important; }
    }

    .uno-inner {
        position: absolute; width: 110%; height: 75%;
        border-radius: 50%; transform: rotate(-30deg);
        box-shadow: inset 0 0 5px rgba(0,0,0,0.3);
    }

    .uno-val { position: relative; z-index: 2; }
    .card-red { background-color: #e52521; } .card-blue { background-color: #004dcf; } .card-green { background-color: #339e35; } .card-yellow { background-color: #ffc400; } .card-black { background-color: #222; }
    .card-black::before, .card-black::after { text-align: center; }

```



```

        box-shadow: 2px 2px 5px rgba(0,0,0,0.05);
    }
    body.theme-notebook .text-slate-200, body.theme
        ant; }
    body.theme-notebook .text-slate-400 { color: #5
    body.theme-notebook .border-slate-600, body.the
        : #94a3b8 !important; }
    body.theme-notebook .font-mono { font-family: "
    body.theme-notebook #top-controls { background-

/* CREEPY BLINKING */
@keyframes creepy-blink {
    0%, 95%, 98%, 100% { transform: scaleY(1);
    96.5% { transform: scaleY(0.1); }
}
.creepy-eye {
    transform-origin: center;
    transform-box: fill-box;
    animation: creepy-blink 4s infinite;
}
.creepy-eye-alt {
    transform-origin: center;
    transform-box: fill-box;
    animation: creepy-blink 5.2s infinite;
    animation-delay: 1.5s;
}
.creepy-emoji {
    display: inline-block;
    transform-origin: center;
    animation: creepy-blink 3.8s infinite;
}
</style>
</head>
<body class="bg-slate-900 text-slate-200 font-sans lead
    <div class="sticky top-0 z-50 w-full bg-slate-950/9
        center items-center gap-4 sm:gap-8 transition-col

```

```

        <path d="M0,0 L6,3 L0,6 Z" fill="#818cf"
    </marker>
    <marker id="arrow-lime" markerWidth="6" mar
        <path d="M0,0 L6,3 L0,6 Z" fill="#a3e63"
    </marker>
    <marker id="arrow-green" markerWidth="6" ma
        <path d="M0,0 L6,3 L0,6 Z" fill="#4ade8"
    </marker>

    <!-- ЛЕГО БЛОКИ -->
    <g id="lego-yellow">
        <rect x="0" y="0" width="40" height="20"
        <rect x="6" y="-5" width="8" height="6"
        <rect x="26" y="-5" width="8" height="6"
        <line x1="2" y1="2" x2="38" y2="2" stro
    </g>
    <g id="lego-red">
        <rect x="0" y="0" width="40" height="20"
        <rect x="6" y="-5" width="8" height="6"
        <rect x="26" y="-5" width="8" height="6"
        <line x1="2" y1="2" x2="38" y2="2" stro
    </g>
    <g id="lego-rem">
        <rect x="0" y="0" width="40" height="10"
        <rect x="6" y="-5" width="8" height="6"
        <rect x="26" y="-5" width="8" height="6"
        <line x1="2" y1="2" x2="38" y2="2" stro
    </g>
    <!-- МЯСОРУБКА -->
    <g id="meat-grinder">
        <path d="M 10 40 L 15 20 Q 25 15 35 20
        <rect x="5" y="40" width="40" height="20"
        <!-- Ручка -->
        <path d="M 45 50 Q 60 50 60 30 L 75 30"
        <circle cx="75" cy="30" r="4" fill="#33
        <!-- Выход -->
        <path d="M 5 45 L -5 45 L -5 55 L 5 55
        <circle cx="25" cy="50" r="15" fill="#f

```

```
    </h3>
    <button id="close-drawer-btn" class="
      <svg xmlns="http://www.w3.org/20
        <path stroke-linecap="round
      </svg>
    </button>
  </div>
  <nav class="space-y-3 text-sm font-medium">

    <details class="group" open>
      <summary class="list-none cursor-pointer text-blue-500 pl-3 font-bold text-blue-300">
        <a href="#question-1" class="text-blue-500 pl-3 font-bold text-blue-300">
          1. Алгебраические структуры
        </a>
      </summary>
      <div class="pl-6 space-y-2 text-sm">
        <a href="#section-1-1" class="text-blue-500 pl-3 font-medium text-blue-300">
        <a href="#section-1-2" class="text-blue-500 pl-3 font-medium text-blue-300">
        <a href="#section-1-3" class="text-blue-500 pl-3 font-medium text-blue-300">
        <a href="#section-1-4" class="text-blue-500 pl-3 font-medium text-blue-300">
      </div>
    </details>

    <details class="group">
      <summary class="list-none cursor-pointer text-indigo-500 pl-3 font-bold text-indigo-300">
        <a href="#question-2" class="text-indigo-500 pl-3 font-bold text-indigo-300">
          2. Комплексные числа
        </a>
      </summary>
      <div class="pl-6 space-y-2 text-sm">
        <a href="#section-2-1" class="text-indigo-500 pl-3 font-medium text-indigo-300">
        <a href="#section-2-2" class="text-indigo-500 pl-3 font-medium text-indigo-300">
        <a href="#section-2-3" class="text-indigo-500 pl-3 font-medium text-indigo-300">
      </div>
    </details>
```

```
<a href="#q5-def" class="block ho
<a href="#q5-group" class="bloc
<a href="#q5-prim" class="block
<a href="#q5-crit" class="block
<a href="#q5-cyclic" class="blo
</div>
</details>

<details class="group">
  <summary class="list-none curso
    <a href="#question-6" class=
er-lime-500 pl-3 font-bold text-lime-300">
      6. Делимость, НОД и Евклид
    </a>
  </summary>
  <div class="pl-6 space-y-2 text
    <a href="#q6-1" class="block ho
    <a href="#q6-2" class="block ho
    <a href="#q6-3" class="block ho
    <a href="#q6-4" class="block ho
    <a href="#q6-5" class="block ho
    <a href="#q6-6" class="block ho
  </div>
</details>

<details class="group">
  <summary class="list-none curso
    <a href="#question-7" class=
er-pink-500 pl-3 font-bold text-pink-300">
      7. Взаимно простые числ
    </a>
  </summary>
  <div class="pl-6 space-y-2 text
    <a href="#q7-1" class="bloc
    <a href="#q7-2" class="bloc
    <a href="#q7-3" class="bloc
    <a href="#q7-4" class="bloc
```

```
<summary class="list-none cursor-pointer-rose-500 pl-3 font-bold text-rose-400">
    10. Делимость и Схема Г
</a>
</summary>
<div class="pl-6 space-y-2 text-rose-500">
    <a href="#q10-1" class="block">
    <a href="#q10-2" class="block">
    <a href="#q10-3" class="block">
a>
        <a href="#q10-4" class="block">
        <a href="#q10-5" class="block">
    </div>
</details>

<details class="group">
    <summary class="list-none cursor-pointer-cyan-500 pl-3 font-bold text-cyan-400">
        11. НОД и НОК многочленов
    </a>
    </summary>
    <div class="pl-6 space-y-2 text-cyan-500">
        <a href="#q11-1" class="block">
        <a href="#q11-2" class="block">
        <a href="#q11-3" class="block">
        <a href="#q11-4" class="block">
    </div>
</details>

<details class="group">
    <summary class="list-none cursor-pointer-blue-500 pl-3 font-bold text-blue-400">
```

```
            border border-rose-500/30"> К
        </div>
    </details>
```

```
    <details class="group">
        <summary class="list-none cursor-pointer text-teal-500 font-medium">
            <a href="#question-15" class="text-teal-500 font-medium">
                15. Производная. Критер
            </a>
        </summary>
        <div class="pl-6 mt-2 space-y-2">
            <a href="#q15-1" class="block text-teal-500 pl-3 font-medium">
            <a href="#q15-2" class="block text-teal-500 pl-3 font-medium">
            <a href="#q15-3" class="block text-teal-500 pl-3 font-medium">
            <a href="#q15-4" class="block text-teal-500 pl-3 font-medium">
        </div>
    </details>
```

```
    <details class="group">
        <summary class="list-none cursor-pointer text-teal-500 font-medium">
            <a href="#question-16" class="text-teal-500 font-medium">
                16. Неприводимые многоч
            </a>
        </summary>
        <div class="pl-6 mt-2 space-y-2">
            <a href="#q16-1" class="block text-teal-500 pl-3 font-medium">
            <a href="#q16-2" class="block text-teal-500 pl-3 font-medium">
            <a href="#q16-3" class="block text-teal-500 pl-3 font-medium">
        </div>
    </details>
```

```
    <div class="mt-4 mb-2 text-xs font-medium">
        <a href="#question-17" class="block text-teal-500 pl-3 font-medium">
            <span class="font-medium text-teal-500">
            </a>
```

```

    });

    // Scroll spy logic
    document.addEventListener('DOMContentLoaded', () => {
        const sections = document.querySelectorAll(
            'h2, h3, h4, h5, h6'
        );
        const navLinks = document.querySelectorAll(
            '#main-nav a'
        );

        let isScrolling = false;

        document.querySelector('.group-1')
            .addEventListener('click', () => {
                isScrolling = true;
                setTimeout(() => isScrolling = false, 200);
            });
    });

    const observer = new IntersectionObserver((entries) => {
        if (isScrolling) return;

        let visibleId = null;
        entries.forEach(entry => {
            if (entry.isIntersecting) {
                visibleId = entry.target.id;
            }
        });

        // Highlight active link
        document.querySelectorAll('#main-nav a')
            .forEach(a => {
                a.style.fontWeight = 'normal';
                if (a.href.endsWith(visibleId)) {
                    a.style.fontWeight = 'bold';
                }
            });

        // Open the parent details
        const correspondingDetails = document.querySelector(
            `#details-${visibleId}`
        );
        if (correspondingDetails) {
            const details = new Details({
                open: true
            });
            details.open(correspondingDetails);
        }
    });

```

```
box.style.padding = '20px';
box.style.borderRadius = '10px';
box.style.display = 'flex';
box.style.flexDirection = 'column';
box.style.color = '#fff';

const header = document.createElement('h2');
header.innerText = 'Экспорт в ' + fileName;
header.style.marginBottom = '10px';

const subHeader = document.createElement('h3');
subHeader.innerText = 'Из-за особенностей браузеров, для сохранения файла необходимо открыть этот документ в новой веб-студии в новой вкладке (через меню)';
subHeader.style.marginBottom = '10px';
subHeader.style.color = '#f87171';
subHeader.style.fontSize = '0.9em';

const textarea = document.createElement('div');
textarea.value = contentRaw;
textarea.style.flex = '1';
textarea.style.width = '100%';
textarea.style.color = '#000';
textarea.style.padding = '10px';
textarea.style.fontFamily = 'monospace';

const btnContainer = document.createElement('div');
btnContainer.style.display = 'flex';
btnContainer.style.gap = '10px';
btnContainer.style.marginTop = '10px';

const copyBtn = document.createElement('button');
copyBtn.innerText = 'Копировать';
copyBtn.style.padding = '10px';
copyBtn.style.backgroundColor = '#fff';
copyBtn.style.color = '#fff';
copyBtn.style.border = 'none';
copyBtn.style.borderRadius = '5px';
copyBtn.style.cursor = 'pointer';
```



```

// Smart markdown parsing based
clone.querySelectorAll('h2').fo
clone.querySelectorAll('h3').fo
clone.querySelectorAll('.analog
clone.querySelectorAll('li').fo
clone.querySelectorAll('svg').fo
    let caption = '';
    if (el.nextElementSibling &
        caption = el.nextElement
    }
    el.textContent = '\n\n[ Илл
});
clone.querySelectorAll('.img-ca

let text = clone.innerText;
text = text.replace(/\n{3,}/g,

if (window !== window.top) {
    fallbackModal(text, 'Markdow
    return;
}
const blob = new Blob([text], {
const url = URL.createObjectURL
const link = document.createEle
link.href = url;
link.download = 'vyisshaya_alge
document.body.appendChild(link)
link.click();
document.body.removeChild(link)
}

function setTheme(theme) {

    const body = document.body;
    body.classList.remove('bg-slate

);

```

```

    or: #94a3b8 !important; }
        body.bg-white .img-capt

        /* Tweak card blocks */
        body.bg-white .card-red

        body.bg-white aside { b
#334155 !important; }
        body.bg-white .text-whi

        /* Ensure active links
        body.bg-white aside a.t
        body.bg-white aside a.t
        body.bg-white aside a.t
        body.bg-white aside a.t
        body.bg-white aside a.t
        body.bg-white aside a.t
        body.bg-white aside a:h

        body.bg-white table tr.
        body.bg-white table tr.
        body.bg-white .text-tea
        body.bg-white .text-ros
        body.bg-white .text-eme
        body.bg-white .text-cya
        body.bg-white .text-amb
        body.bg-white .text-fuc
        body.bg-white .text-lim
        body.bg-white .text-blur
        body.bg-white .text-gre
        body.bg-white .text-ind

        `;
    } else if (theme === 'terminal'
        body.classList.add('bg-black
        document.getElementById('th
        style.innerHTML = `
            body.bg-black .bg-slate

```

```

        </button>
        <button onclick="downloadHTML()"
t-sm font-bold flex items-center justify-center
        <span> </span> HTML
        </button>
    </div>

    <!-- Смена темы -->
    <div>
        <div class="text-xs text-slate-500">
            <div class="flex items-center gap-2">
                <button id="theme-dark" onclick="toggleTheme('dark')"
justify-center text-amber-300 hover:text-white
line-amber-500" title="Темная тема"> </button>
                <button id="theme-light" onclick="toggleTheme('light')"
r justify-center text-amber-50 hover:text-white
">* </button>
                <button id="theme-terminal" onclick="toggleTheme('terminal')"
-center justify-center text-green-400 hover:text-white
терминала"> </button>
            </div>
        </div>
    </div>
</div>

</aside>

<!-- ОСНОВНОЙ КОНТЕНТ -->
<main class="flex-1 bg-slate-800 p-4 sm:p-8 md:p-12 lg:p-16"
-base min-w-0">
    <header class="text-center mb-12 border-b border-slate-700">
        <h1 class="text-4xl md:text-5xl font-bold">
            ВЫСШАЯ АЛГЕБРА
        </h1>
        <p class="text-slate-400 italic text-lg">
            <!-- Print Table of Contents -->

```

```

        <div class="bg-orange-900/40 border-4 border-yellow-500">
ольцо полиномов</div>
        <div class="bg-cyan-900/40 border-4 border-yellow-500">
сые числа</div>

        <!-- Row 1: База -->
        <a href="#question-1" onclick="setMainQuestion(1)">
-4 border-yellow-500 hover:bg-slate-700 transition-colors duration-200
        <div class="text-xs text-yellow-400">База
        <div class="font-bold text-slate-200">База
        </a>
        <a href="#question-9" onclick="setMainQuestion(9)">
-4 border-orange-500 hover:bg-slate-700 transition-colors duration-200
        <div class="text-xs text-orange-400">База
        <div class="font-bold text-slate-200">База
        </a>
        <a href="#question-2" onclick="setMainQuestion(2)">
-4 border-cyan-500 hover:bg-slate-700 transition-colors duration-200
        <div class="text-xs text-cyan-500">База
        <div class="font-bold text-slate-200">База
        </a>

        <!-- Row 2: Базовые операции / Евклид -->
        <a href="#question-6" onclick="setMainQuestion(6)">
-4 border-yellow-500 hover:bg-slate-700 transition-colors duration-200
        <div class="text-xs text-yellow-400">Базовые операции
        <div class="font-bold text-slate-200">Базовые операции
        </a>
        <a href="#question-10" onclick="setMainQuestion(10)">
-1-4 border-orange-500 hover:bg-slate-700 transition-colors duration-200
        <div class="text-xs text-orange-400">Базовые операции
        <div class="font-bold text-slate-200">Базовые операции
        </a>
        <a href="#question-3" onclick="setMainQuestion(3)">
-4 border-cyan-500 hover:bg-slate-700 transition-colors duration-200
        <div class="text-xs text-cyan-500">Базовые операции
        <div class="font-bold text-slate-200">Базовые операции
        </a>

```

```

        </a>
        <a href="#question-4" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
        <div class="text-xs text-cyan-500"
        <div class="font-bold text-slate-700"
        </a>

        <!-- Row 6: Кратность -->
        <div class="hidden md:block"></div>
        <a href="#question-14" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
        <div class="text-xs text-orange-500"
        <div class="font-bold text-slate-700"
        </a>
        <div class="hidden md:block"></div>

        <!-- Row 7: Производная -->
        <div class="hidden md:block"></div>
        <a href="#question-15" onclick="setM
-1-4 border-orange-500 hover:bg-slate-700 t
        <div class="text-xs text-orange-500"
        <div class="font-bold text-slate-700"
        </a>
        <div class="hidden md:block"></div>

        <!-- КЛАСТЕР 3: НЕПРИВОДИМОСТЬ -->
        <div class="col-span-1 md:col-span-1"
b-2 border-b border-emerald-500/30 pb-2">Кл

        <a href="#question-17" onclick="setM
-1-4 border-teal-500 hover:bg-slate-700 tra
        <div class="text-xs text-teal-500"
        <div class="font-bold text-slate-700"
        </a>
        <a href="#question-18" onclick="setM
-1-4 border-blue-500 hover:bg-slate-700 tra
        <div class="text-xs text-blue-500"
        <div class="font-bold text-slate-700"

```

```
        <div class="bg-red-500/20 p-3 rounded-2xl">
          <svg class="w-8 h-8 text-red-500"
            rg/2000/svg">
            <path stroke-linecap="round"
              110 4m-6 8a2 2 0 100-4m0 4a2 2 0 110-4m0 4
            </svg>
          </div>
          <h2 class="text-3xl pr-4 border"
        </div>
```

```
    <div class="prose prose-invert max-w-4xl">
      <p class="text-lg">
        Хватит тупить. Доказательств
        есь для того, чтобы жать на кнопки. Вот жес
      </p>
```

```
    <h2 class="text-2xl font-bold border"
      (Меметика)</h2>
```

```
    <!-- Прямое -->
    <div class="bg-slate-800/80 p-6">
      <h3 class="text-xl text-blue-400">
      <p class="text-sm text-slate-400">
      <ul class="list-disc pl-5">
        <li><span class="text-white">
          k$).</li>
        <li><span class="text-white">
        <li><span class="text-white">
          ово.</li>
      </ul>
    </div>
```

```
    <!-- От противного -->
    <div class="bg-slate-800/80 p-6">
      <h3 class="text-xl text-fuchsia-400">
      <p class="text-sm text-slate-400">
      <ul class="list-disc pl-5">
        <li><span class="text-white">
```

```
<p class="text-sm text-slate-500">
  <ul class="list-disc pl-5 space-y-2">
    <li><span class="text-w"
вверх по уравнениям).</li>
    <li><span class="text-w"
рху вниз, показывая, что  $\delta \vdots r_i$ 
    <li><span class="text-w"
х. Твой  $d$  больше.</li>
  </ul>
</div>
```

```
<h2 class="text-2xl font-bold text-slate-500">
  алгоритмы экзекуции)</h2>
```

```
<div class="grid grid-cols-1 md:grid-cols-2">
  <div class="bg-slate-800 p-4">
    <h4 class="text-red-400">
    <p class="text-sm text-slate-500">
    <p class="text-xs font-slate-500">
со следующим. Если в конце 0 – пациент мертв
  </div>
```

```
<div class="bg-slate-800 p-4">
  <h4 class="text-red-400">
  <p class="text-sm text-slate-500">
  <p class="text-xs font-slate-500">
 $c$ . Пока получается 0 – корень жив. Как то
корня минус один.</p>
</div>
```

```
<div class="bg-slate-800 p-4">
  <h4 class="text-red-400">
  <p class="text-sm text-slate-500">
  <p class="text-xs font-slate-500">
ициентов (они делятся на  $p$ ), кроме старше
зложить.</p>
</div>
```

```
        <div>
          <h4 class="text-cyan">
            <p class="text-sm text-cyan">
              есть сложение — есть ноль. Если есть умножение, то
              <code class="text-x">
            </div>
          </li>

          <li class="bg-slate-800/50 p-4">
            <span class="text-2xl">
              <div>
                <h4 class="text-cyan">
                  <p class="text-sm text-cyan">
                    авь их драться. Умножение всегда врывается и
                    <code class="text-x">
                  </div>
                </li>
              </ul>
            </div>
          </section>
        </div>

        <div id="questions-container">
          <!-- ===== ВОПРОС 1 =====>
```

РАЗДЕЛ: УТИЛИТЫ

ФАЙЛ 74/82: src/utils/cn.ts

КАТЕГОРИЯ: Утилиты | СТРОК: 7 | РАЗМЕР: 169 В

```
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";
```



```

        if (res.ok) parts.push(await res.text());
      } catch {
        /* чужой домен – пропускаем */
      }
    }
  }
  return parts.join("\n");
}

const EXTRA_CSS = `
/* – правки для автономного файла – */
html { color-scheme: dark; }
body { background: #020617; color: #e2e8f0; font-family
a.term-hint { text-decoration: none; }
.export-shell { max-width: 62rem; margin: 0 auto; padding
.export-head { border-bottom: 1px solid #1e293b; padding
.export-note { background: #0f172a; border: 1px solid #
  border-radius: .5rem; padding: .85rem 1rem; font-size
.export-note a { color: #a5b4fc; }
.export-gloss { margin-top: 3rem; border-top: 1px solid
.export-gloss h2 { color: #fff; font-size: 1.25rem; fon
.export-term { background: #0f172a; border: 1px solid #
.export-term h3 { color: #c7d2fe; font-size: .95rem; fon
.export-term p { margin: 0 0 .3rem; font-size: .85rem;
.export-term .formal { color: #94a3b8; font-size: .8rem
.export-term .cx { color: #6ee7b7; font-size: .78rem; f
.export-foot { margin-top: 3rem; border-top: 1px solid
@media print {
  body { background: #fff; color: #0f172a; }
  .export-note, details { break-inside: avoid; }
  details { display: block; }
  details > div, details > p { display: block !important
  pre { white-space: pre-wrap; }
}
`;

const escapeHtml = (s: string) =>
  s.replace(/&/g, "&amp;").replace(/</g, "&lt;").replac

```



```
-----
export async function downloadChapterHtml(chapter: Chapter) {
  const css = await collectCss();
  const { body, termIds } = prepareBody(chapter.content);
  const html = wrap({
    title: chapter.title,
    subtitle: chapter.category || "Универсальное пособие",
    css,
    body,
    gloss: glossarySection(termIds),
    origin: window.location.origin,
  });
  triggerDownload(html, safeName(chapter.title, chapter.id));
}
```

/** Выгрузить всё пособие одним файлом со сквозным оглавлением */

```
export async function downloadBookHtml(chapters: Chapter[]) {
  const css = await collectCss();
  const allTerms: string[] = [];
  const bodies: string[] = [];

  const toc = chapters
    .map((c) => `<li style="margin:.2rem 0"><a href="#${c.id}">${c.title}</a></li>`)
    .join("\n");

  chapters.forEach((c) => {
    const { body, termIds } = prepareBody(c.content);
    termIds.forEach((t) => {
      if (!allTerms.includes(t)) allTerms.push(t);
    });
    bodies.push(`<div id="ch-${escapeHtml(c.id)}" style="display:none">${body}</div>`);
  });

  const body = `<nav id="toc" style="background:#0f172a;color:#fff;padding:1rem">
    <h2 style="color:#fff;font-size:1.05rem;font-weight:800;margin:0">Оглавление</h2>
    <ol style="margin:0;padding-left:1.1rem;font-size:.85rem">
      <li>1. Введение</li>
      <li>2. Основы</li>
      <li>3. Практика</li>
      <li>4. Заключение</li>
    </ol>
  </nav>`;

  const html = wrap({
    title: "Универсальное пособие",
    subtitle: "полное содержание",
    css,
    body,
    gloss: glossarySection(allTerms),
    origin: window.location.origin,
  });
  triggerDownload(html, "Универсальное пособие.pdf");
}
```

```

-----
fs.mkdirSync(TMP, { recursive: true });
const bundle = path.join(TMP, "content.bundle.mjs");

await build({
  entryPoints: [path.join(ROOT, "src/data/content.ts")]
  bundle: true,
  format: "esm",
  platform: "node",
  outfile: bundle,
  logLevel: "error",
});

const { chapters } = await import(`file://${bundle}`);

// Список интерактивов берём из реестра визуализаторов
const registry = fs.readFileSync(path.join(ROOT, "src/c
const registryBody = registry.slice(registry.indexOf("V
const vizIds = new Set([...registryBody.matchAll(/^\s{2

const rows = chapters.map((c) => {
  const html = c.content ?? "";
  const analogies = (html.match(/Аналогия/gi) ?? []).length;
  return {
    id: c.id,
    title: c.title,
    num: Number.parseInt(c.title.match(/^(\\d+)\\.\\/)?.[1]
    analogies,
    hasAnalogy: analogies > 0,
    inlineSvg: /<svg[\\s>]/i.test(html),
    image: /<img[\\s>]/i.test(html),
    interactive: vizIds.has(c.id),
    chars: html.length,
  };
});

const has2D = (r) => r.interactive || r.inlineSvg || r.
const gapBoth = rows.filter((r) => !r.hasAnalogy && !ha
const gapAnalogy = rows.filter((r) => !r.hasAnalogy &&

```

```
* Проверка покрытия глав всплывающими подсказками:
* в каждой ли теме находятся термины из глоссария.
*
*   node scripts/audit-terms.mjs
*/
import { build } from "esbuild";
import path from "node:path";

const ROOT = process.cwd();
await build({
  entryPoints: ["src/data/content.ts", "src/data/glossary.ts"],
  bundle: true, format: "esm", platform: "node",
  outdir: "tmp/audit-terms", outExtension: { ".js": ".mjs" },
  external: ["react", "react-dom", "motion", "lucide-react"],
});

const { chapters } = await import(path.join(ROOT, "tmp/audit-terms.mjs"));
const { GLOSSARY_LABELS, GLOSSARY } = await import(path.join(ROOT, "tmp/audit-terms.mjs"));

const esc = (s) => s.replace(/[\.\*\+\?\^\$\{\}\(\)|[\]\[\]]/g, "\\");
const re = new RegExp(
  `(?<![\\p{L}\\p{N}_-])(?=${GLOSSARY_LABELS.map((l) => esc(l.label))})`
  "giu"
);

const map = new Map(GLOSSARY_LABELS.map((l) => [l.label, l.term]));

// те же лимиты, что и в src/hooks/useTermHints.ts
const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;

const bad = [];
const used = new Set();
console.log("тем | подсв. | глава");
for (const c of chapters) {
  const text = c.content
    .replace(/<(script|style|code|pre)[\s\S]*?<\/\1>/gi, "")
    .replace(/<[^\>]+>/g, " ");
```

```
-----  
* Склеивает отрендеренные страницы tmp/export-pages/pages/ в  
* в единый документ public/export/full_code_all_pages.pdf  
*/
```

```
import fs from "node:fs";  
import path from "node:path";  
import PDFDocument from "pdfkit";  
import { ROOT, OUT_DIR, PAGES_DIR, PDF_PATH, ensureDir,
```

```
// A4 в пунктах PDF
```

```
const A4 = { width: 595.28, height: 841.89 };
```

```
async function main() {  
  if (!fs.existsSync(PAGES_DIR)) {  
    throw new Error(`Нет ${path.relative(ROOT, PAGES_DIR)}`);  
  }
```

```
  const pages = fs  
    .readdirSync(PAGES_DIR)  
    .filter((f) => f.endsWith(".png"))  
    .sort();
```

```
  if (pages.length === 0) throw new Error("Не найдено ни одной страницы");
```

```
  ensureDir(OUT_DIR);
```

```
  const doc = new PDFDocument({  
    size: [A4.width, A4.height],  
    margin: 0,  
    autoFirstPage: false,  
    info: {  
      Title: "Универсальное пособие – полный исходный код",  
      Author: "CI: rebuild-pdf",  
      Subject: "Экспорт всего кода проекта (код -> txt)",  
      CreationDate: new Date(),  
    },  
  });
```

```
  const stream = fs.createWriteStream(PDF_PATH);
```

```
import fs from "node:fs";
import path from "node:path";
import { execFileSync } from "node:child_process";
import { ROOT, OUT_DIR, TXT_PATH, HR, SUBHR, ensureDir,

/** Расширения, которые считаем «кодом» и включаем в да
const CODE_EXT = new Set([
  ".ts", ".tsx", ".js", ".jsx", ".mjs", ".cjs",
  ".css", ".scss", ".html", ".json", ".md", ".yaml", ".y
]);

/** Явные исключения: генерируемое, бинарное и просто ш
const EXCLUDE_FILES = new Set(["package-lock.json", "bu
const EXCLUDE_DIRS = ["node_modules", "dist", "build",

/** Человечитаемые категории для оглавления. */
const CATEGORIES = [
  [/^src\/data\/tickets\/$/, "Билеты (учебный контент)"],
  [/^src\/data\/$/, "Контент и данные пособия"],
  [/^src\/components\/visualizers\/$/, "Визуализаторы (в
  [/^src\/components\/$/, "Интерактивные компоненты и си
  [/^src\/layout\/$/, "HTML-разметка (layout)"],
  [/^src\/utils\/$/, "Утилиты"],
  [/^src\/$/, "Ядро приложения"],
  [/^scripts\/$/, "Скрипты сборки и экспорта"],
  [/^\.github\/$/, "CI / автоматизация"],
  [/^[^\/]+\$/ , "Конфигурация проекта"],
];

const categoryOf = (rel) => CATEGORIES.find(([re]) => re

/** Порядок разделов в документе. */
const ORDER = [
  "Конфигурация проекта",
  "Ядро приложения",
  "Контент и данные пособия",
  "Билеты (учебный контент)",
  "Интерактивные компоненты и симуляторы",
```

```

        const ib = ORDER.indexOf(categoryOf(b));
        return ia !== ib ? ia - ib : a.localeCompare(b);
    });
}

/** Достаём заголовки глав из данных пособия (без запус
function extractChapters(files) {
    const chapters = [];
    const seen = new Set();
    for (const rel of files.filter((f) => f.startsWith("s
        const src = fs.readFileSync(path.join(ROOT, rel), "
        const re = /id:\s*"([^"]+)"\s*,\s*\n?\s*title:\s*"
        let m;
        while ((m = re.exec(src)) !== null) {
            const [, id, title] = m;
            if (seen.has(id)) continue;
            seen.add(id);
            chapters.push({ id, title, file: rel });
        }
    }
    return chapters.sort((a, b) => {
        const na = parseInt(a.title.match(/(\d+)/)?.[0] ??
        const nb = parseInt(b.title.match(/(\d+)/)?.[0] ??
        return na - nb;
    }));
}

function main() {
    const files = listFiles();
    const chapters = extractChapters(files);
    ensureDir(OUT_DIR);

    const out = [];
    const push = (s = "") => out.push(s);

    const stamp = new Date().toLocaleString("ru-RU", { ti
    const totalBytes = files.reduce((s, f) => s + fs.stat

```



```

    if (cat !== current) {
      current = cat;
      push(HR);
      push(`РАЗДЕЛ: ${cat.toUpperCase()}`);
      push(HR);
      push();
    }
    const content = fs.readFileSync(path.join(ROOT, rel));
    const lines = content.split("\n");
    push(SUBHR);
    push(`ФАЙЛ ${i + 1}/${files.length}: ${rel}`);
    push(`КАТЕГОРИЯ: ${cat} | СТРОК: ${lines.length} | `);
    push(SUBHR);
    push();
    push(content.replace(/\n+$/, ""));
    push();
    push();
  });

  push(HR);
  push(`КОНЕЦ ДОКУМЕНТА • ${files.length} файлов • сгенерировано`);
  push(HR);

  const txt = out.join("\n") + "\n";
  fs.writeFileSync(TXT_PATH, txt, "utf8");

  console.log("[1/3] код -> txt");
  console.log(`файлов: ${files.length}`);
  console.log(`глав: ${chapters.length}`);
  console.log(`строк: ${txt.split("\n").length}`);
  console.log(`выход: ${path.relative(ROOT, TXT_PATH)}`);
}

main();

```

```
    rows: 76,
  };

export const HR = "=".repeat(PAGE.cols);
export const SUBHR = "-".repeat(PAGE.cols);

export function ensureDir(dir) {
  fs.mkdirSync(dir, { recursive: true });
}

export function humanSize(bytes) {
  if (bytes < 1024) return `${bytes} B`;
  if (bytes < 1024 * 1024) return `${(bytes / 1024).toFixed(2)} KB`;
  return `${(bytes / 1024 / 1024).toFixed(2)} MB`;
}

/** ImageMagick 7 (`magick`) или 6 (`convert`). */
export function imageMagickCmd() {
  for (const cmd of ["magick", "convert"]) {
    try {
      execFileSync(cmd, ["-version"], { stdio: "ignore" });
      return cmd;
    } catch {
      /* пробуем следующий */
    }
  }
  throw new Error(
    "ImageMagick не найден. Установите его: sudo apt-get install imagemagick"
  );
}
```

```

    for (let i = 0; i < source.length; i += PAGE.rows) {
      pages.push(source.slice(i, i + PAGE.rows));
    }
    return pages;
  }

  async function main() {
    if (!fs.existsSync(TXT_PATH)) {
      throw new Error(`Нет ${path.relative(ROOT, TXT_PATH)}`);
    }

    const im = imageMagickCmd();
    const pages = paginate(fs.readFileSync(TXT_PATH, "utf8"));

    fs.rmSync(PAGES_DIR, { recursive: true, force: true });
    ensureDir(PAGES_DIR);

    const total = pages.length;
    const jobs = pages.map((lines, idx) => {
      const num = String(idx + 1).padStart(4, "0");
      const header = `Универсальное пособие • полный исходный код
      Math.max(1, PAGE.cols - 44 - `стр. ${idx + 1} / ${total}`);
    });стр. ${idx + 1} / ${total}`;
    const body = [header, "-".repeat(PAGE.cols), ...lines];
    const txtFile = path.join(PAGES_DIR, `page-${num}.txt`);
    const pngFile = path.join(PAGES_DIR, `page-${num}.png`);
    fs.writeFileSync(txtFile, body + "\n", "utf8");
    return { txtFile, pngFile, num };
  });

  const args = (job) => [
    "-density", String(PAGE.density),
    "-page", PAGE.size,
    "-font", fs.existsSync(PAGE.fontFile) ? PAGE.fontFile : "Helvetica",
    "-pointsize", String(PAGE.pointsize),
    `text:${job.txtFile}`,
    "-background", "white",
    "-flatten",
  ];

```

```
});
```

РАЗДЕЛ: CI / АВТОМАТИЗАЦИЯ

ФАЙЛ 82/82: .github/workflows/rebuild-pdf.yml

КАТЕГОРИЯ: CI / автоматизация | СТРӨК: 128 | РАЗМЕР: 4.1 КБ

name: Rebuild code PDF

Пайплайн пересборки PDF при каждом коммите:

```
#
#   исходный код  → full_code_all_pages.txt   (script)
#               |
#               → page-0001.png ... page-NNNN.png (artifacts)
#                               |
#                               → full_code_all_page.pdf (artifacts)
#
# Готовые TXT и PDF коммитаются обратно в ветку (их отдаёт GitHub)
# промежуточные PNG сохраняются как artifacts запуска.
```

on:

push:

branches:

- "**"

paths-ignore:

чтобы коммит самого workflow не запускал бесконечно

- "public/export/**"

- "**/*.md"

workflow_dispatch:

inputs:

density:

description: "DPI растеризации страниц (по умолчанию 300)"

required: false

- ```
 fi
 done
 convert -version
```
- name: Install dependencies  
run: npm ci --no-audit --no-fund
  - name: Step 1 – код → txt  
run: npm run export:txt
  - name: Step 2 – txt → png  
env:  
 EXPORT\_DENSITY: \${github.event.inputs.density}
run: npm run export:png
  - name: Step 3 – png → pdf  
run: npm run export:pdf
  - name: Type-check приложения  
run: npx tsc --noEmit  
continue-on-error: true
  - name: Summary  
run: |  
 {  
 echo "### Экспорт пересобран"  
 echo ""  
 echo "| Артефакт | Размер |"  
 echo "| --- | --- |"  
 echo "| \`public/export/full\_code\_all\_pages`  
 echo "| \`public/export/full\_code\_all\_pages`  
 echo "| PNG-страниц | \$(ls tmp/export-pages  
 } >> "\$GITHUB\_STEP\_SUMMARY"
  - name: Upload artifacts (PDF + TXT)  
uses: actions/upload-artifact@v4  
with:  
 name: full-code-export